go to 109

Priority Queue and Heap

PRIORITY QUEUE

Priority Queue

- Normal Queue: (Consider the values as age)

- Priority Queue:

Priority Queue

- Priority Queue: (Priority given to the seniors)

Priority Queue

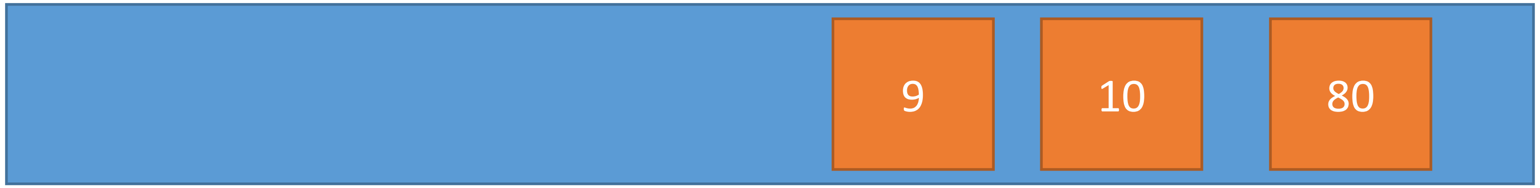
- Priority Queue: (Priority given to the seniors)

Priority Queue

- Priority Queue: (Priority given to the seniors)

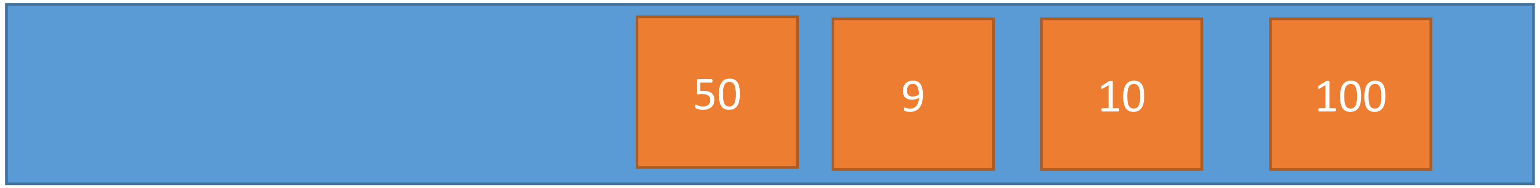
Priority Queue

- Priority Queue: (Priority given to the seniors)



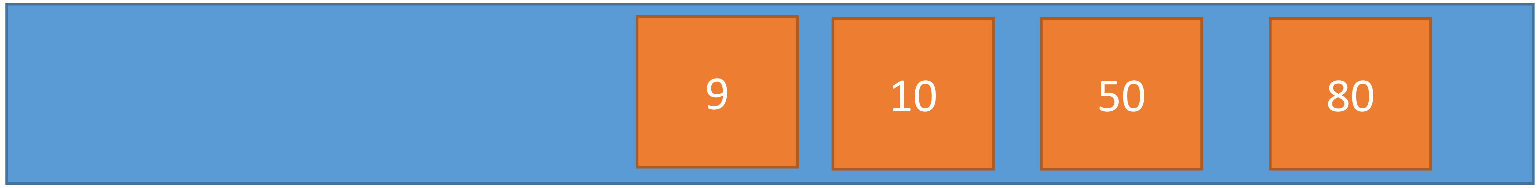
Priority Queue

- Priority Queue: (Priority given to the seniors)



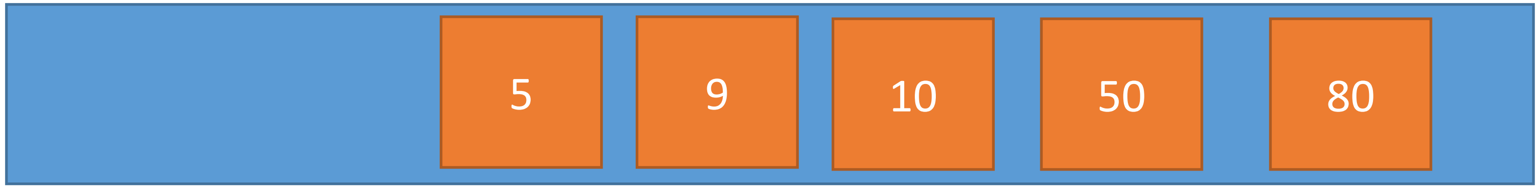
Priority Queue

- Priority Queue: (Priority given to the seniors)



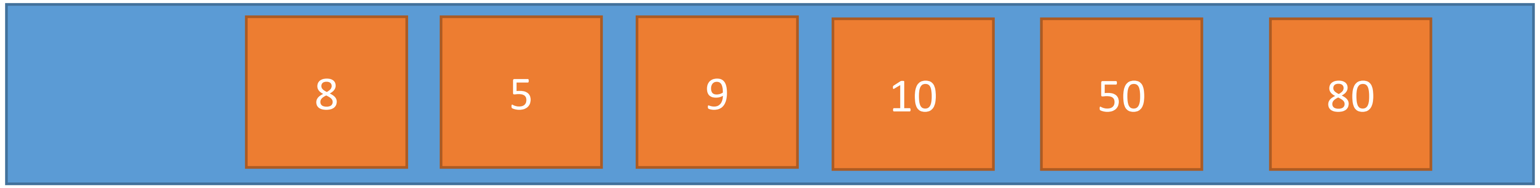
Priority Queue

- Priority Queue: (Priority given to the seniors)



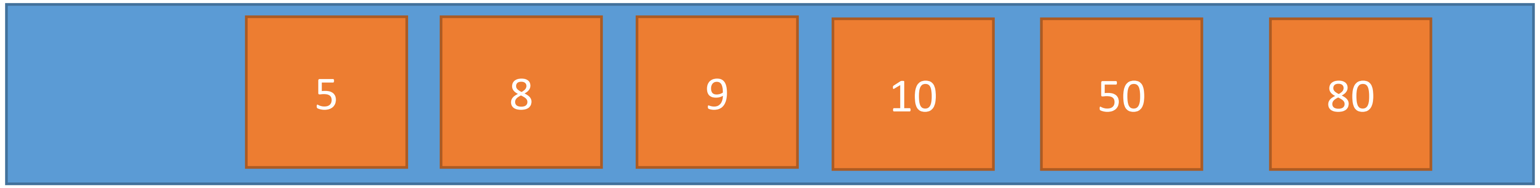
Priority Queue

- Priority Queue: (Priority given to the seniors)



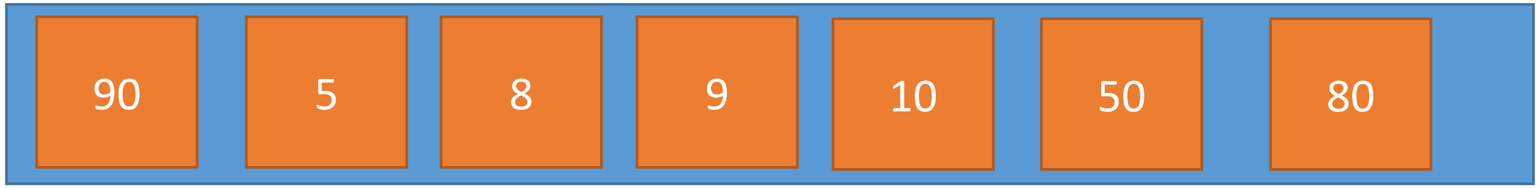
Priority Queue

- Priority Queue: (Priority given to the seniors)



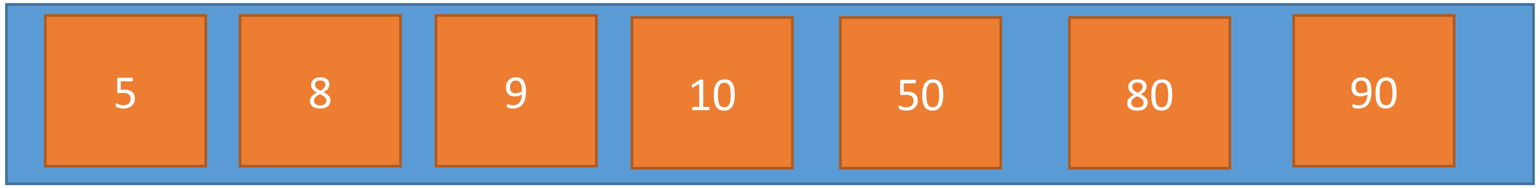
Priority Queue

- Priority Queue: (Priority given to the seniors)



Priority Queue

- Priority Queue: (Priority given to the seniors)



Applications of Priority Queue

- Line-up of Incoming Planes at Airport
  - VIP Planes
  - Commercial Planes
  - Emergency Planes

- Operating Systems Priority Queues
  - Alarm clock app
  - Messenger app

BINARY HEAP

Binary Heaps

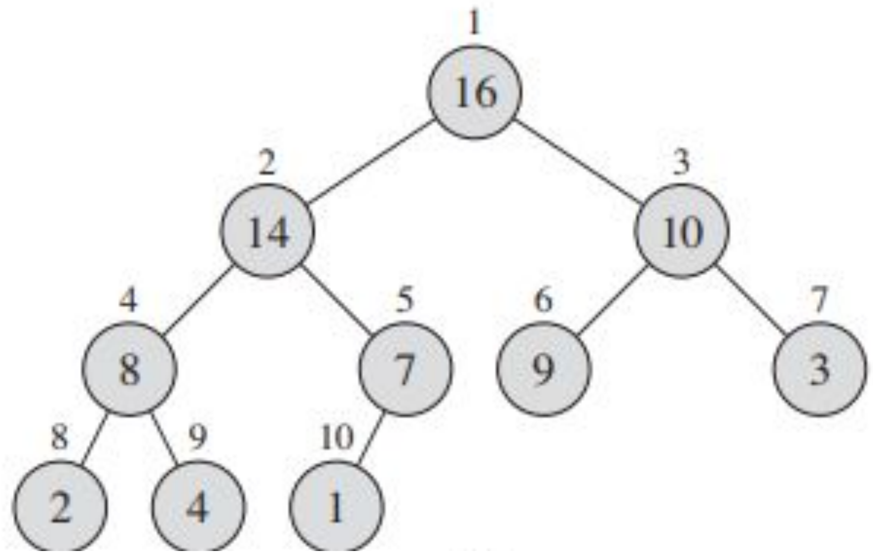
- The (binary) heap data structure is an **array object** that can be viewed as a **complete binary tree.**
  - Each node of the tree corresponds to an element of the array that stores the value in the node.
  - The tree is completely filled on all levels except possibly the lowest, where it is filled from the left up to a point.

- Despite the fact that it is in fact an **array.**
- We will **imagine** the entire data structure to be a **tree.**

Binary Heaps

- How can we visualize this array as a complete binary tree?

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



Binary Heaps

- Some **basic** variables and functions of a heap:
  - A.heap-size
  - A.length
  - PARENT(i)
  - LEFT(i)
  - RIGHT(i)
- Let us observe the array again

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

Binary Heaps

- Question: Who is the parent of the 7th array element?

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

PARENT(i)

1 **return** $\lfloor i/2 \rfloor$

LEFT(i)

1 **return** $2i$

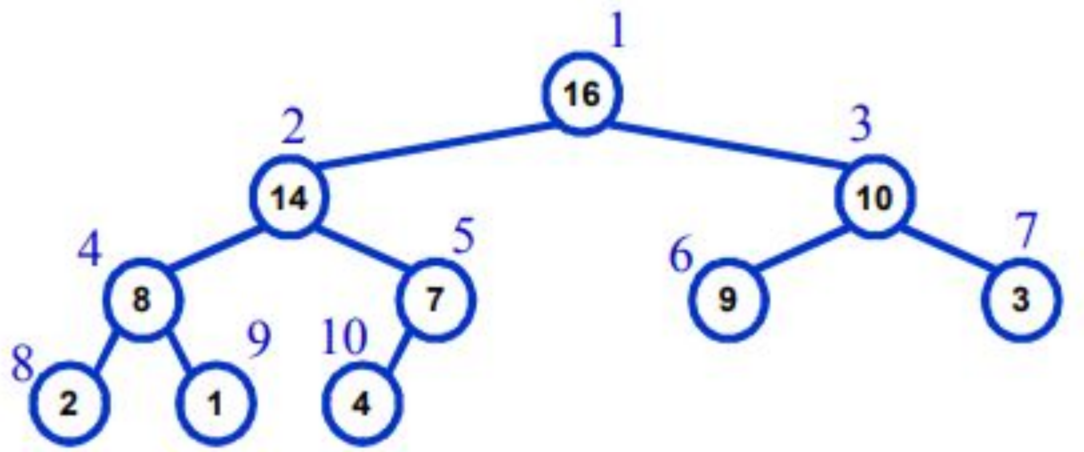
RIGHT(i)

1 **return** $2i + 1$

- Answer:
  - We can draw the tree and find out
  - We can use these functions and determine directly

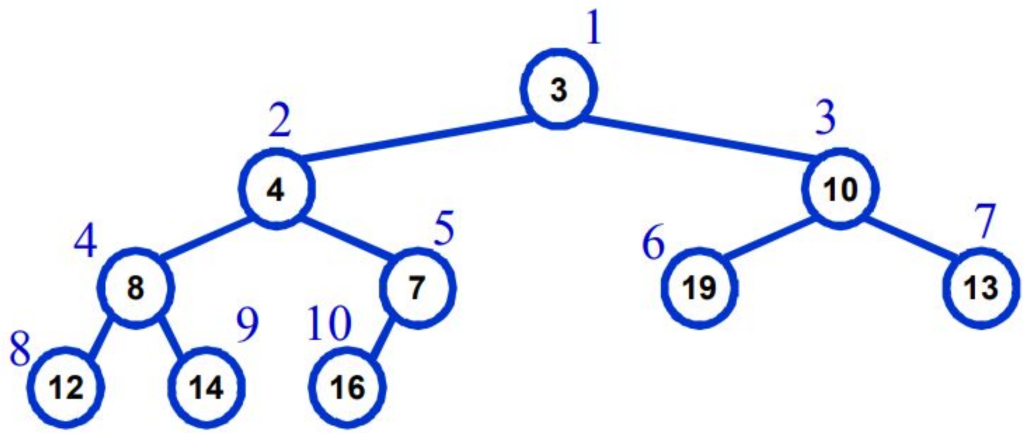
Max Heap and Min Heap

- Max-Heaps
  - Satisfies the heap property: **A[Parent(i)] >= A[i]** for all nodes i > 1
    - The value of a node is at most the value of its parent
  - The **largest element** in a max-heap is stored at the **root**
  - Where is the **smallest element** ?
    - Ans: At one of the **leaves** [leaf indices are n/2+1 to n]



Max Heap and Min Heap

- Min-Heaps
  - Satisfies the heap property: **A[Parent(i)] <= A[i]** for all nodes i > 1
    - The value of a node is at least the value of its parent
  - The **smallest element** in a max-heap is stored at the **root**
  - Where is the **largest** element ?
    - Ans: At one of the leaves [leaf indices are n/2+1 to n]

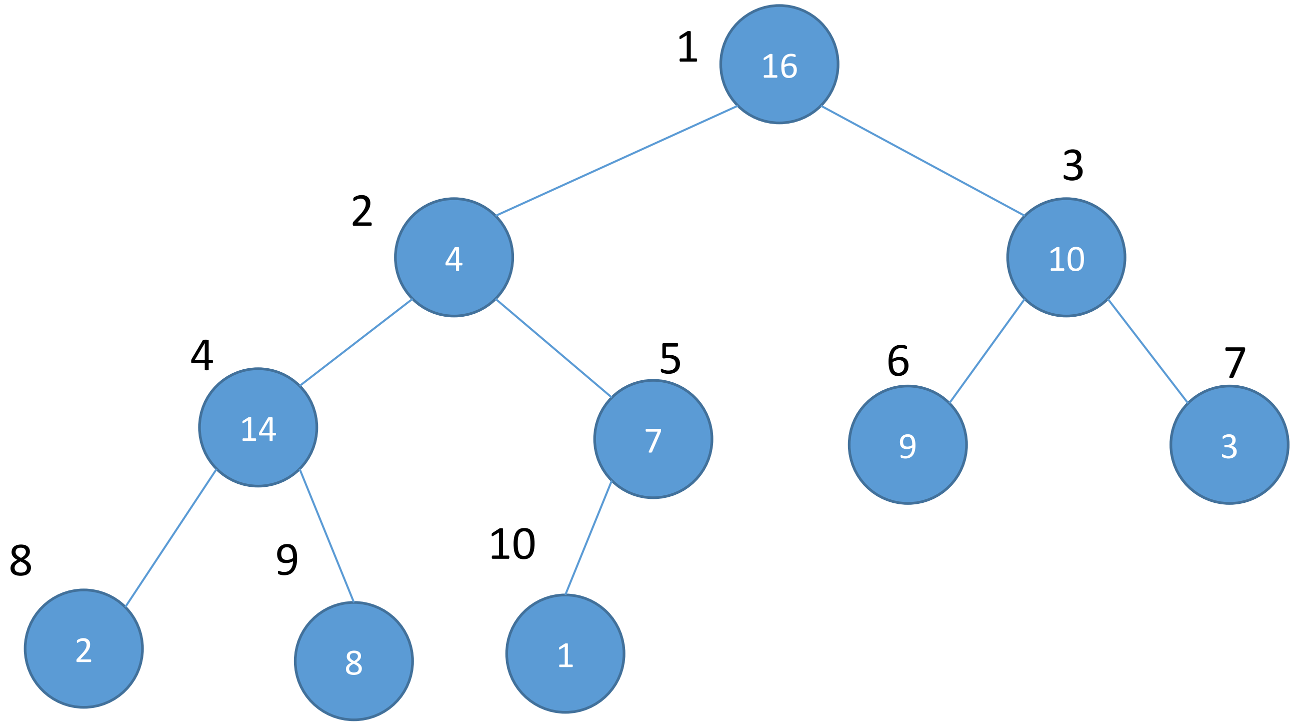


Important Functions of Max Heap

- **MAX-HEAPIFY** –
  - **O(lg n)** time
  - **Tries** to make the array into a heap
- **BUILD-MAX-HEAP**
  - **O(n)** time
  - Converts an array into a heap
- **HEAPSORT**
  - O **(n lg n)** time
  - **Another sorting algorithm**
  - Makes use of the concept of heap
- **Priority Queue functions**
  - **MAX-HEAP-INSERT**
  - **HEAP-EXTRACT-MAX**
  - **HEAP-INCREASE-KEY**
  - **HEAP-MAXIMUM**
  - run in O(lg n) time

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

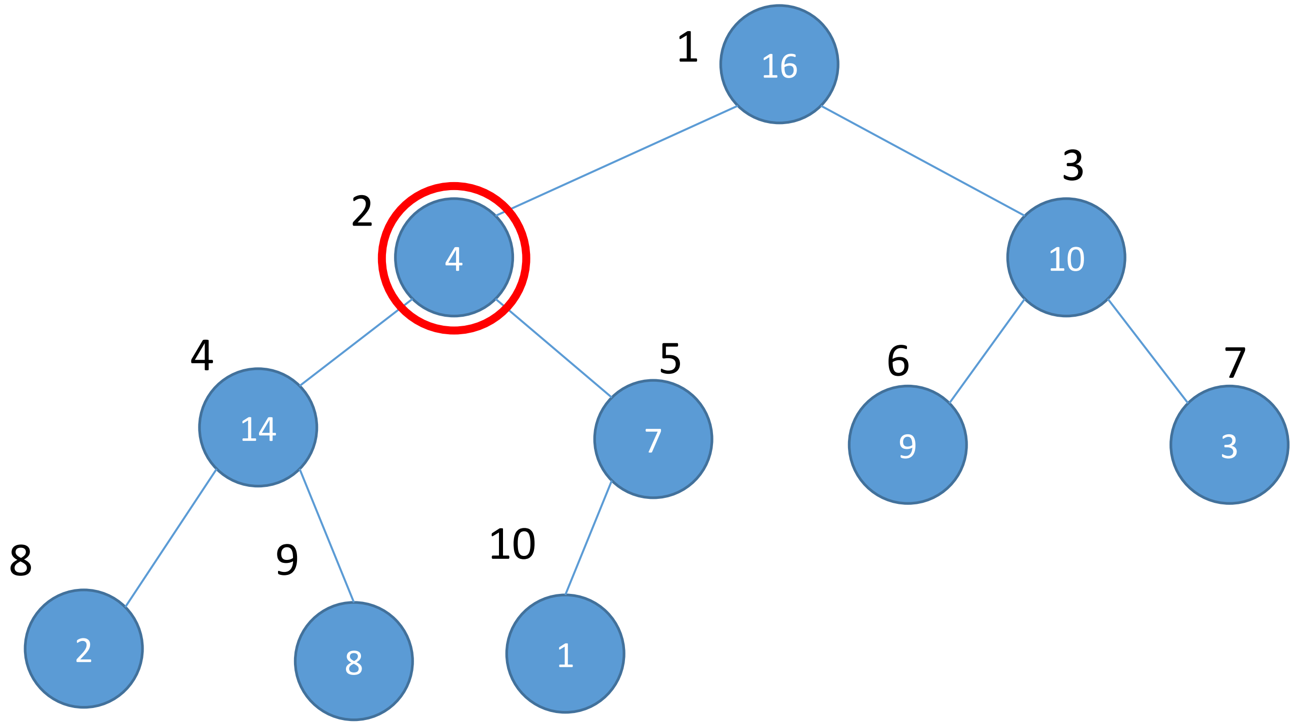


1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

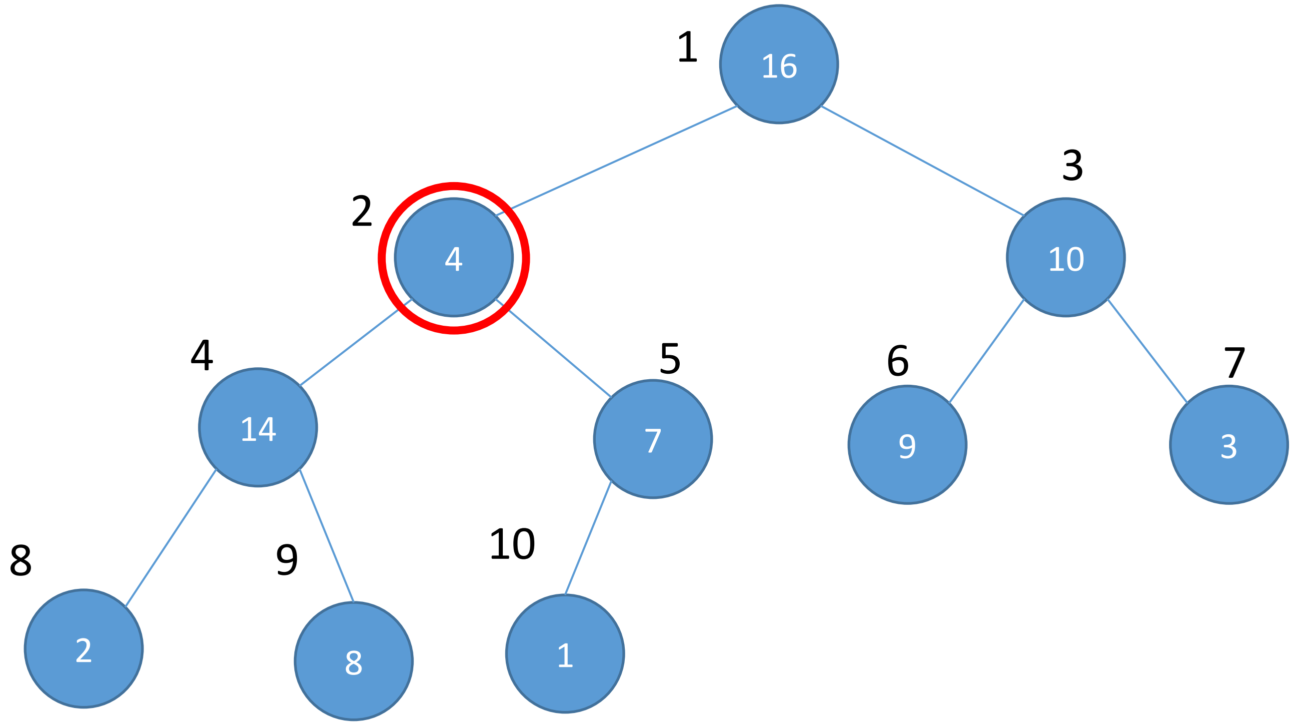
1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1



MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1



<u>How the function works:</u>

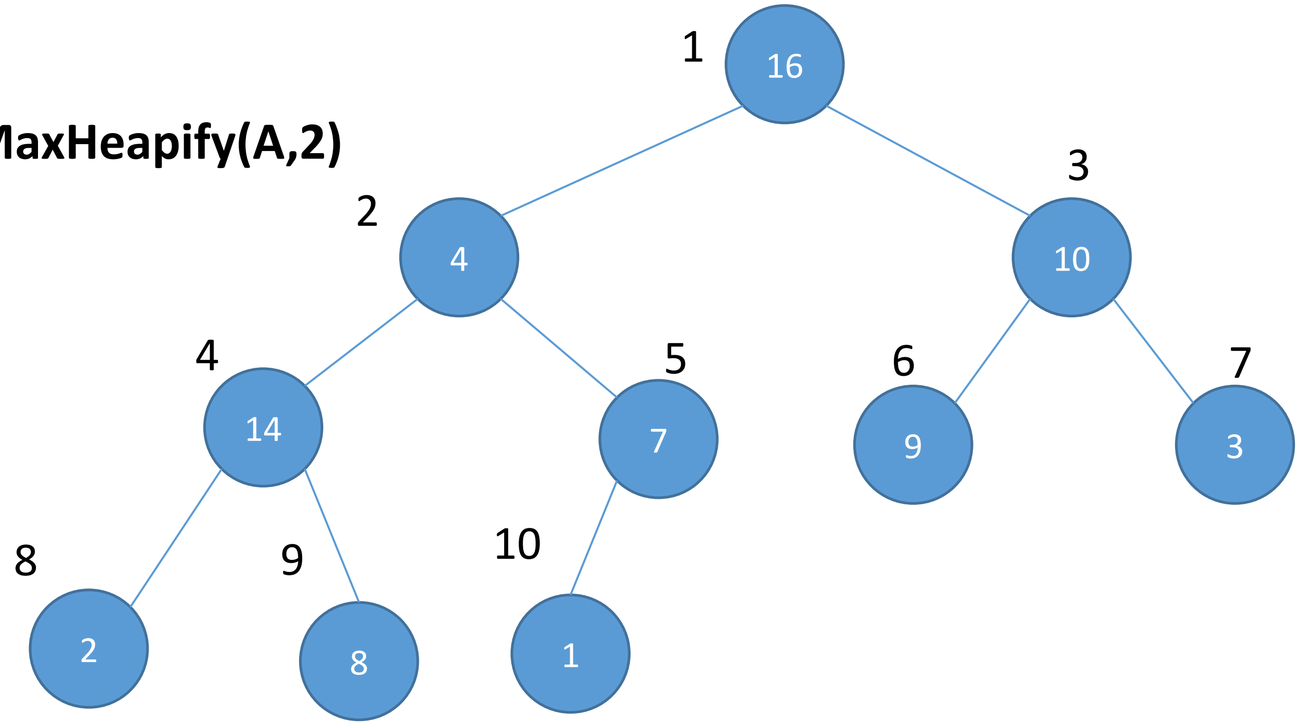
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MaxHeapify(A,2)



<u>How the function works:</u>

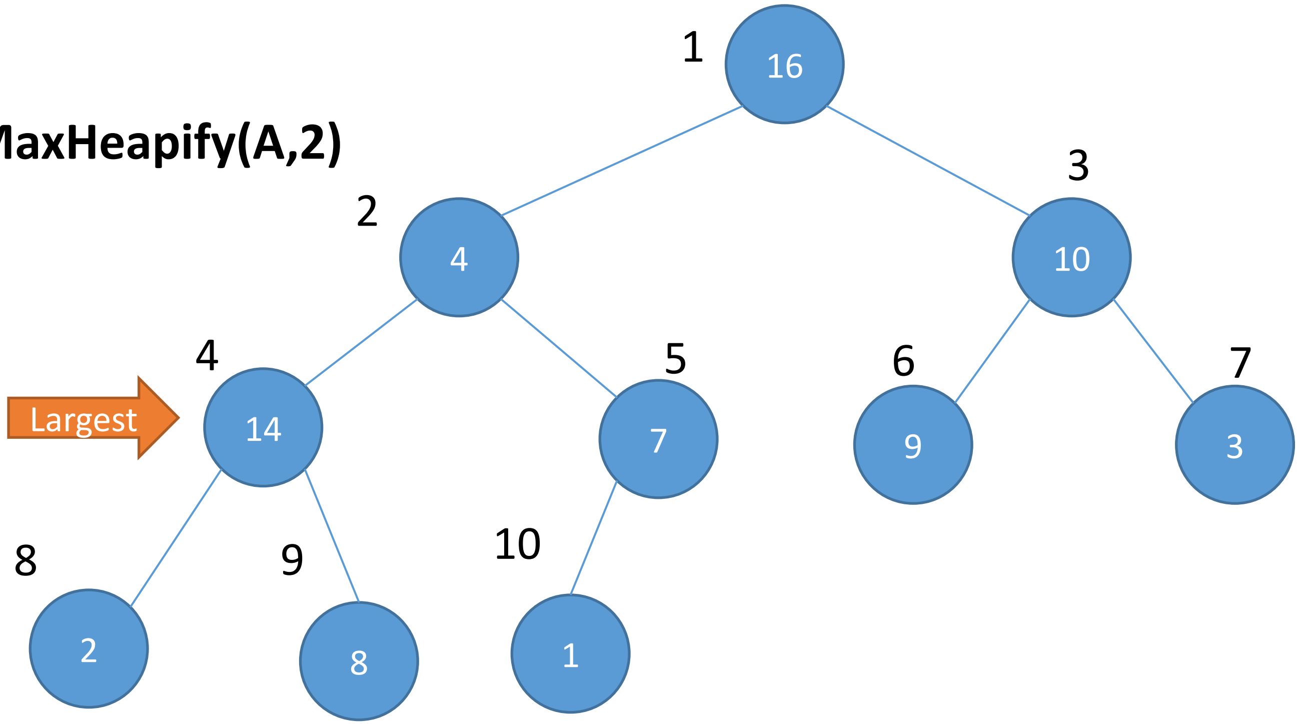
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MaxHeapify(A,2)



<u>How the function works:</u>

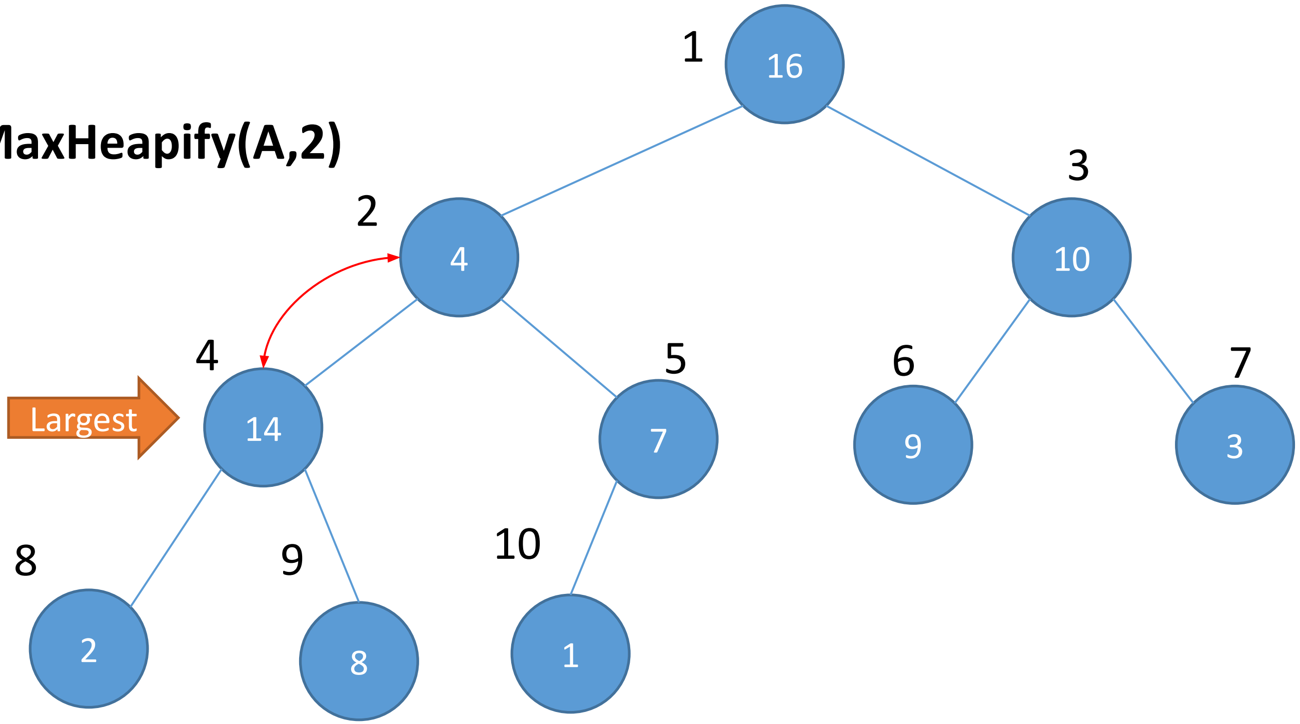
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
16	4	10	14	7	9	3	2	8	1

MaxHeapify(A,2)



<u>How the function works:</u>

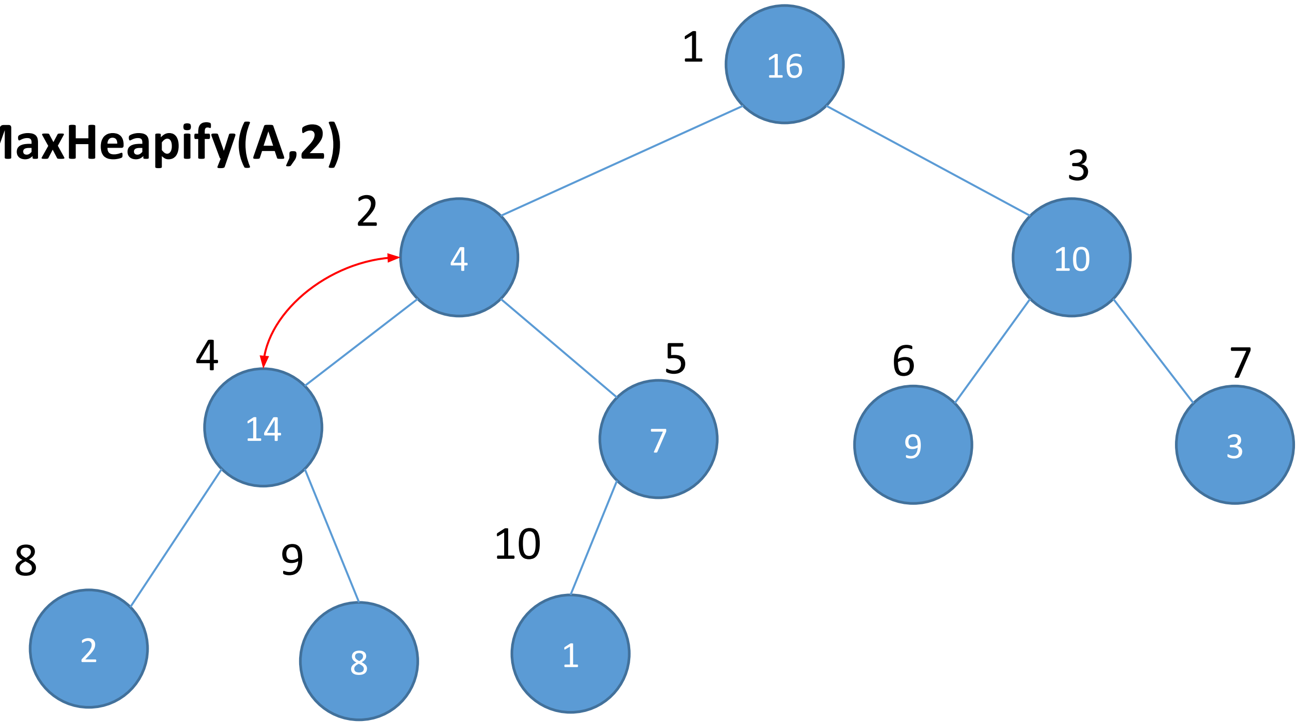
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	4	10	14	7	9	3	2	8	1

MaxHeapify(A,2)



<u>How the function works:</u>

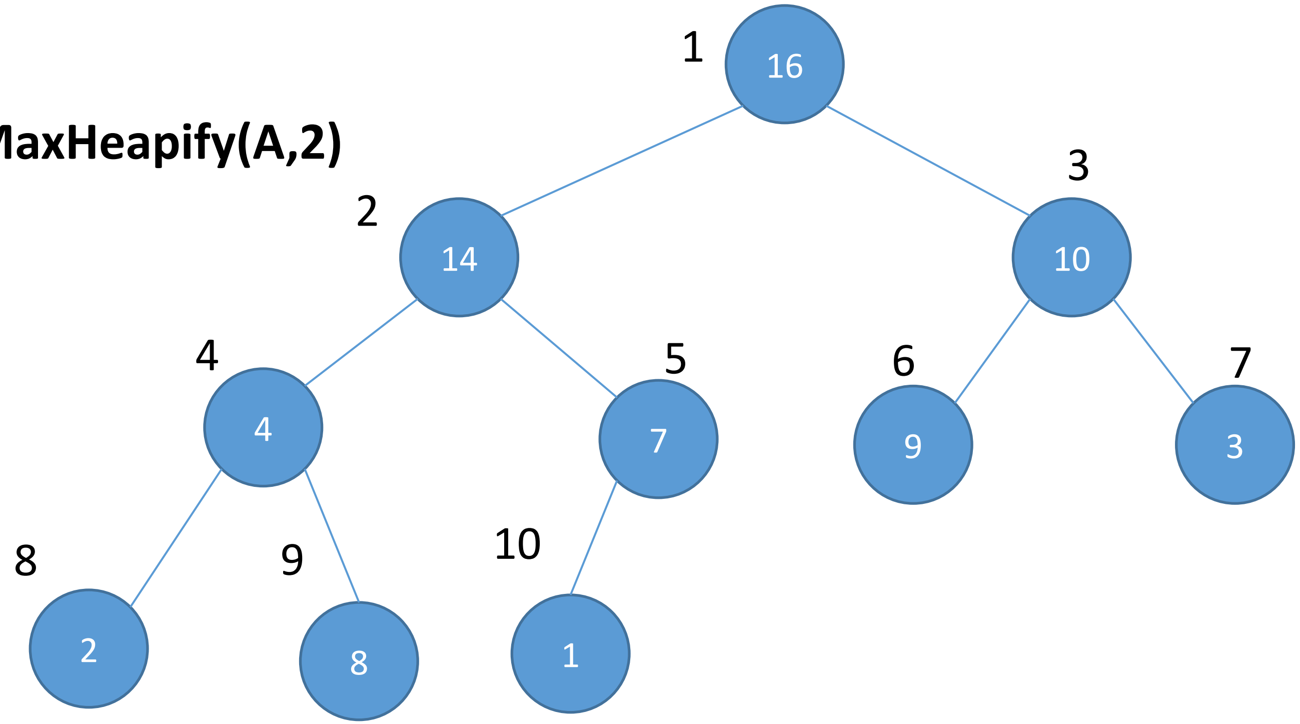
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**4**	7	9	3	2	8	1

MaxHeapify(A,2)



<u>How the function works:</u>

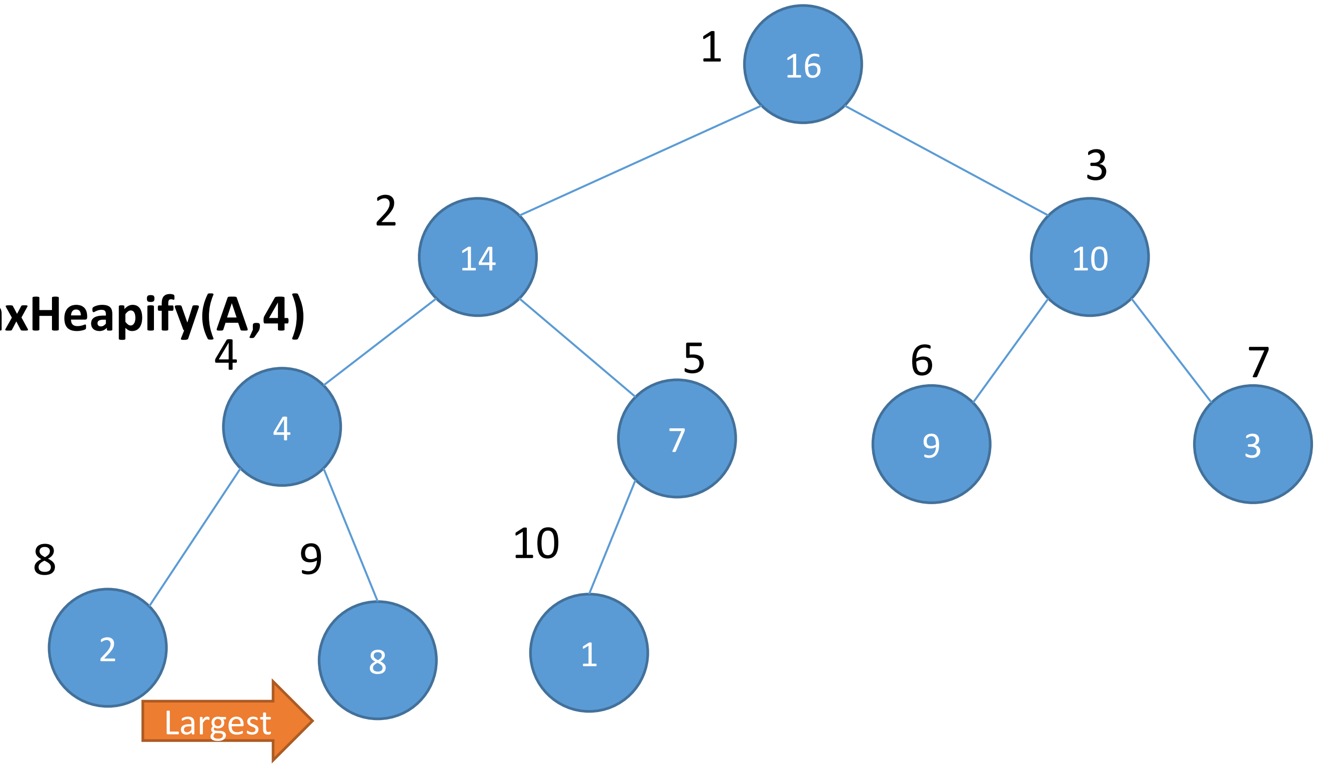
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**4**	7	9	3	2	8	1

MaxHeapify(A,4)



<u>How the function works:</u>

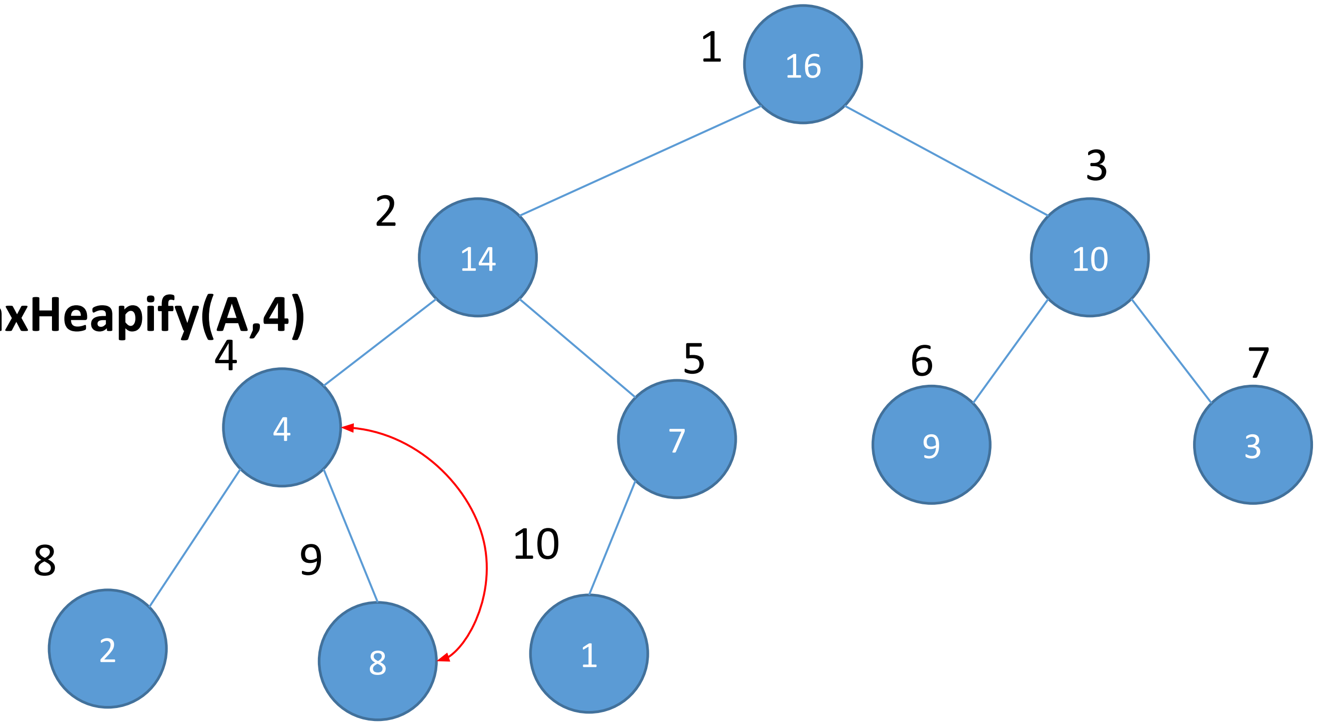
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**4**	7	9	3	2	8	1

MaxHeapify(A,4)



How the function works:

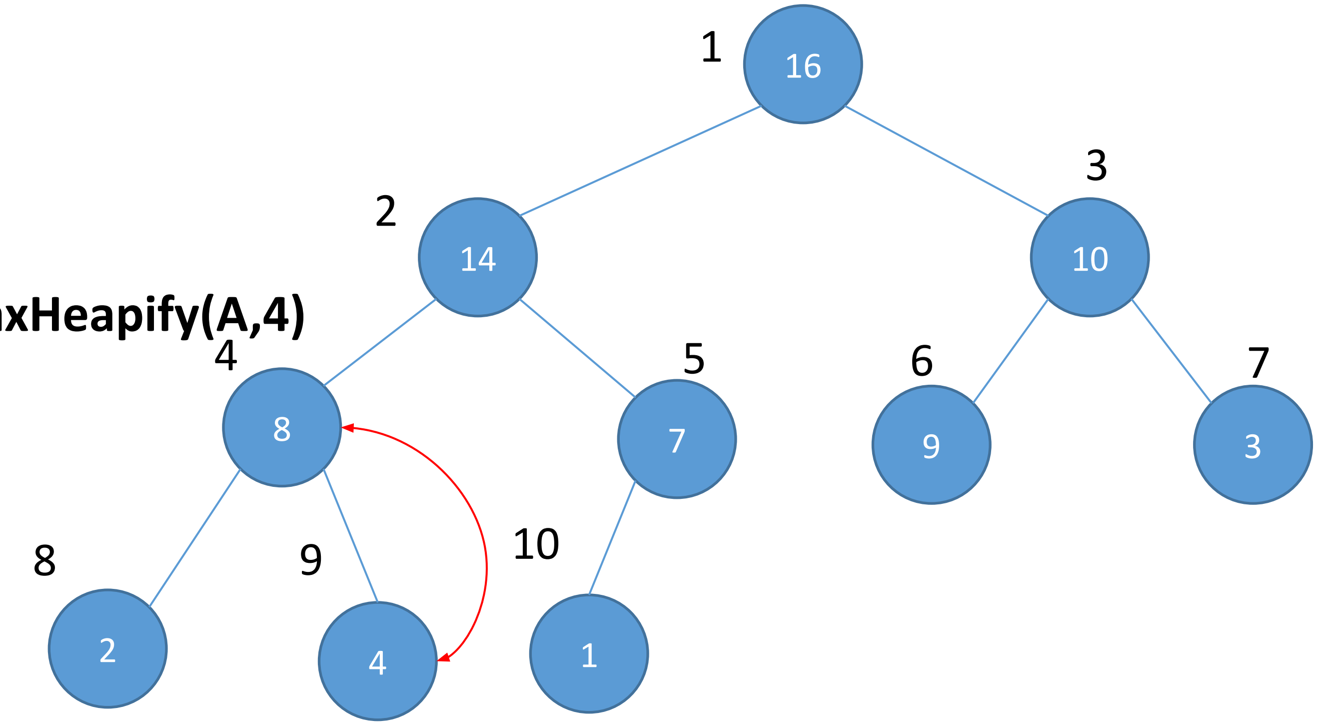
1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
16	**14**	10	**8**	7	9	3	2	**4**	1

MaxHeapify(A,4)



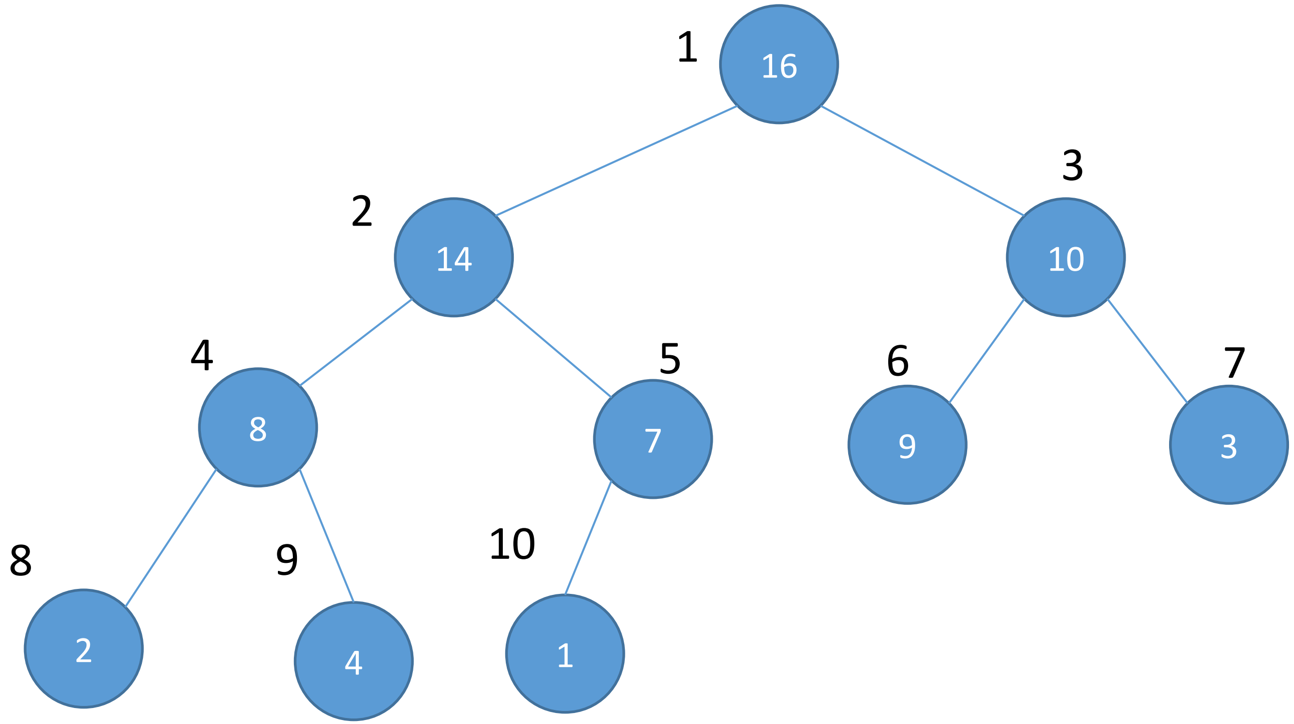
<u>How the function works:</u>

1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

MAX HEAPIFY

- **Tries** to make the array into a max heap
- (parent value>= child value)
- MaxHeapify(A,index)

1	2	3	4	5	6	7	8	9	10
16	**14**	10	**8**	7	9	3	2	**4**	1



MaxHeapify(A,9)

<u>How the function works:</u>

1. It is applied on an index.
2. It checks if the parent is greater than the child
3. If the parent is greater, return.
4. If the child is greater, then swap values with child
   a) Run the function on the child (recursively)

Let us write the function for Max Heapify

Let us write the function for Max Heapify

```
MAX-HEAPIFY(A, i)
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
        largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
        largest = r
if largest ≠ i
        exchange A[i] with A[largest]
        MAX-HEAPIFY(A, largest)
```

Max Heapify

- It is TRYING to make the array into a heap
- But it may not fully succeed
- Let us consider the following example

Max Heapify

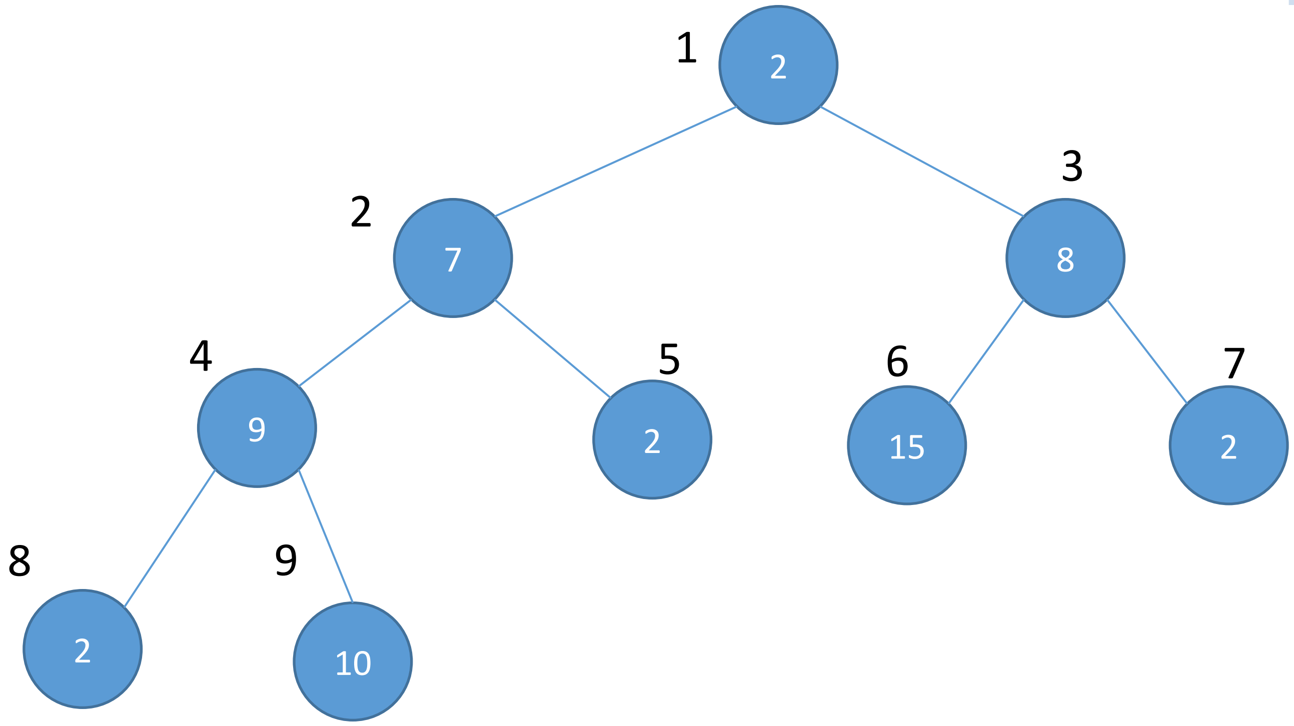
- It is TRYING to make the array into a heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
2	7	8	9	2	15	2	2	10

Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
2	7	8	9	2	15	2	2	10

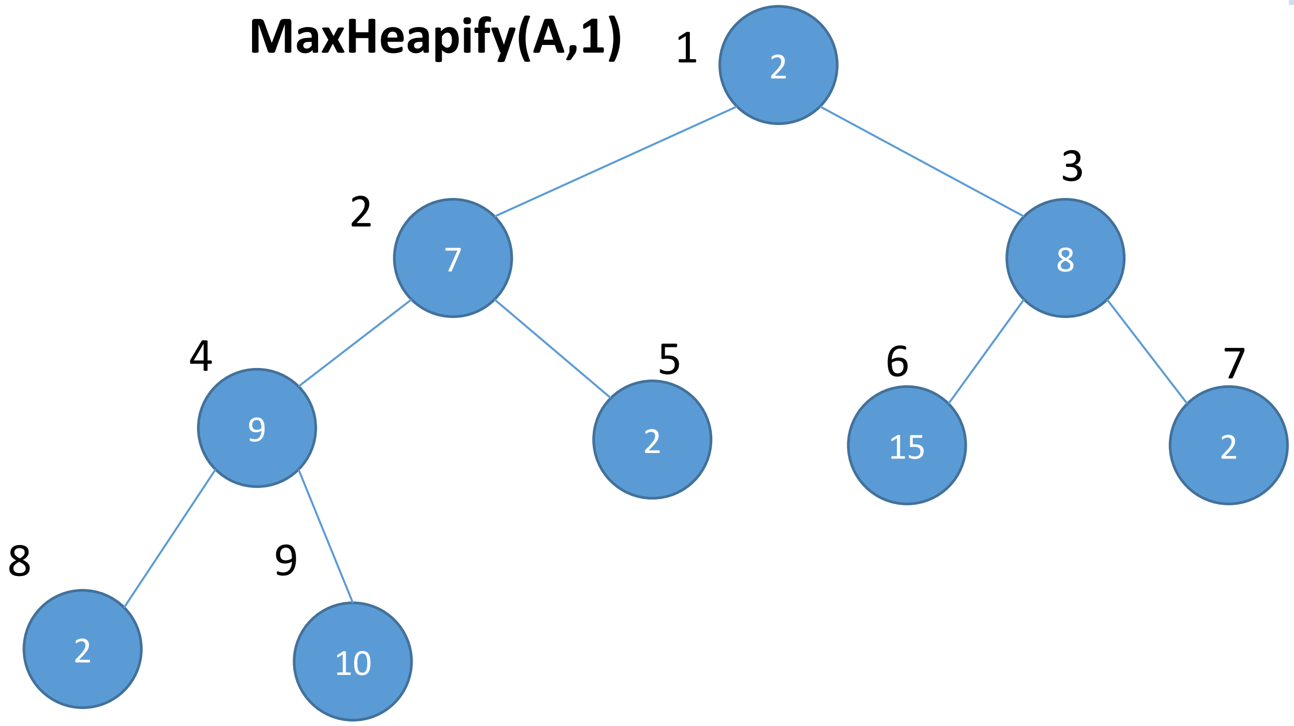


Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
2	7	8	9	2	15	2	2	10

MaxHeapify(A,1)

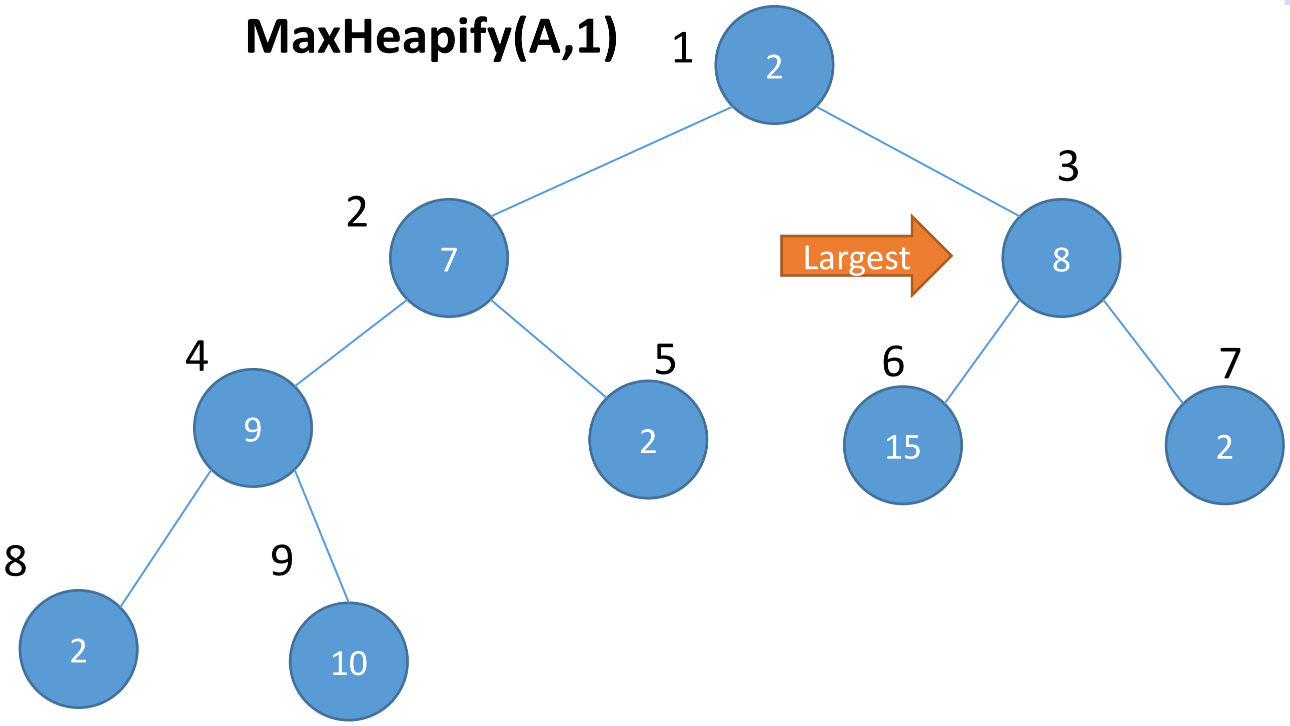


Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
2	7	8	9	2	15	2	2	10

MaxHeapify(A,1)

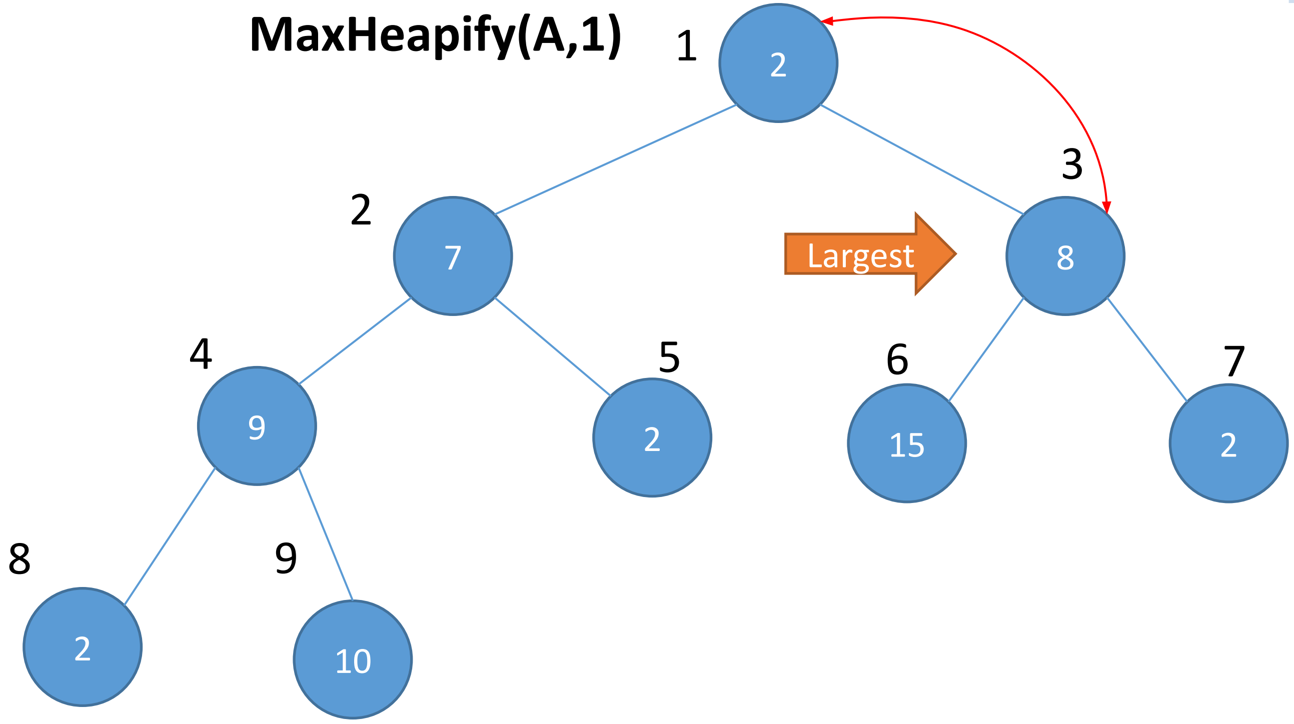


Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
2	7	8	9	2	15	2	2	10

MaxHeapify(A,1)

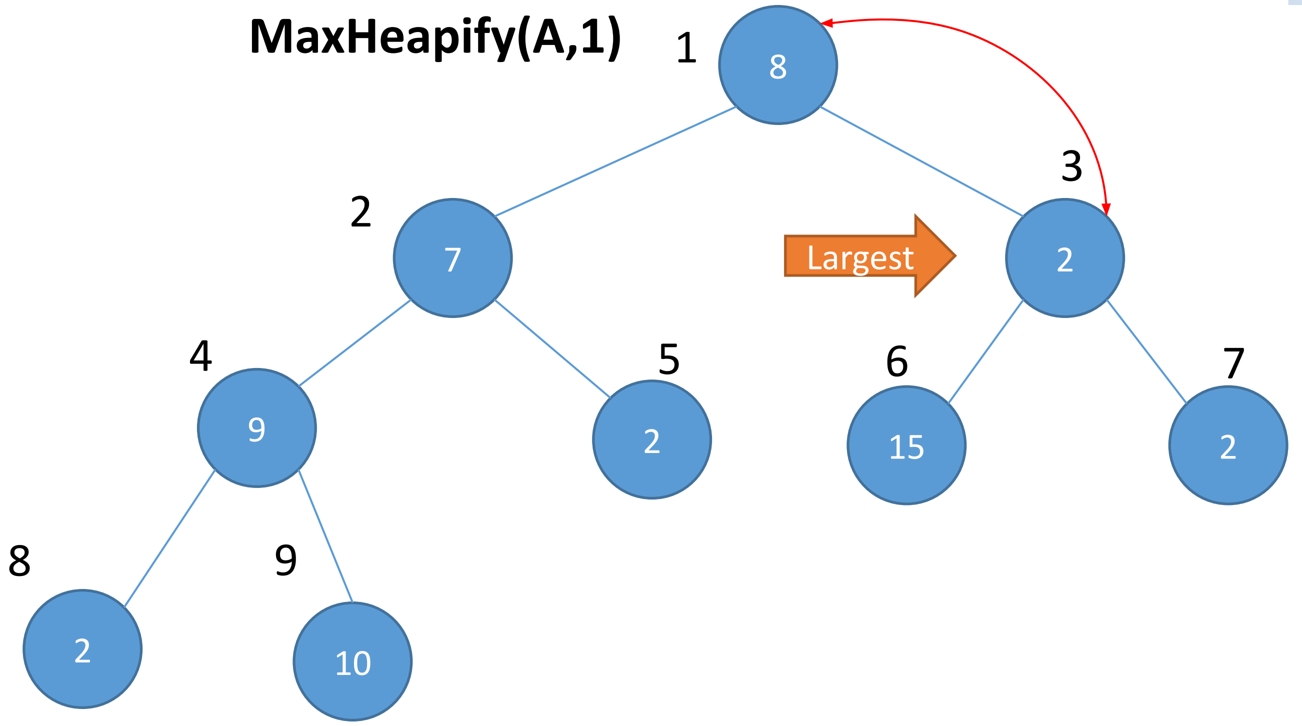


Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
8	7	2	9	2	15	2	2	10

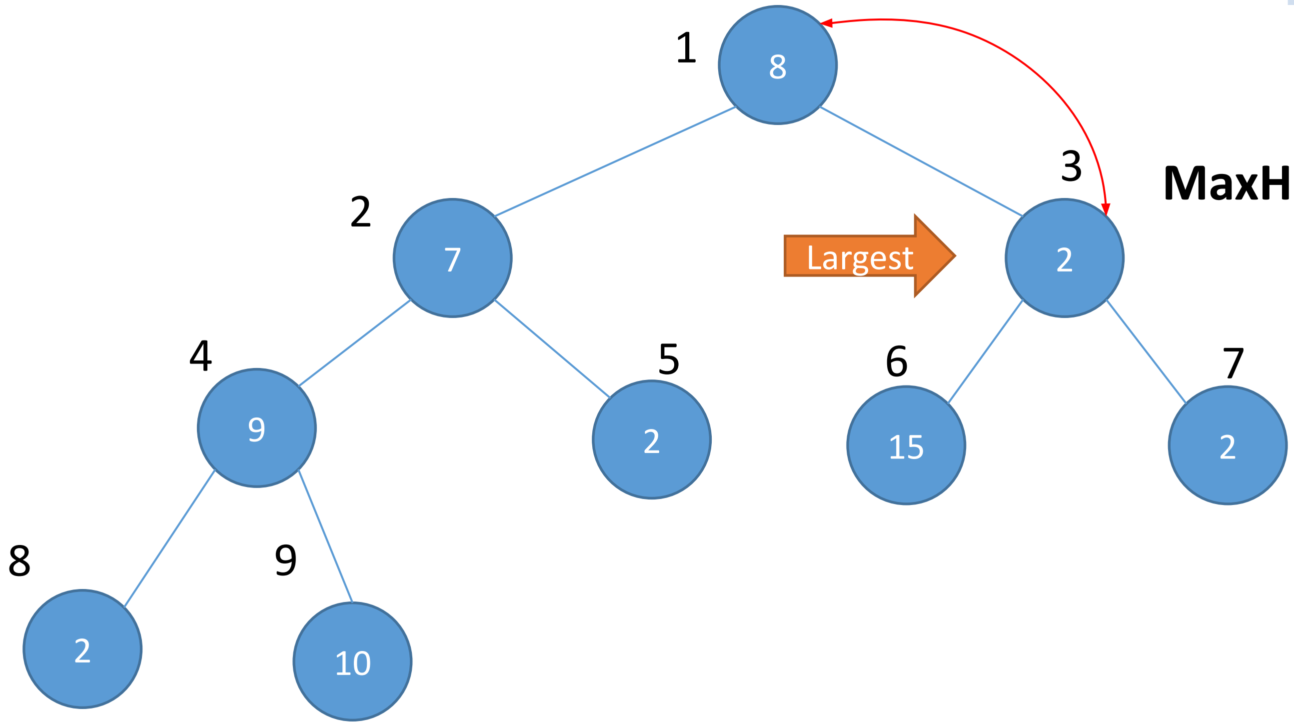
MaxHeapify(A,1)



Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
8	7	2	9	2	15	2	2	10

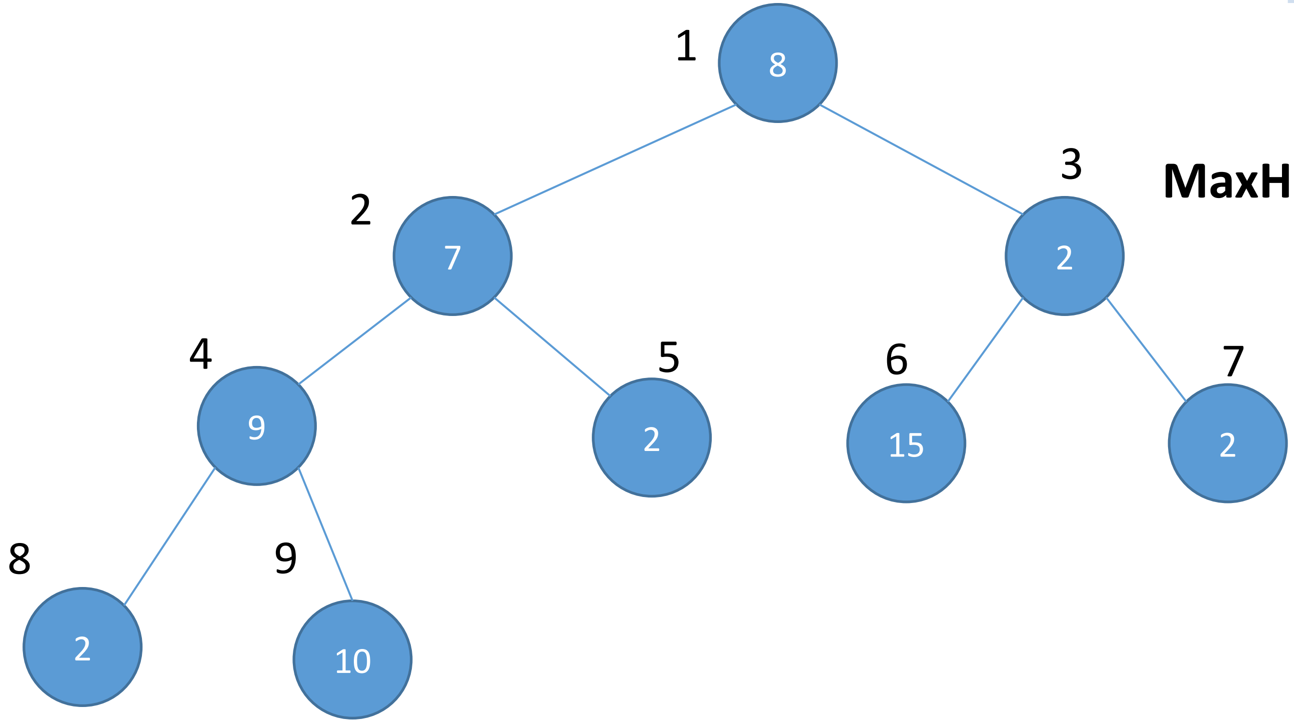


MaxHeapify(A,3)

Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
8	7	2	9	2	15	2	2	10

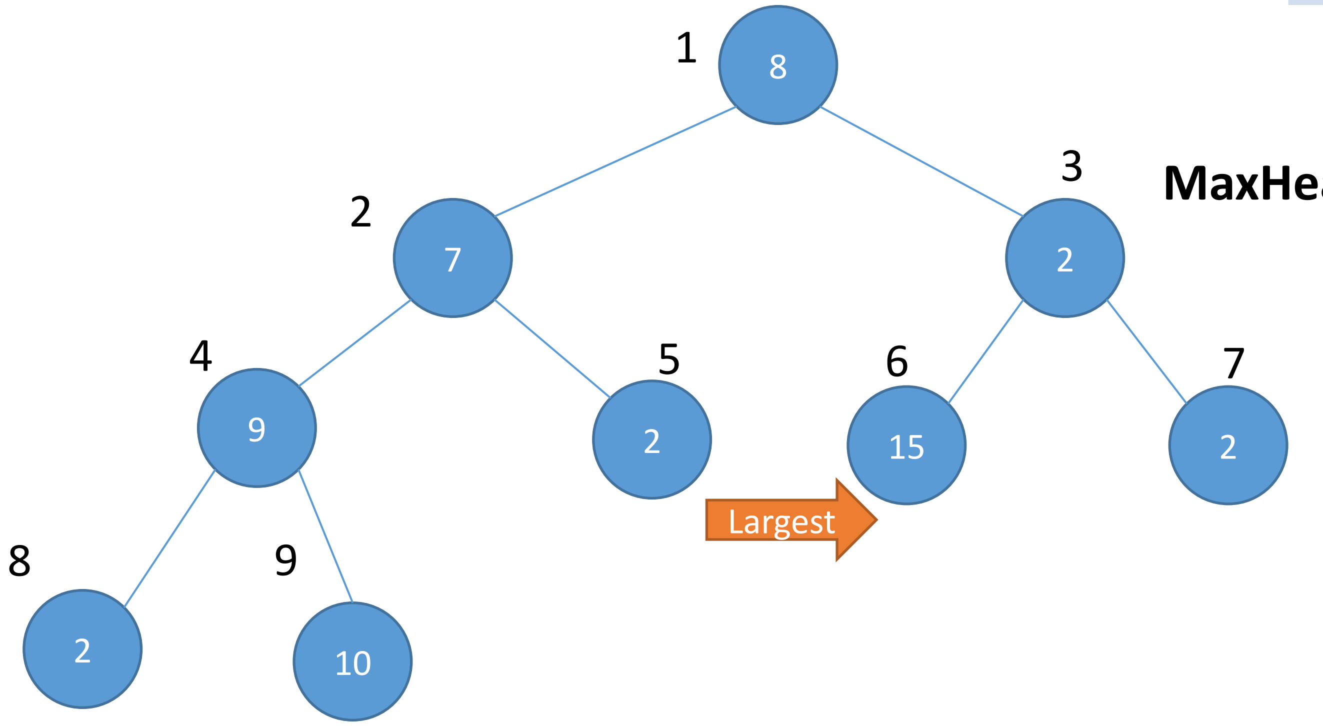


MaxHeapify(A,3)

Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
8	7	2	9	2	15	2	2	10

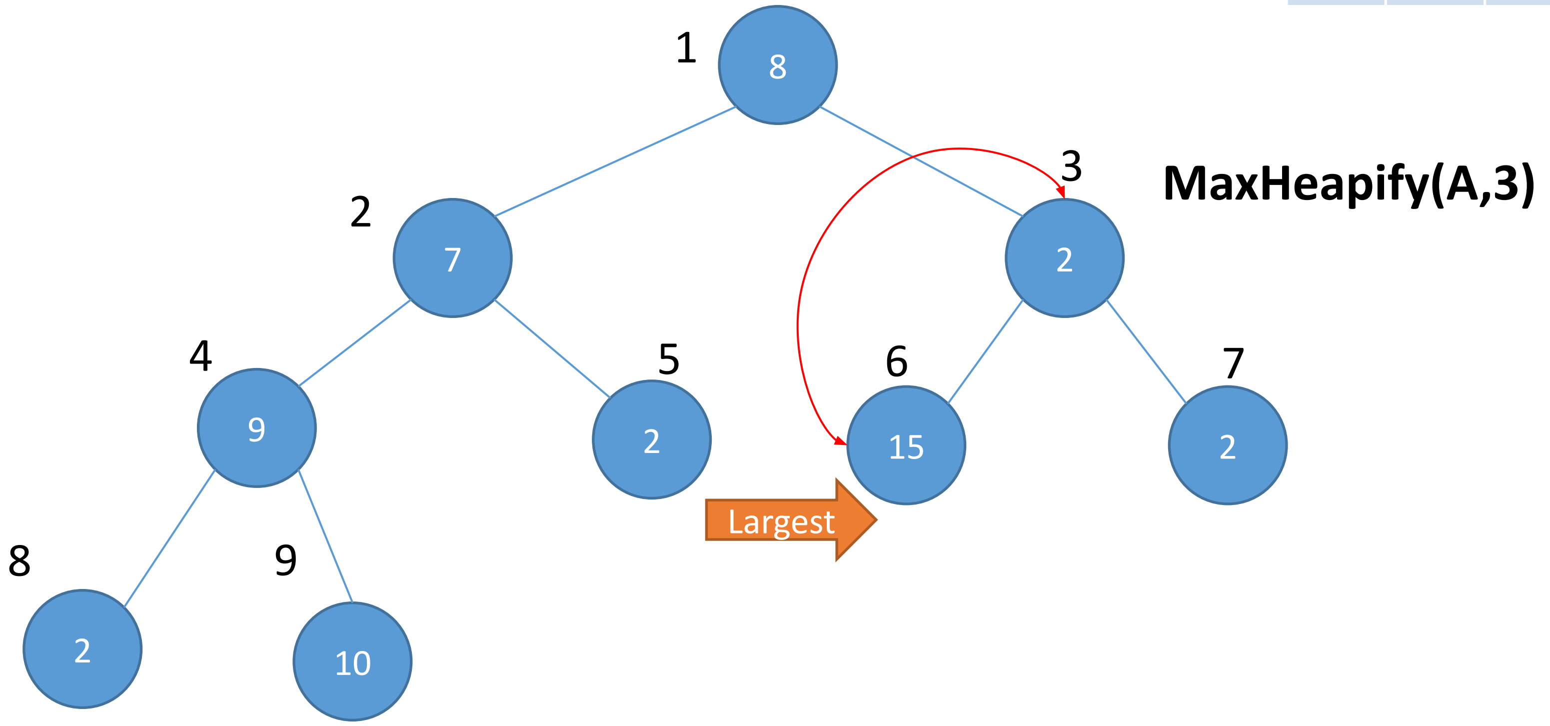


MaxHeapify(A,3)

Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

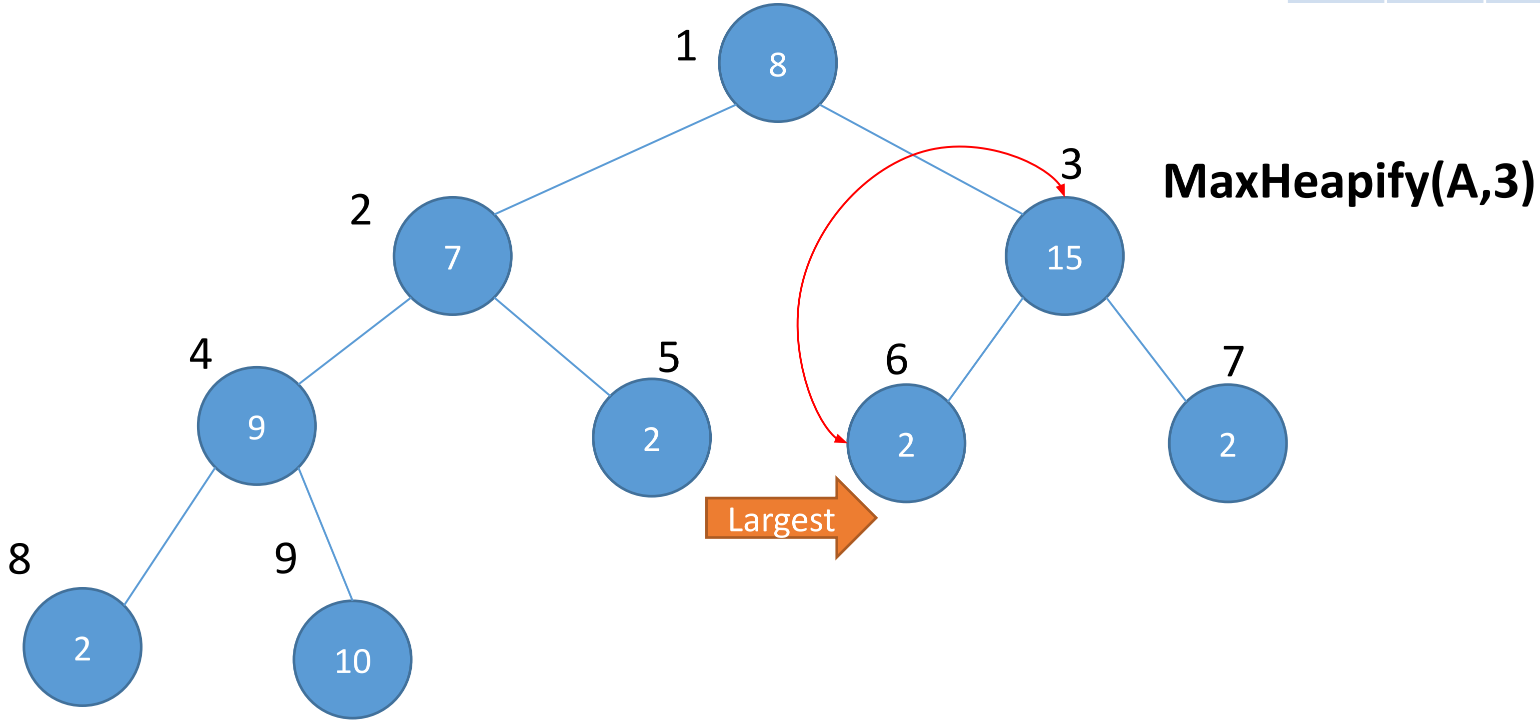
1	2	3	4	5	6	7	8	9
8	7	2	9	2	15	2	2	10



Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

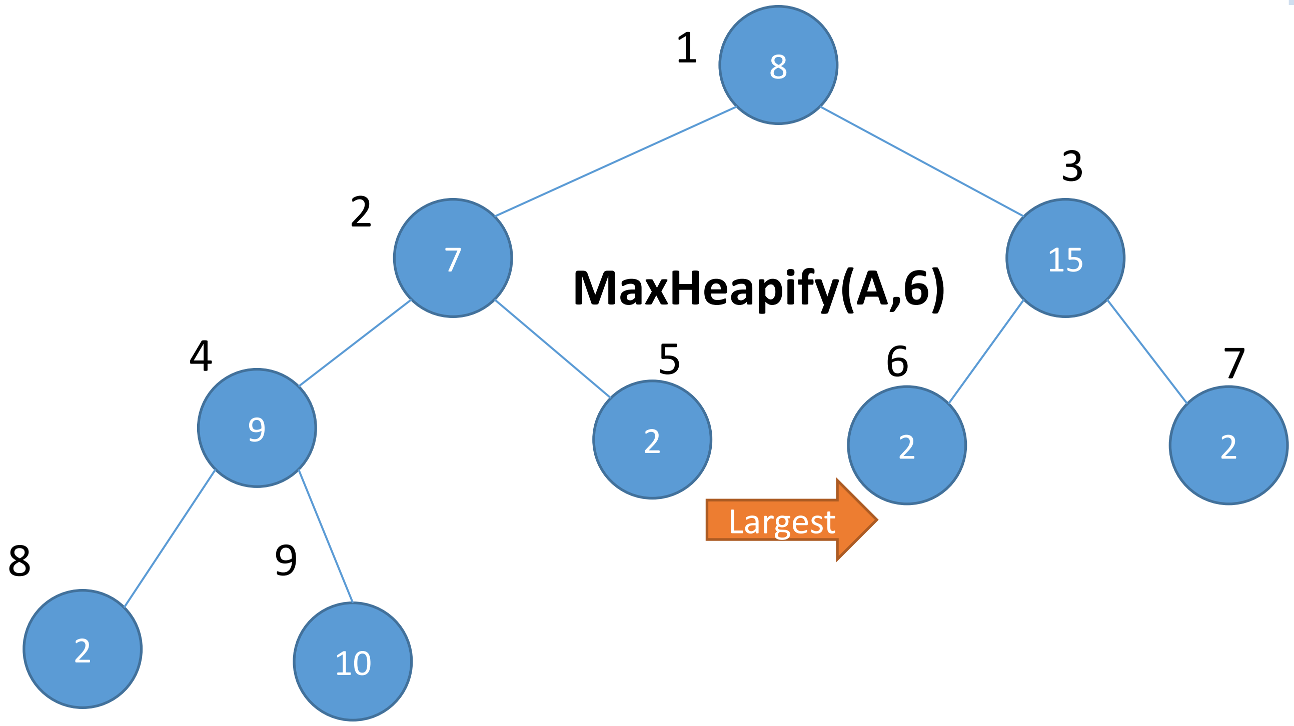
1	2	3	4	5	6	7	8	9
8	7	15	9	2	2	2	2	10



Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

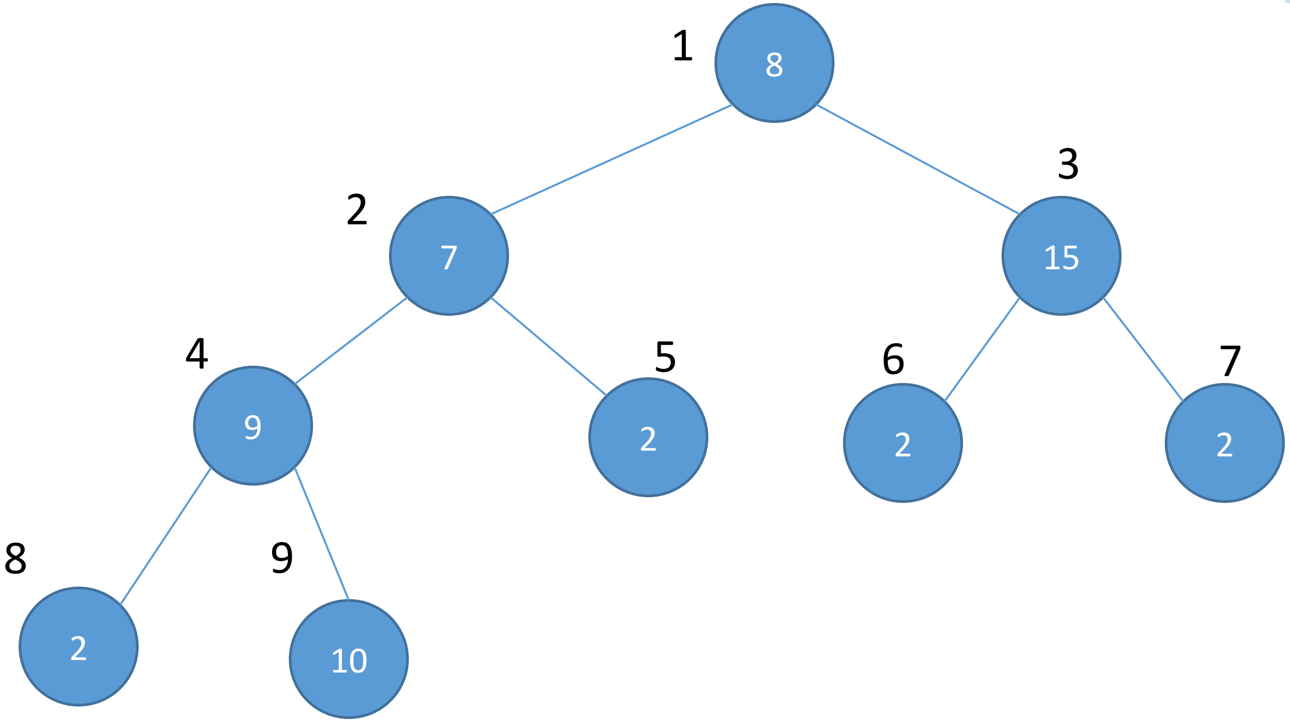
1	2	3	4	5	6	7	8	9
8	7	15	9	2	2	2	2	10



Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
8	7	15	9	2	2	2	2	10

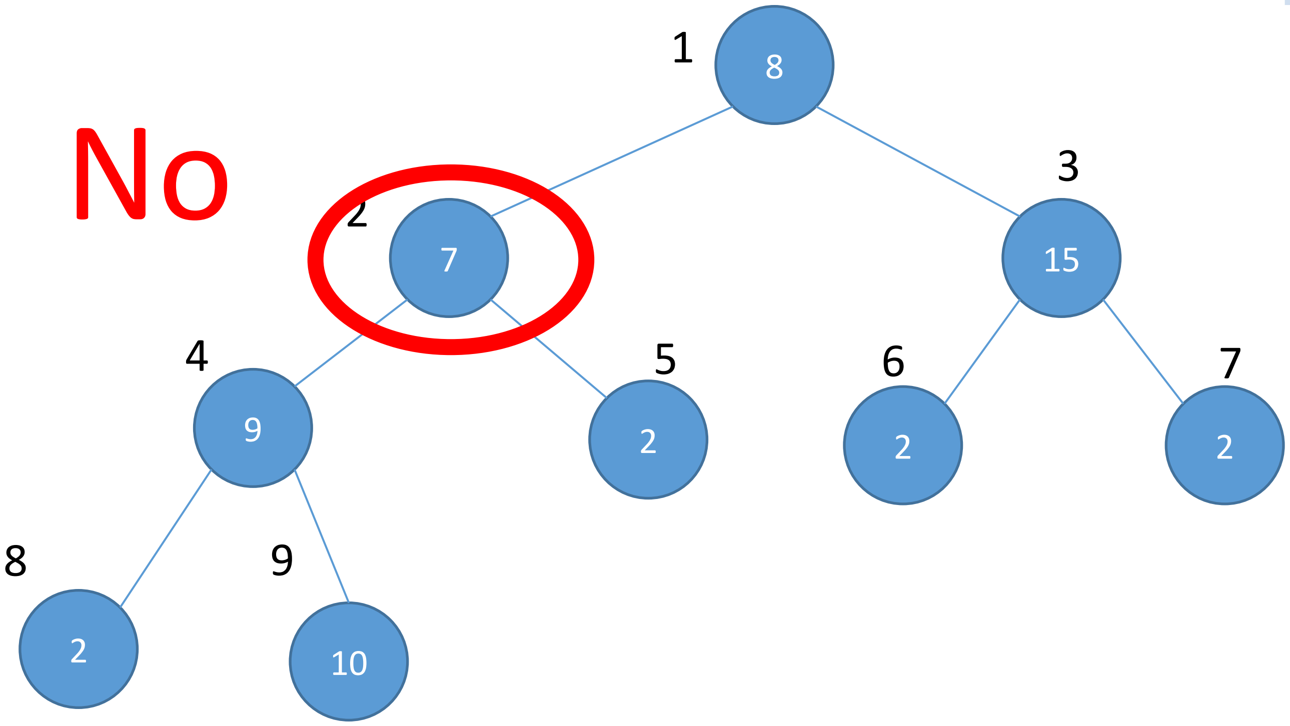


IS THIS A MAX HEAP NOW?

Max Heapify

- It is TRYING to make the array into a max heap
- But it may not fully succeed
- Let us consider the following example

1	2	3	4	5	6	7	8	9
2	7	8	9	2	15	2	2	10



IS THIS A MAX HEAP NOW?

Build Max Heap

- Converts an array into a heap

```
BUILD-MAX-HEAP(A)
  A.heap-size = A.length
  for i = ⌊A.length/2⌋ downto 1
      MAX-HEAPIFY(A, i)
```

Build Max Heap

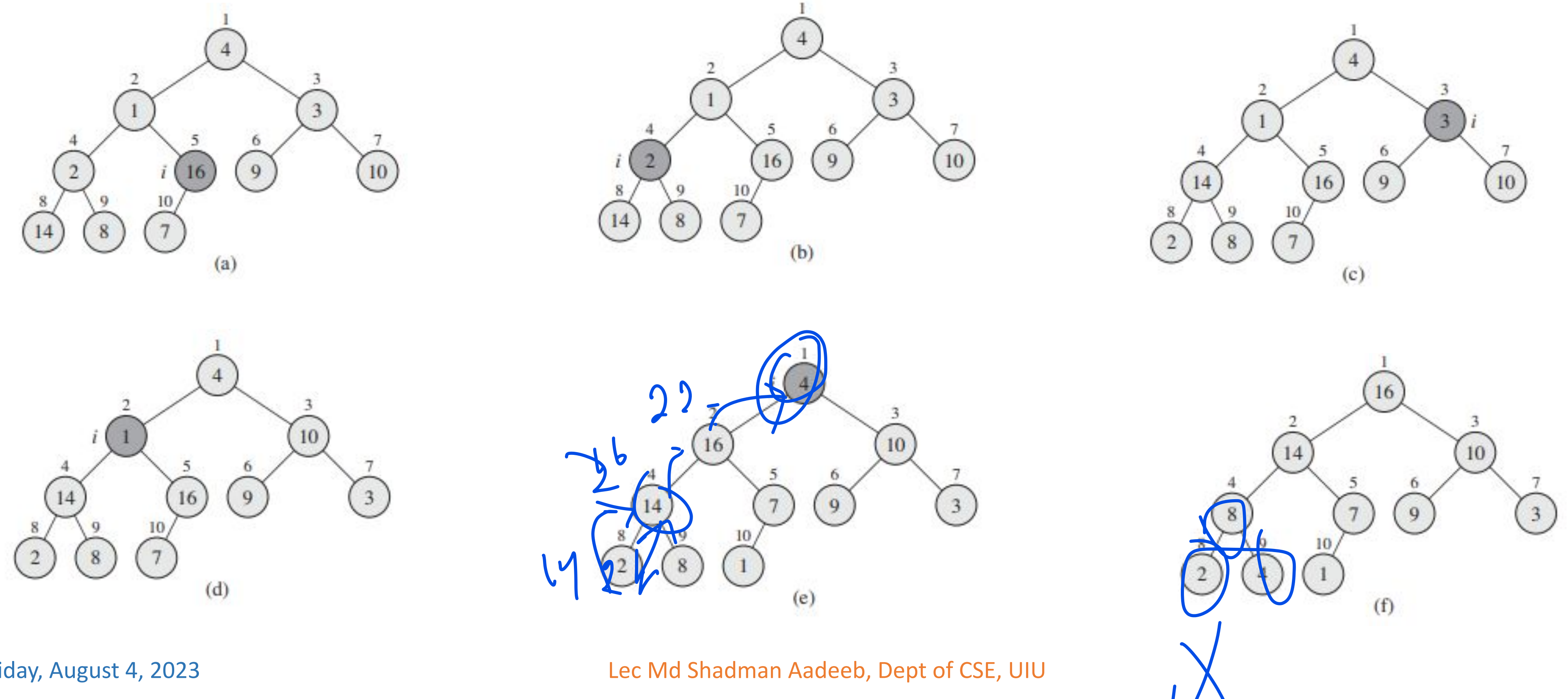
- Converts an array into a heap

```
BUILD-MAX-HEAP(A)
    A.heap-size = A.length                Ensuring that all the elements are on the imaginary tree
    for i = ⌊A.length/2⌋ downto 1         Running Max Heapify on all nodes with children
        MAX-HEAPIFY(A, i)
```

Build Max Heap

- Consider the following array

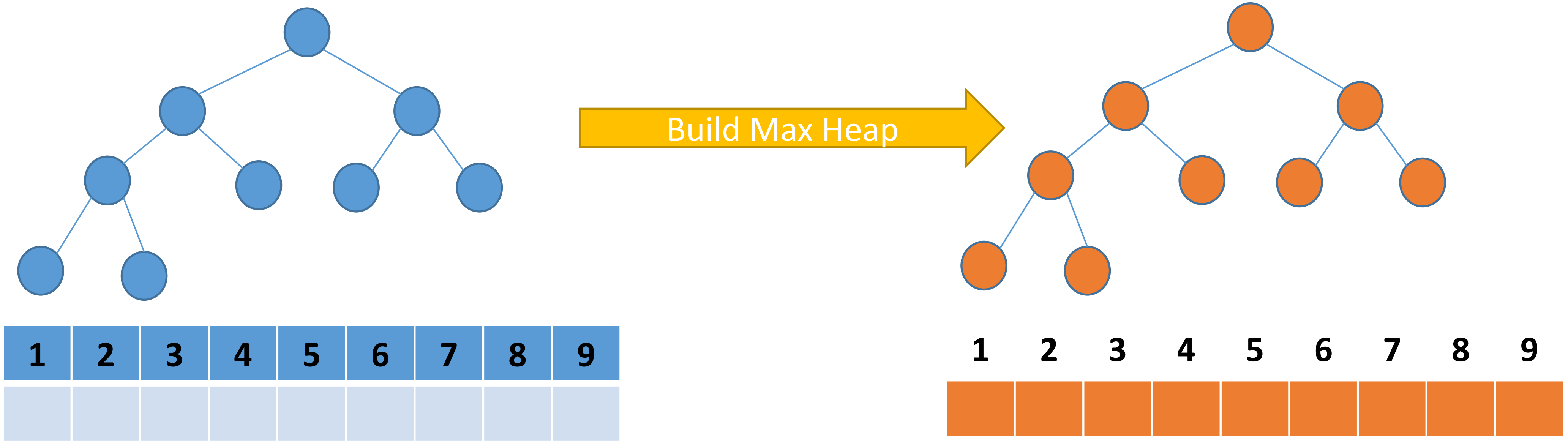
A	4	1	3	2	16	9	10	14	8	7



Heap Sort

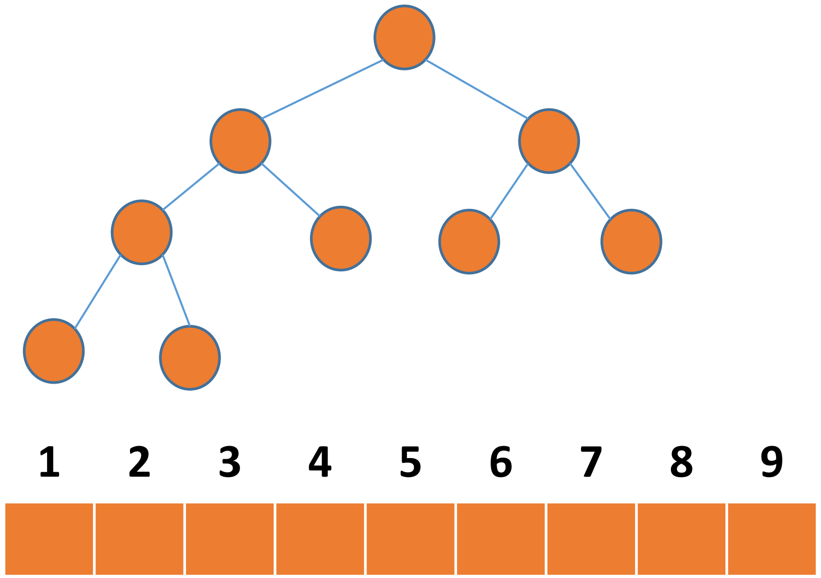
```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```

Parents are larger than children



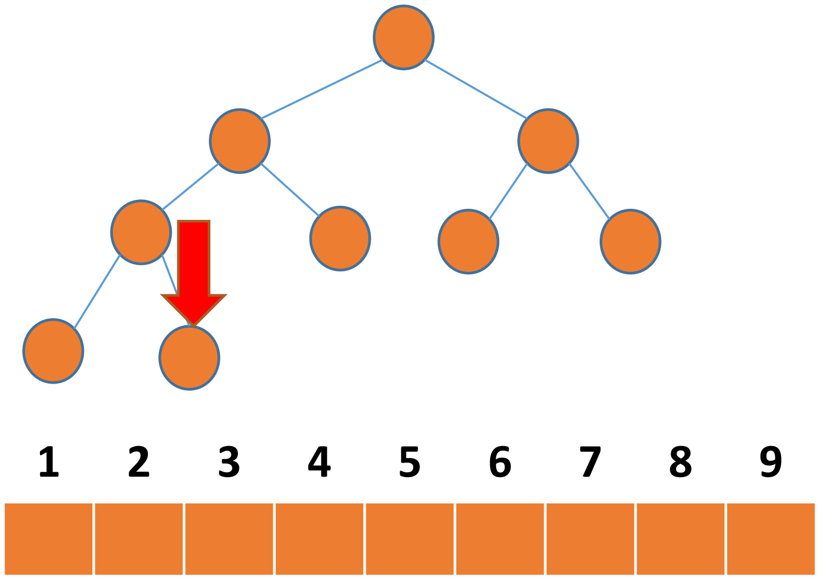
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



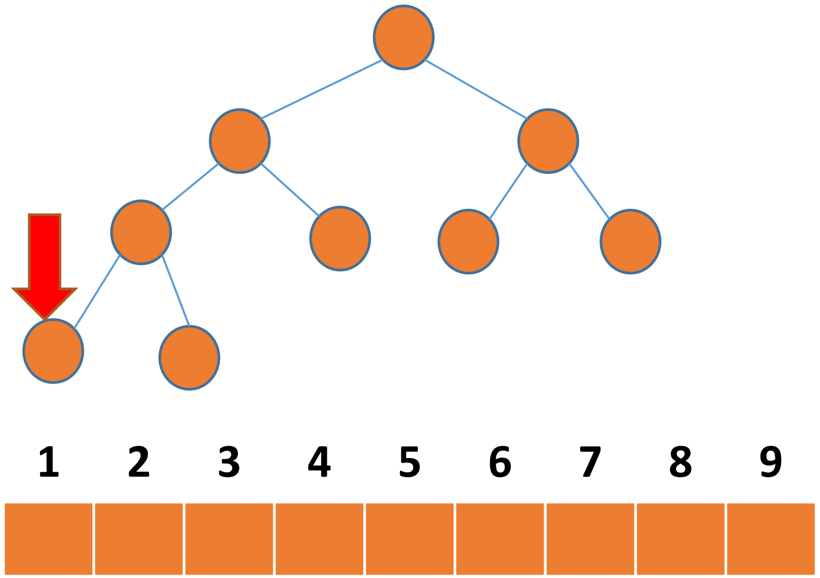
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



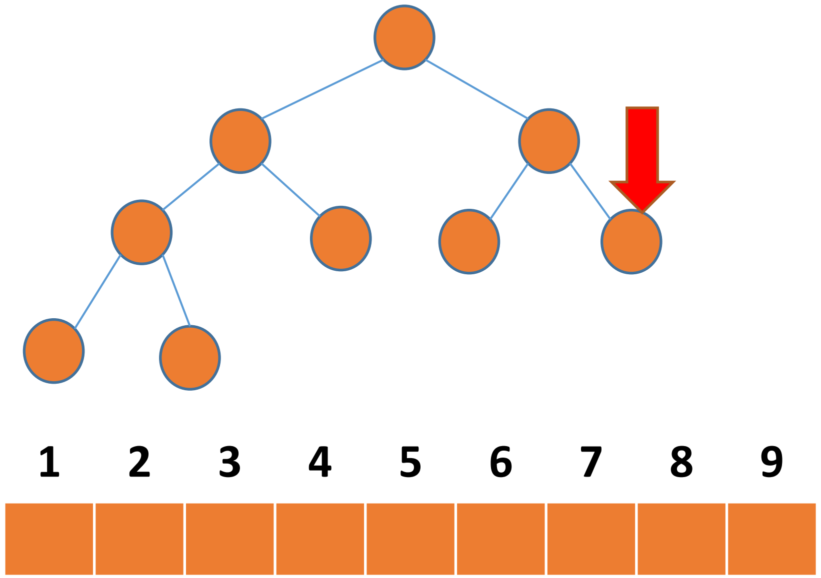
Heap Sort

```
HEAPSORT(A)
1  BUILD-MAX-HEAP(A)
2  for i = A.length downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```



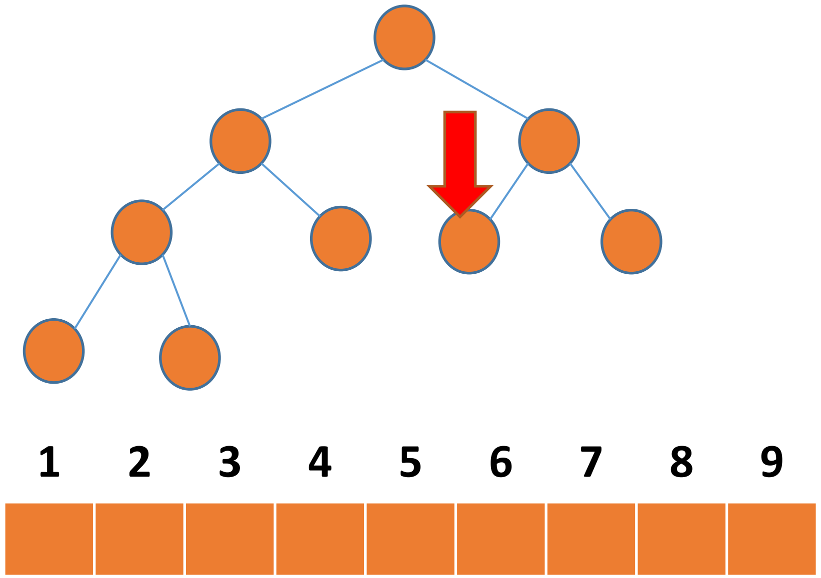
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```



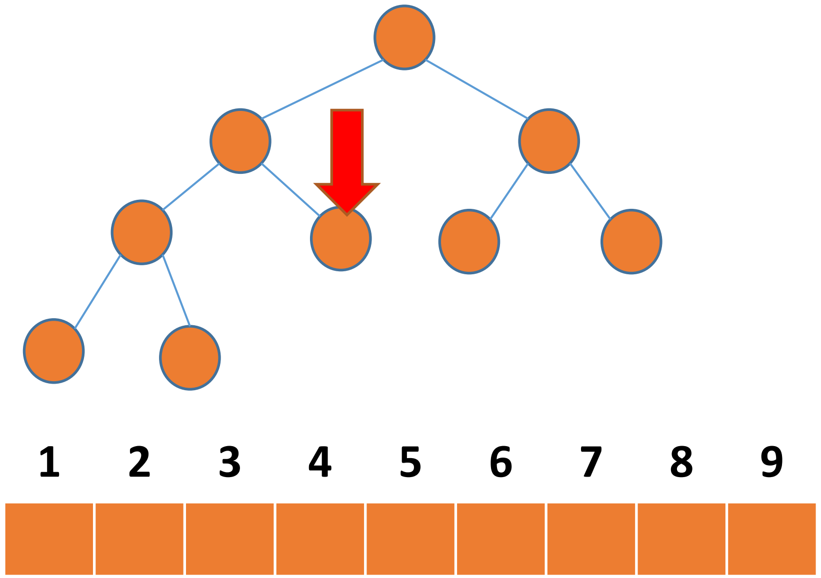
Heap Sort

```
HEAPSORT(A)
    BUILD-MAX-HEAP(A)
    for i = A.length downto 2
        exchange A[1] with A[i]
        A.heap-size = A.heap-size − 1
        MAX-HEAPIFY(A, 1)
```



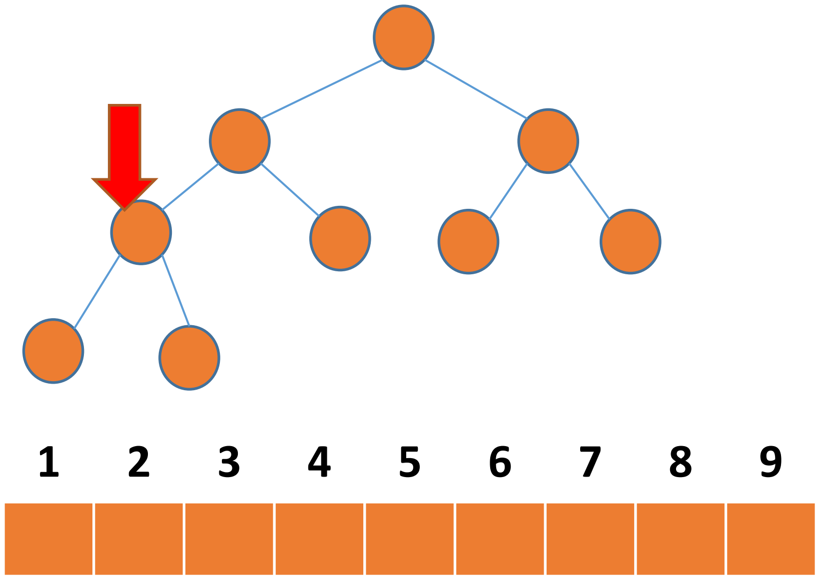
Heap Sort

```
HEAPSORT(A)
  BUILD-MAX-HEAP(A)
  for i = A.length downto 2
      exchange A[1] with A[i]
      A.heap-size = A.heap-size - 1
      MAX-HEAPIFY(A, 1)
```



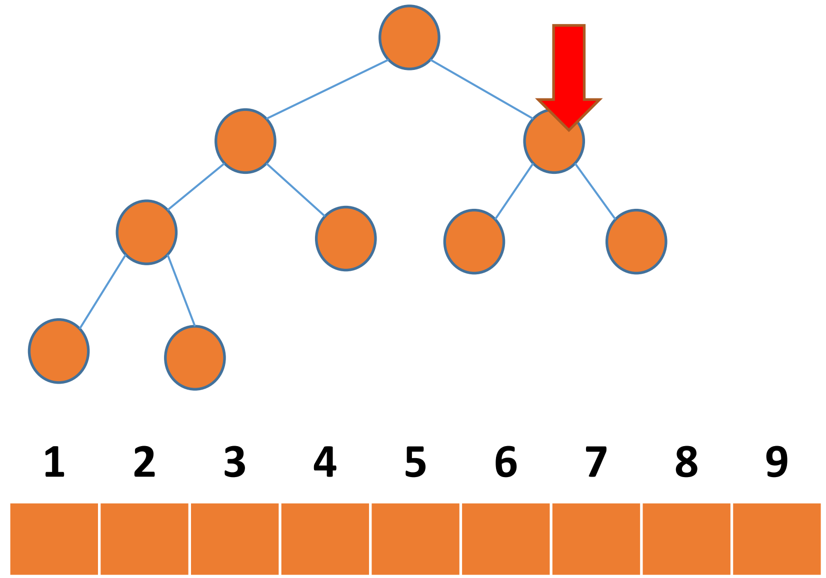
Heap Sort

```
HEAPSORT(A)
    BUILD-MAX-HEAP(A)
    for i = A.length downto 2
        exchange A[1] with A[i]
        A.heap-size = A.heap-size − 1
        MAX-HEAPIFY(A, 1)
```



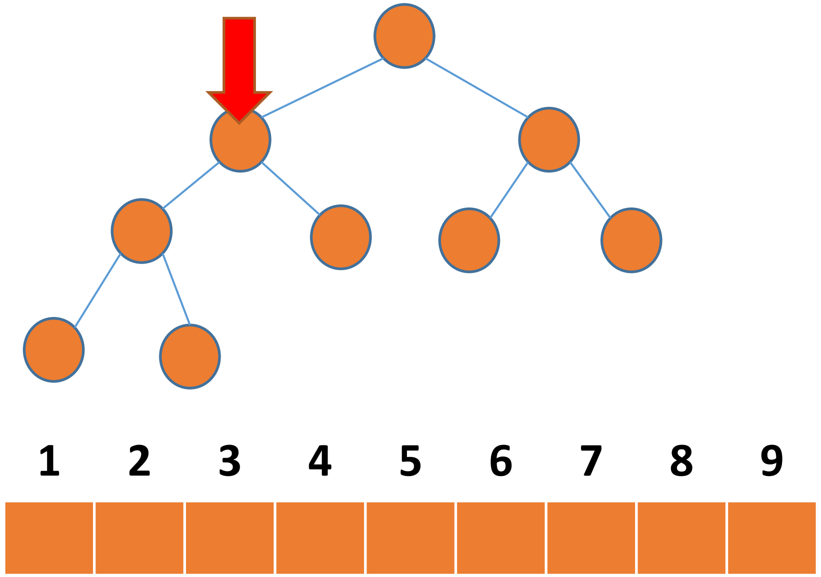
Heap Sort

```
HEAPSORT(A)
    BUILD-MAX-HEAP(A)
    for i = A.length downto 2
        exchange A[1] with A[i]
        A.heap-size = A.heap-size − 1
        MAX-HEAPIFY(A, 1)
```



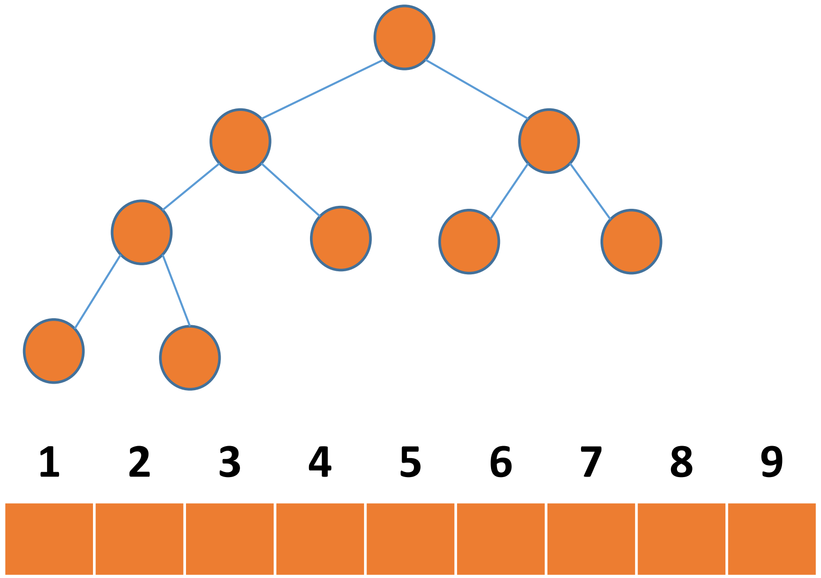
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



Heap Sort

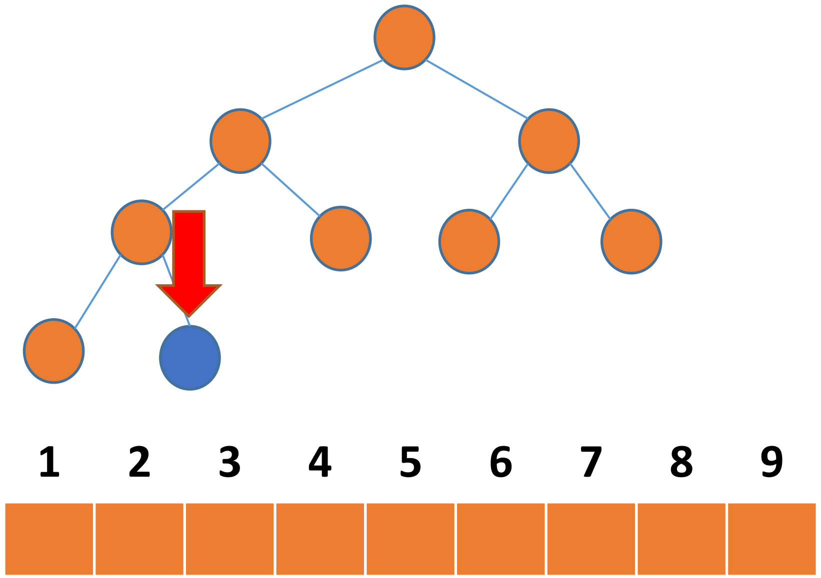
```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



Heap Sort

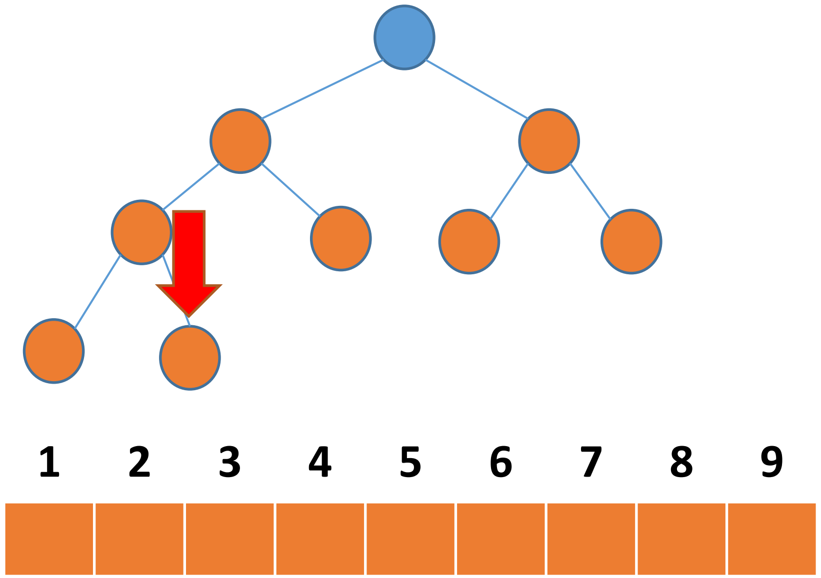
HEAPSORT(A)

```
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



Heap Sort

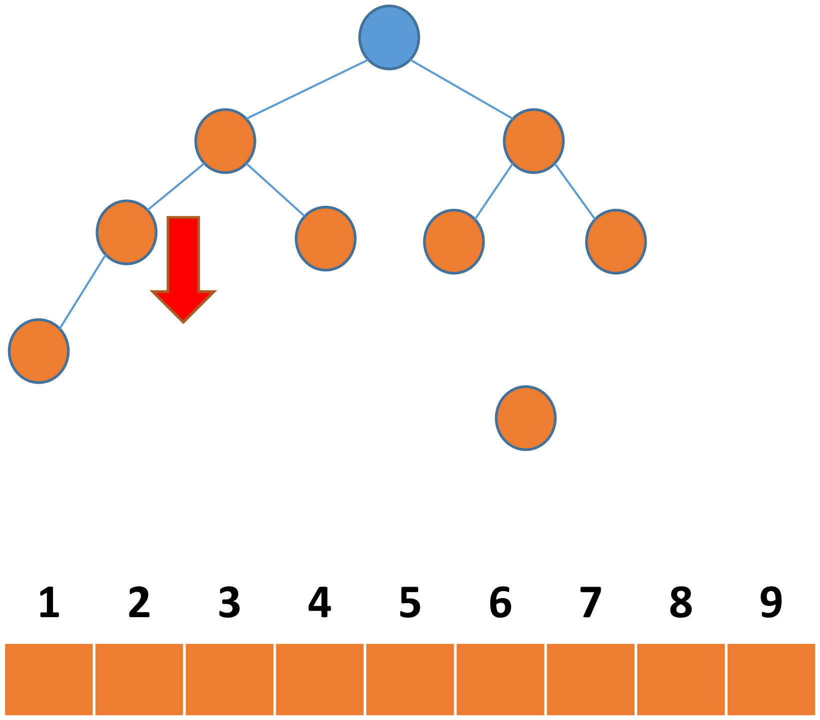
```
HEAPSORT(A)

1  BUILD-MAX-HEAP(A)
2  for i = A.length downto 2
3      exchange A[1] with A[i]
4      A.heap-size = A.heap-size − 1
5      MAX-HEAPIFY(A, 1)
```



Heap Sort

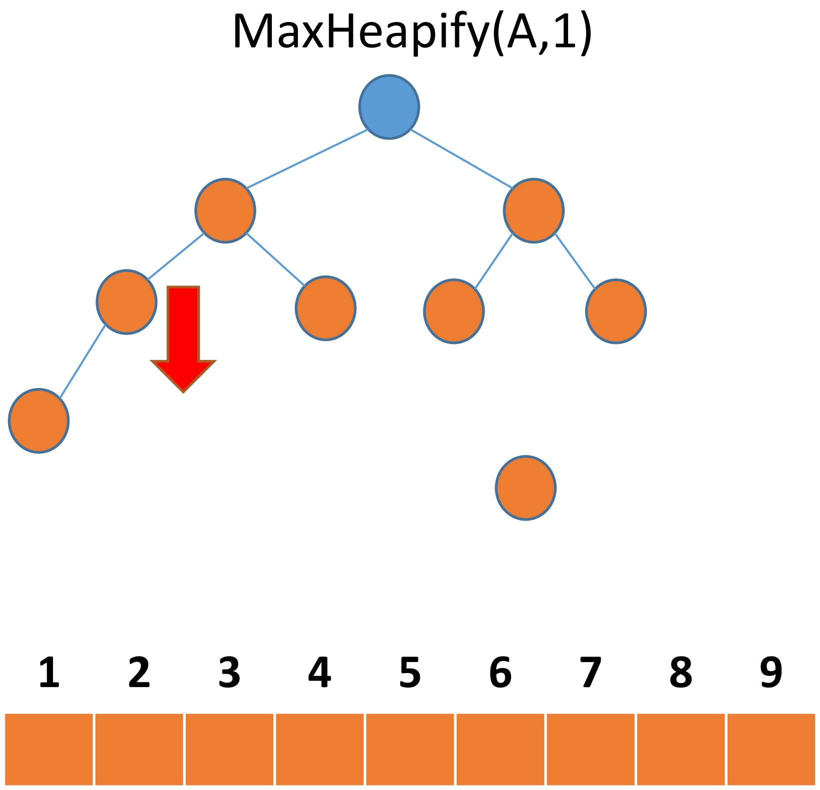
HEAPSORT(A)

1 BUILD-MAX-HEAP(A)
2 **for** $i = A.length$ **downto** 2
3   exchange $A[1]$ with $A[i]$
4     $A.heap\text{-}size = A.heap\text{-}size - 1$
5   MAX-HEAPIFY(A, 1)



Heap Sort

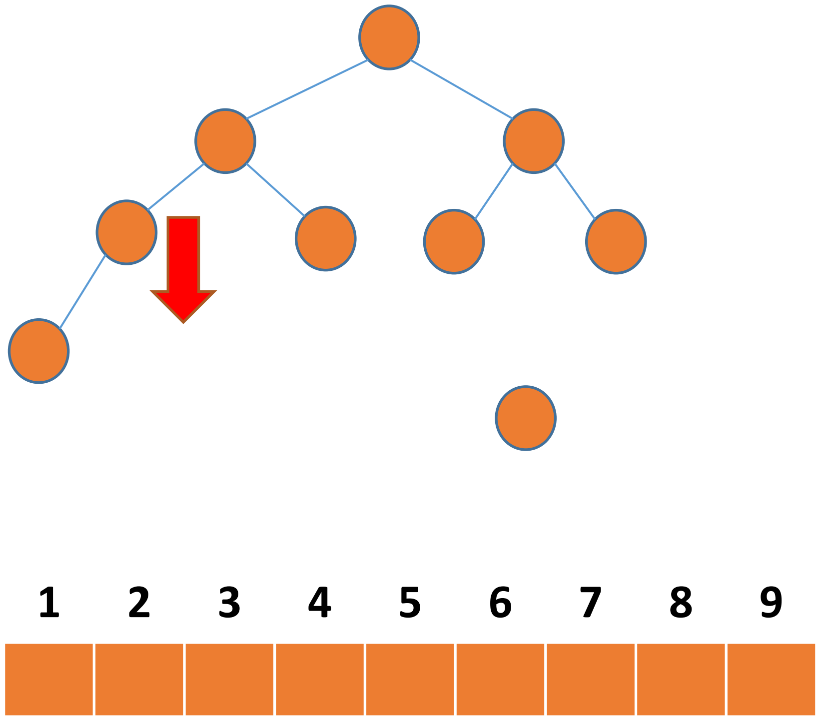
```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



Heap Sort

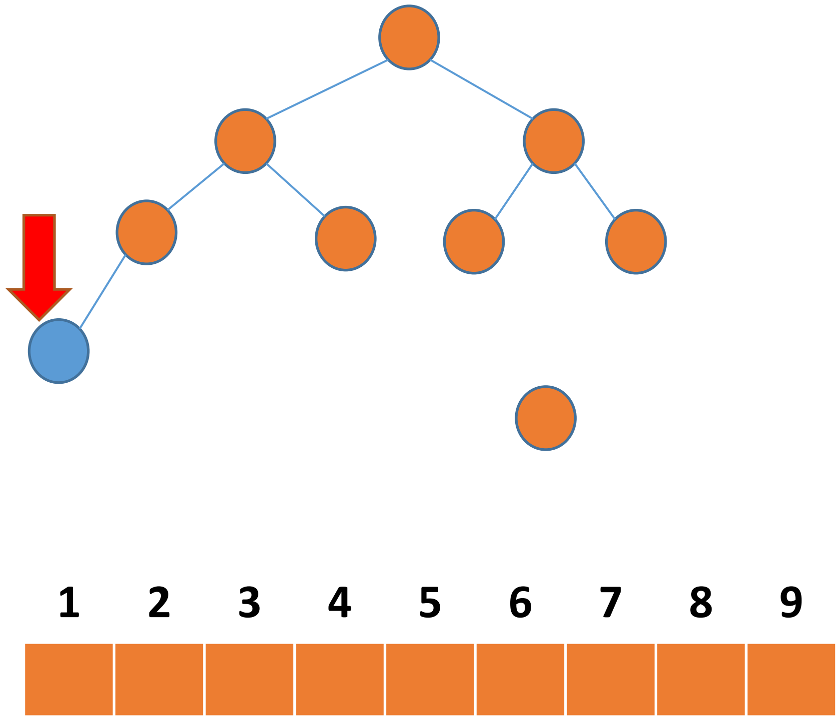
HEAPSORT(A)

1 BUILD-MAX-HEAP(A)
2 **for** $i = A.length$ **downto** 2
3   exchange $A[1]$ with $A[i]$
4     $A.heap\text{-}size = A.heap\text{-}size - 1$
5   MAX-HEAPIFY(A, 1)



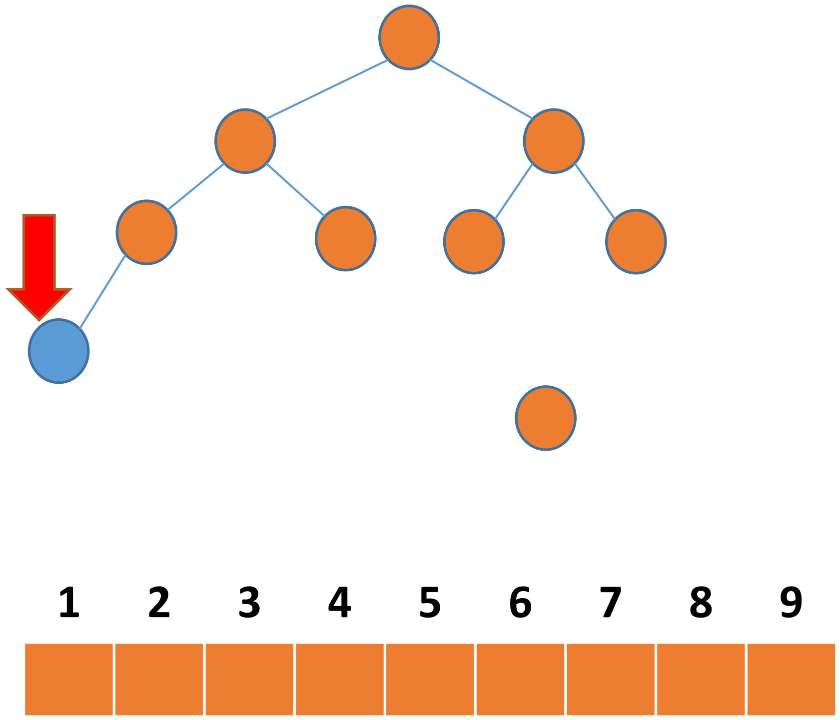
Heap Sort

```
HEAPSORT(A)
  BUILD-MAX-HEAP(A)
  for i = A.length downto 2
      exchange A[1] with A[i]
      A.heap-size = A.heap-size − 1
      MAX-HEAPIFY(A, 1)
```



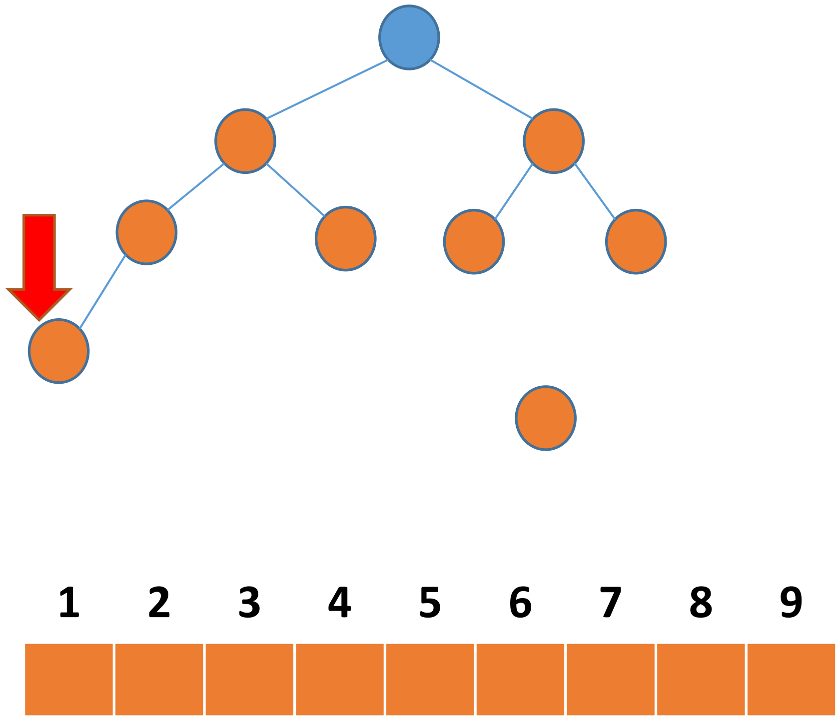
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



Heap Sort

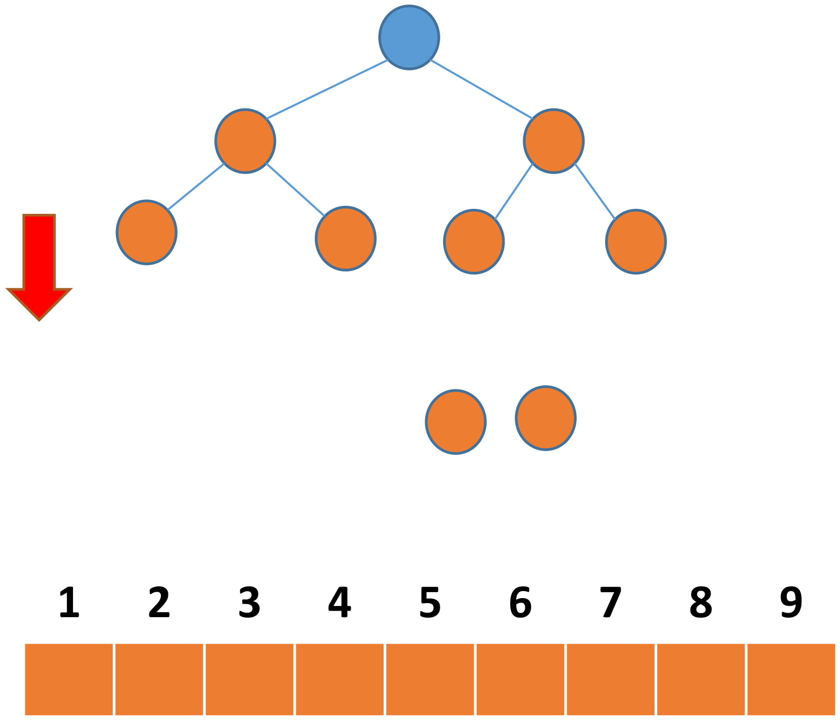
```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```



Heap Sort

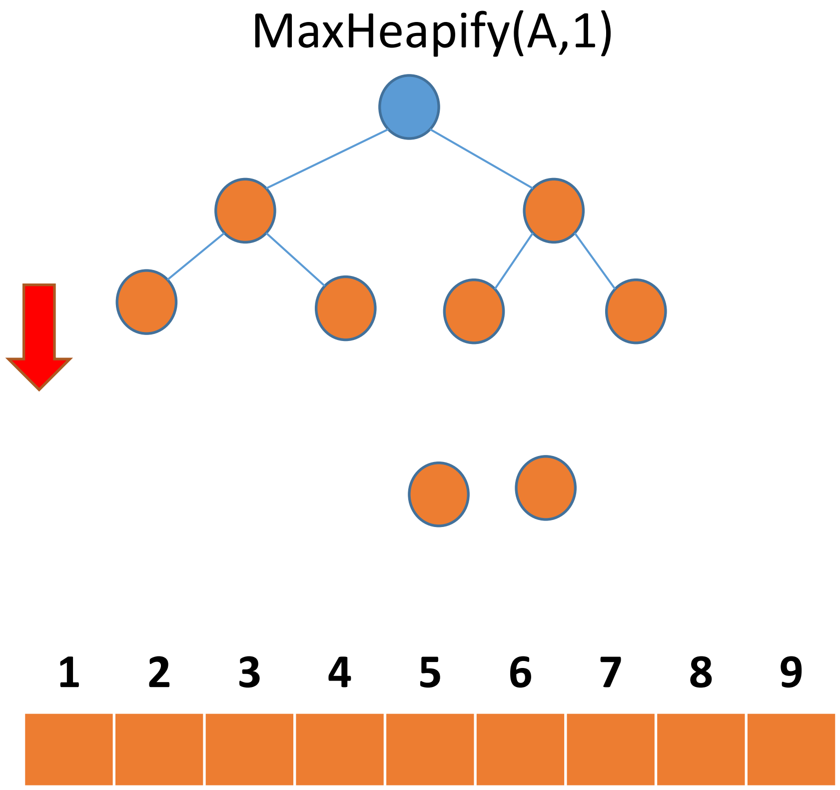
HEAPSORT(A)

BUILD-MAX-HEAP(A)
for i = $A.length$ **downto** 2
    exchange $A[1]$ with $A[i]$
    $A.heap\text{-}size = A.heap\text{-}size - 1$
    MAX-HEAPIFY(A, 1)



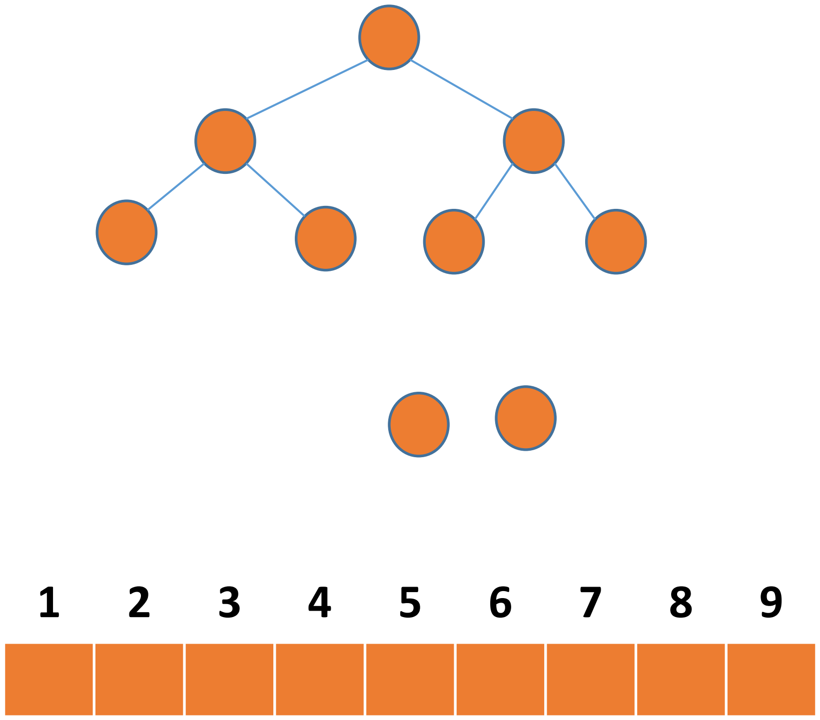
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```



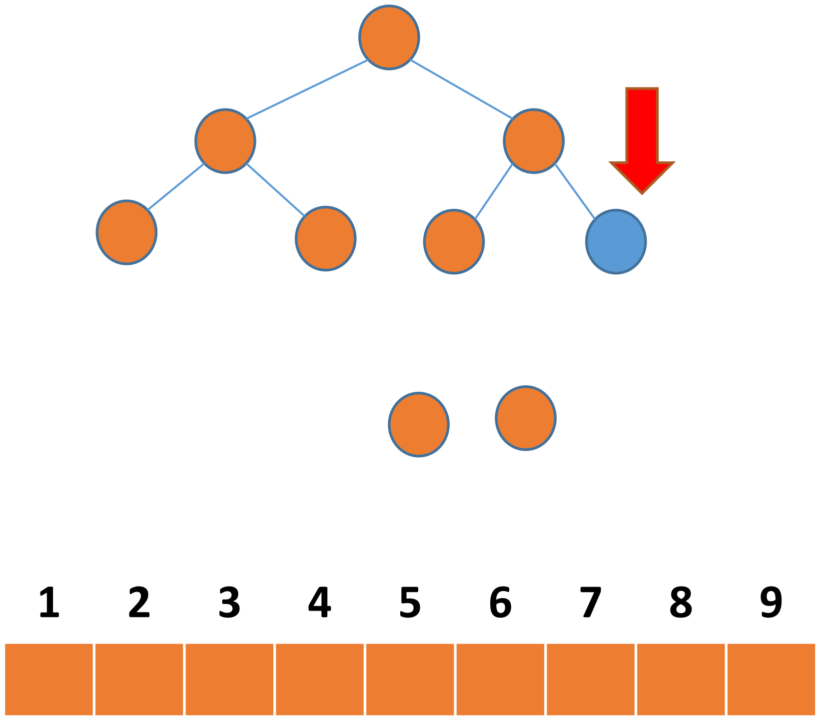
Heap Sort

```
HEAPSORT(A)
  BUILD-MAX-HEAP(A)
  for i = A.length downto 2
      exchange A[1] with A[i]
      A.heap-size = A.heap-size - 1
      MAX-HEAPIFY(A, 1)
```



Heap Sort

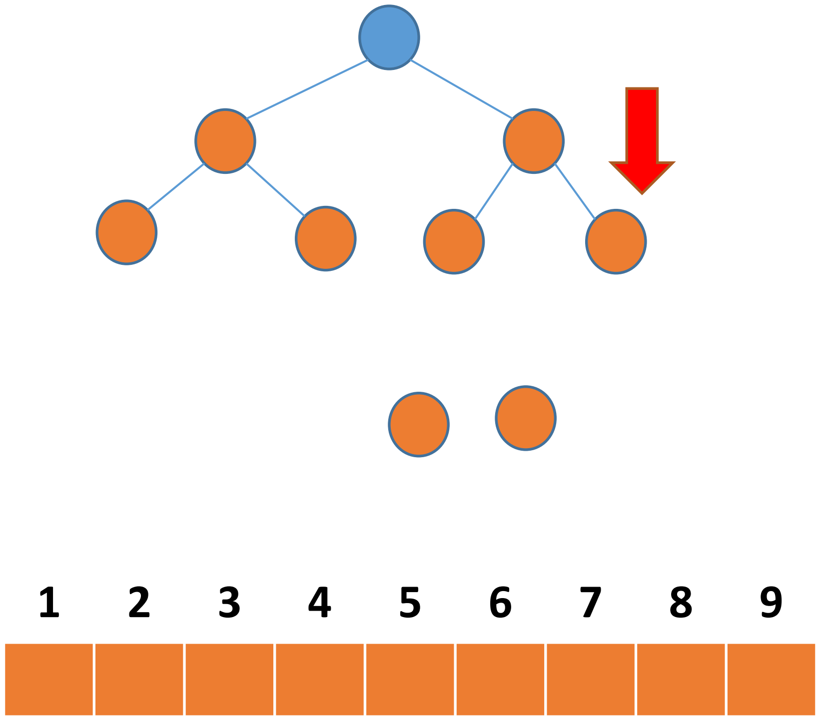
```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



Heap Sort

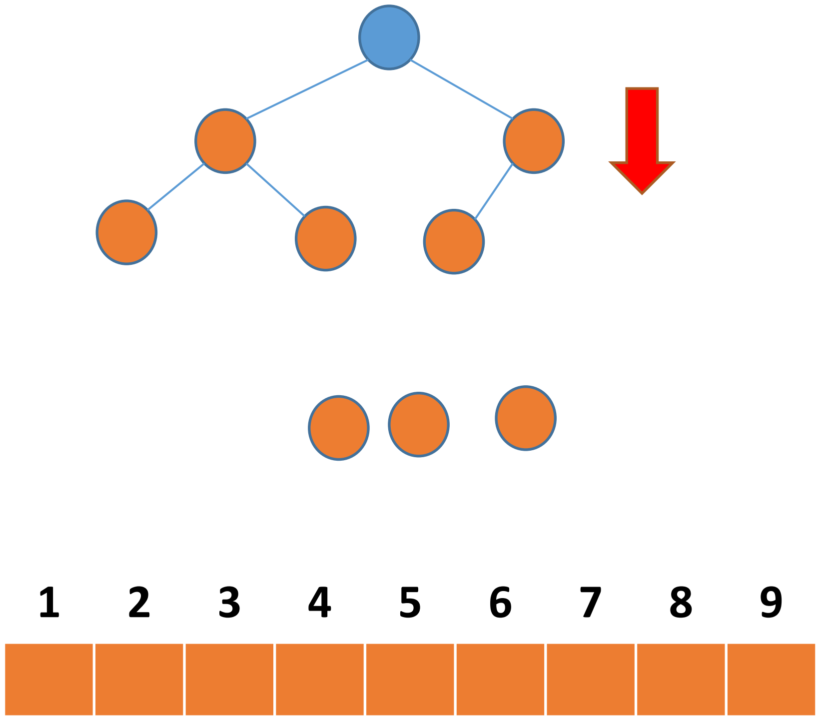
HEAPSORT(A)

1 BUILD-MAX-HEAP(A)
2 **for** $i = A.length$ **downto** 2
3   exchange $A[1]$ with $A[i]$
4     $A.heap\text{-}size = A.heap\text{-}size - 1$
5   MAX-HEAPIFY(A, 1)



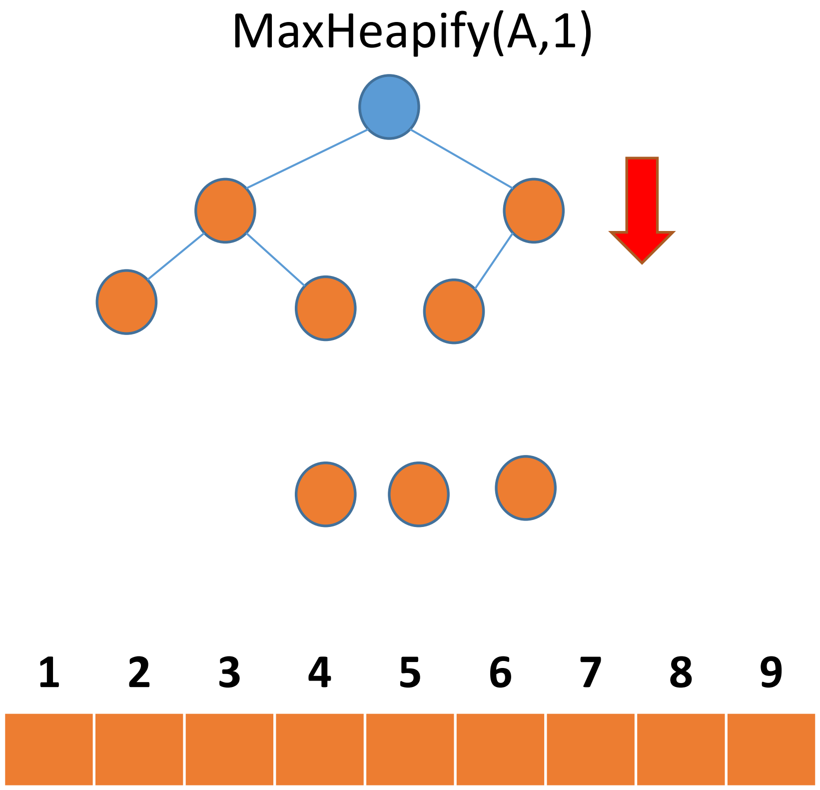
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



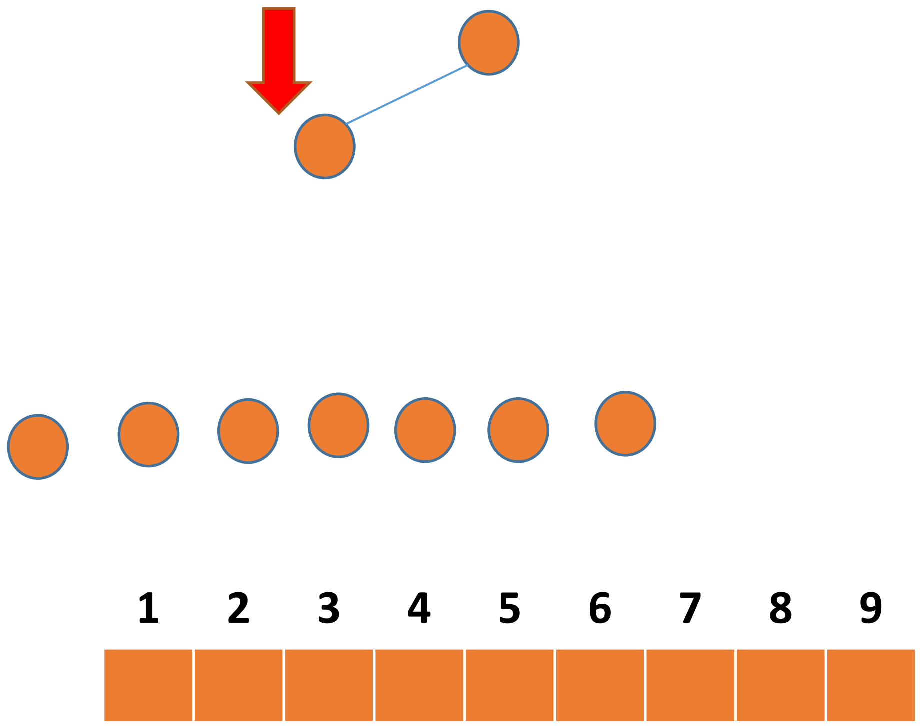
Heap Sort

```
HEAPSORT(A)
    BUILD-MAX-HEAP(A)
    for i = A.length downto 2
        exchange A[1] with A[i]
        A.heap-size = A.heap-size − 1
        MAX-HEAPIFY(A, 1)
```



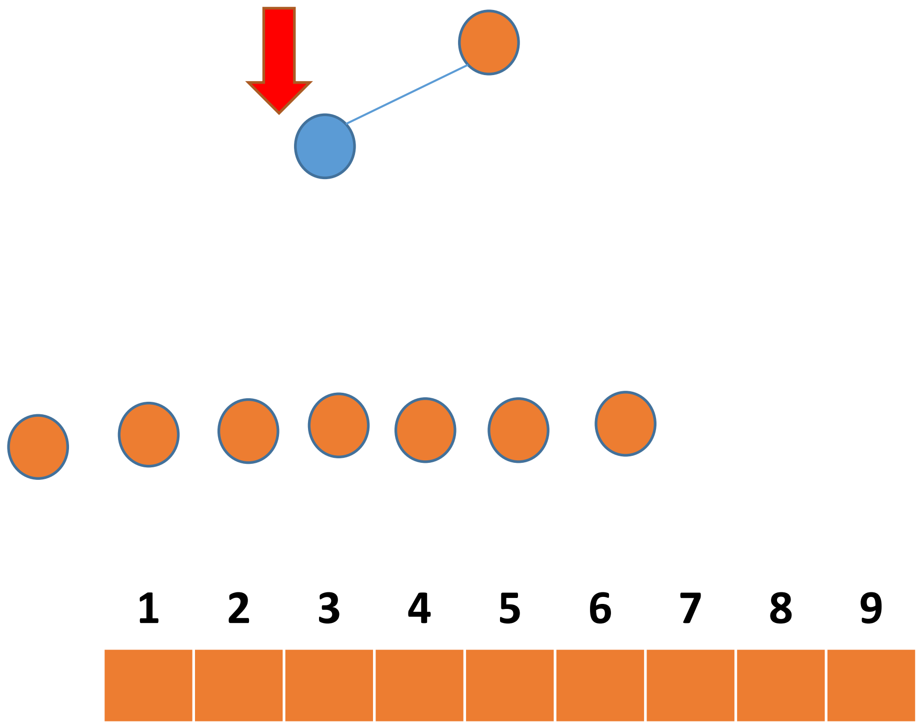
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```



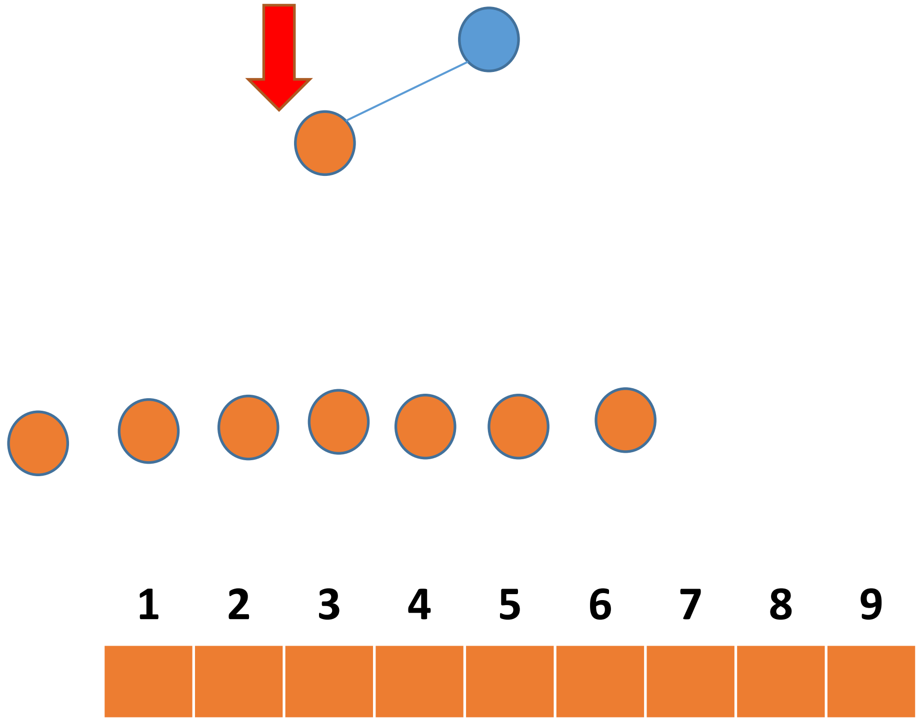
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```



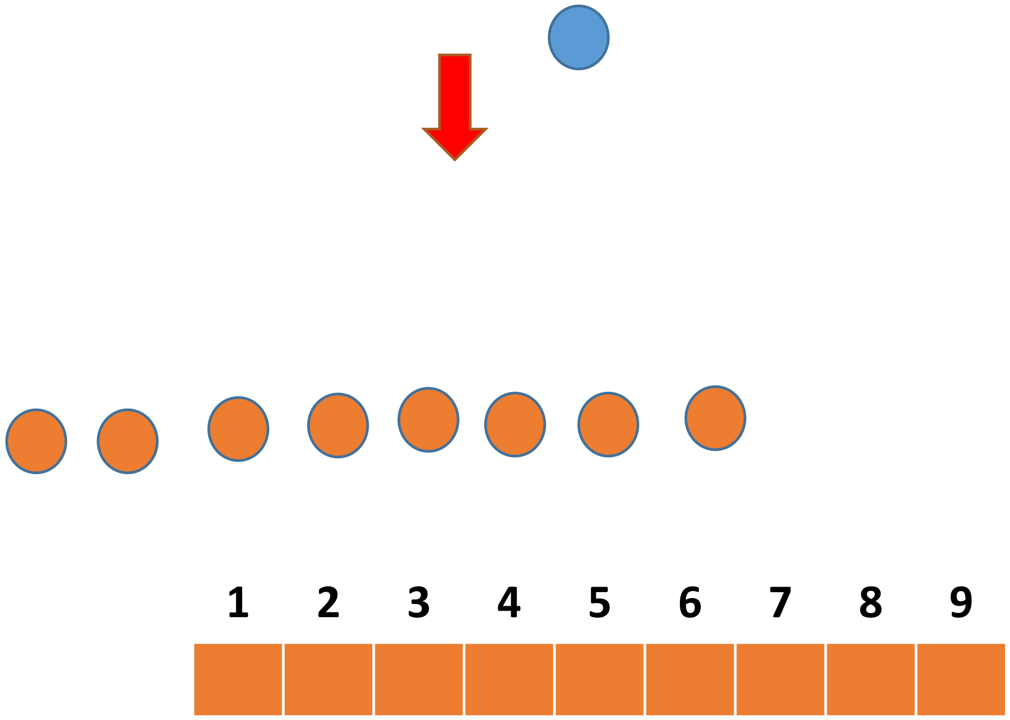
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



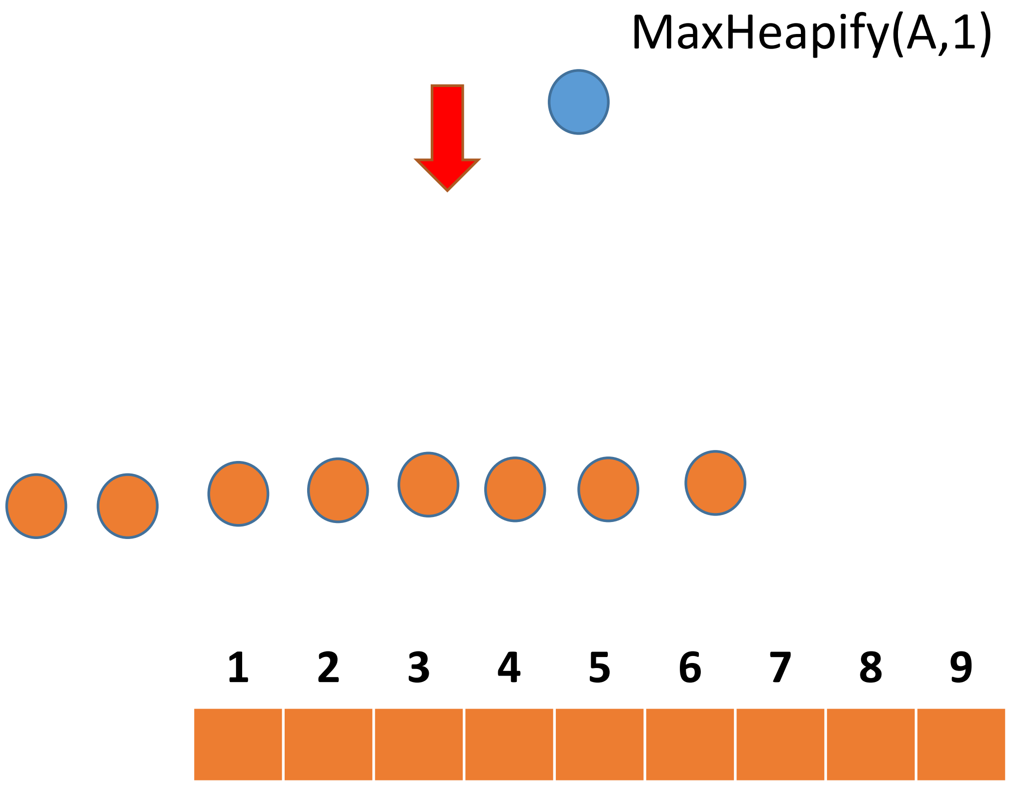
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



Heap Sort

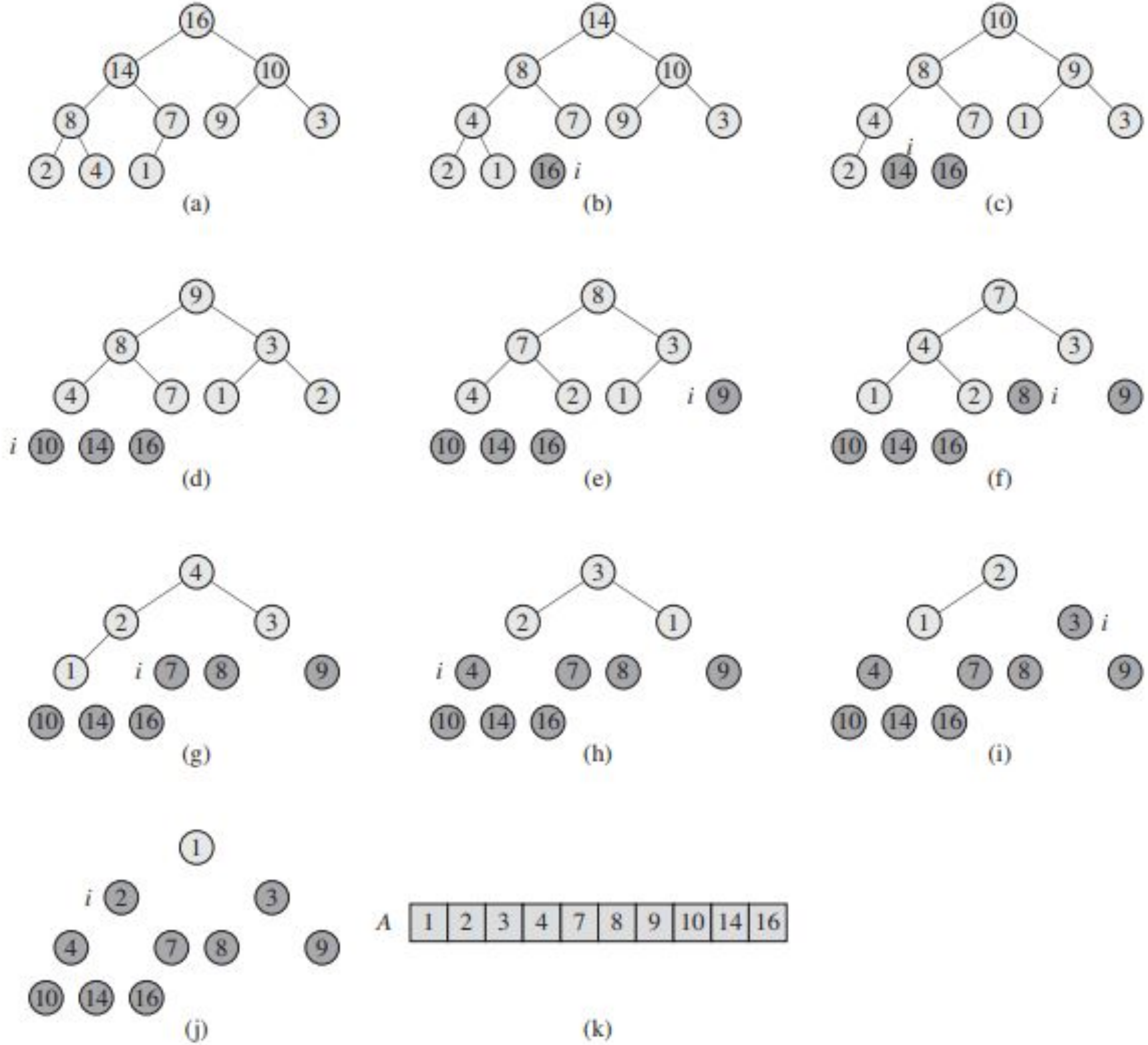
```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```



Let us Run Heap Sort At First

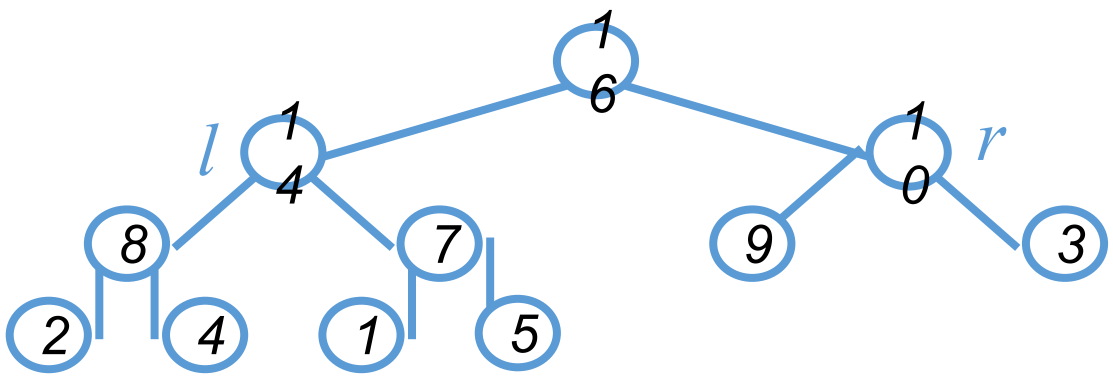
Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size − 1
    MAX-HEAPIFY(A, 1)
```


(a)
(b)
(c)
(d)
(e)
(f)
(g)
(h)
(i)
(j)
A 1 2 3 4 7 8 9 10 14 16
(k)

Time Complexity Analysis-Max Heapify

- **Max Heapify**
  - Fixing up relationships among the elements $A[i]$, $A[l]$, and $A[r]$ takes $\Theta(1)$ time
  - *If the heap at i has n elements, at most how many elements can the subtrees at l or r have?*

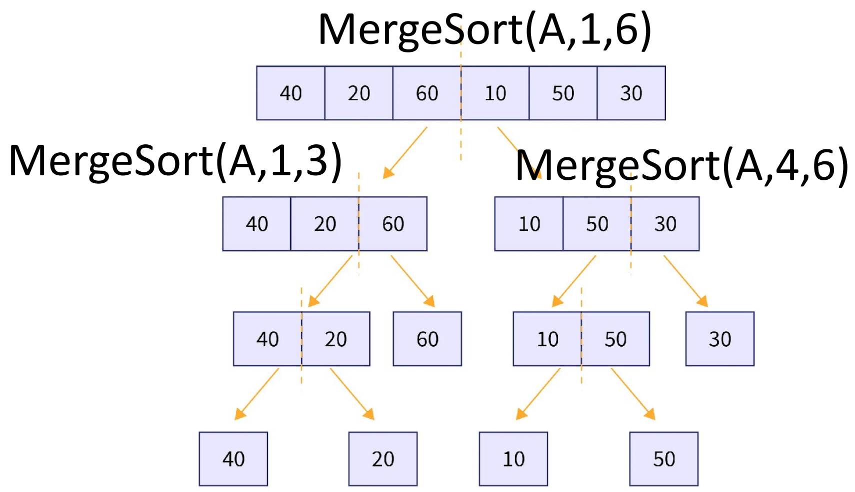



```
MAX-HEAPIFY(A, i)
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

- Answer: $2n/3$ (worst case: bottom row half full)
- So time taken by **`Heapify()`** is given by
  $T(n) \leq T(2n/3) + \Theta(1)$

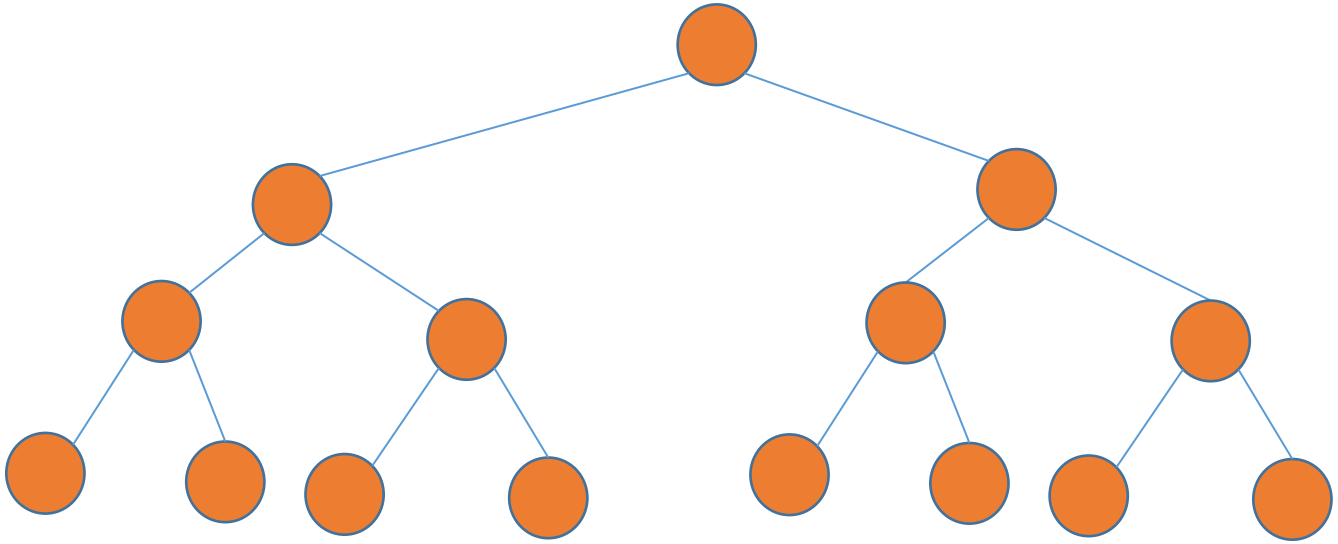
Time Complexity Analysis of Max Heapify

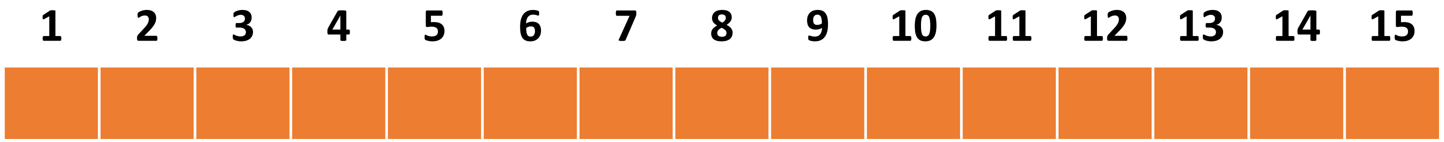
- Recall the time complexity analysis of merge sort
- Merge sort was a recursive algorithm



$T(n) = T(n/2) + T(n/2) + bn$

Time Complexity Analysis of Max Heapify

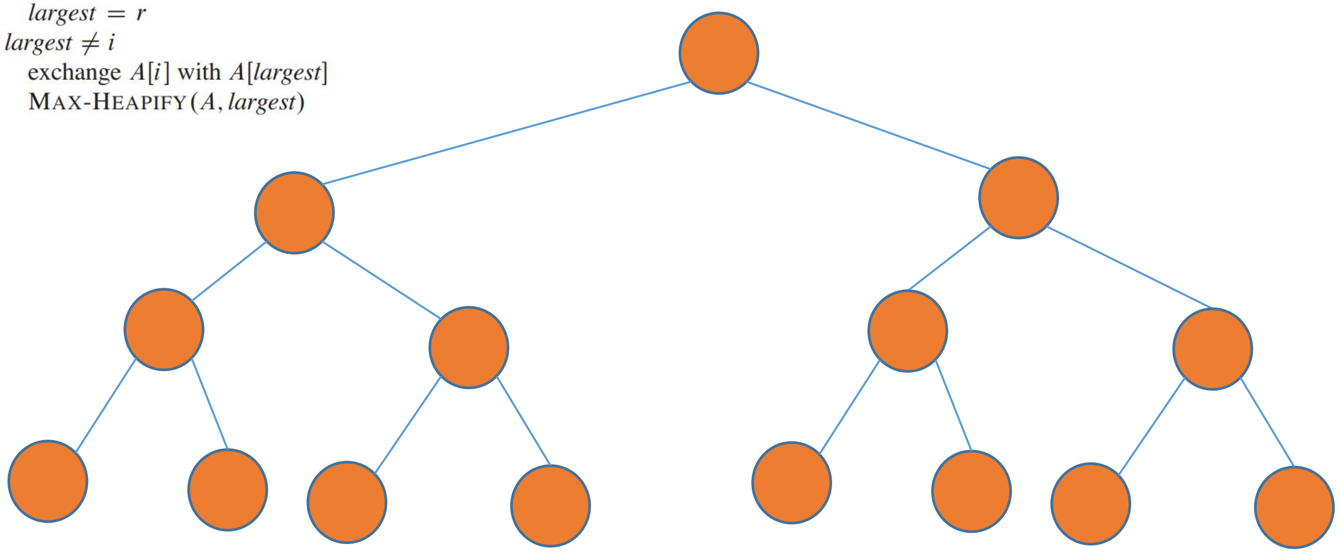


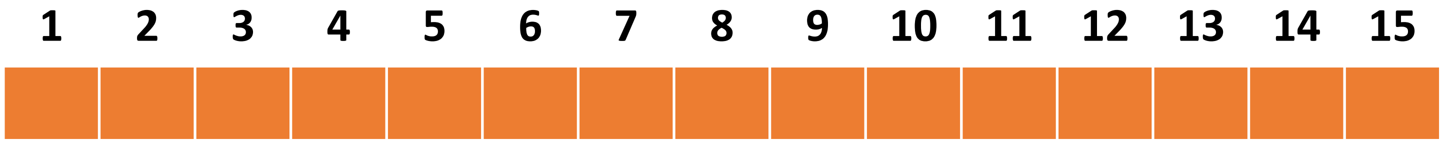


Time Complexity Analysis of Max Heapify

```
MAX-HEAPIFY(A, i)
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

MaxHeapify



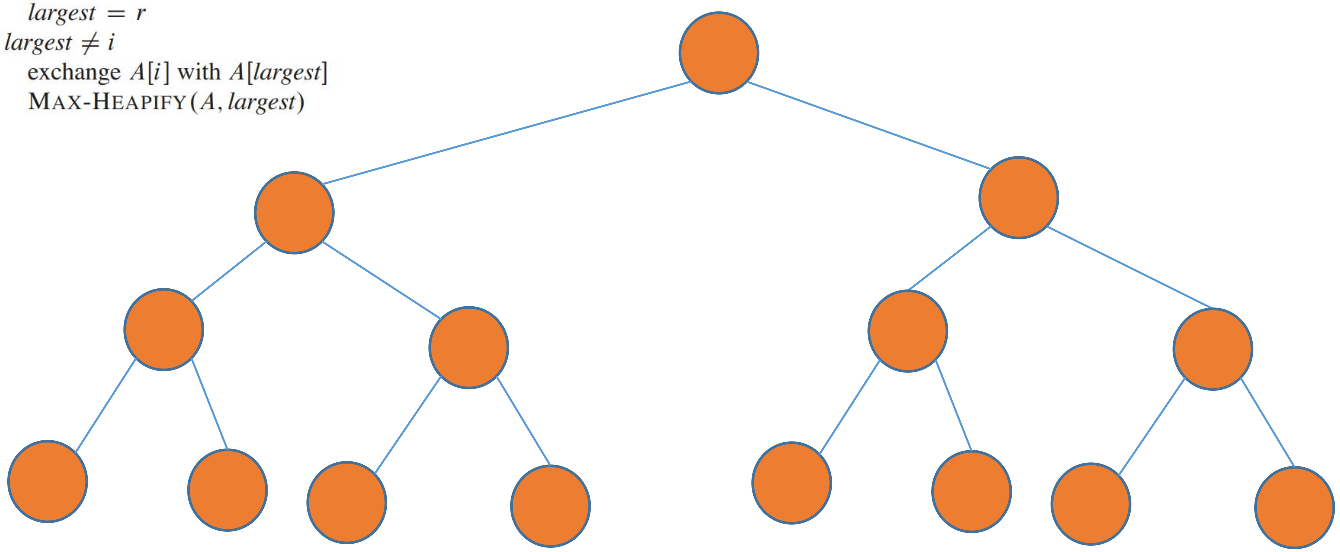


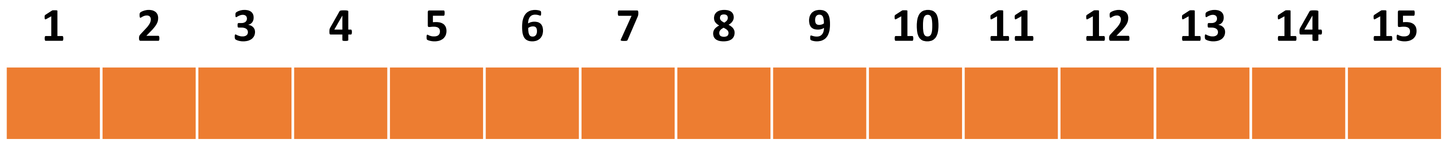
Time Complexity Analysis of Max Heapify

```
MAX-HEAPIFY(A, i)
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

MaxHeapify

T(n)



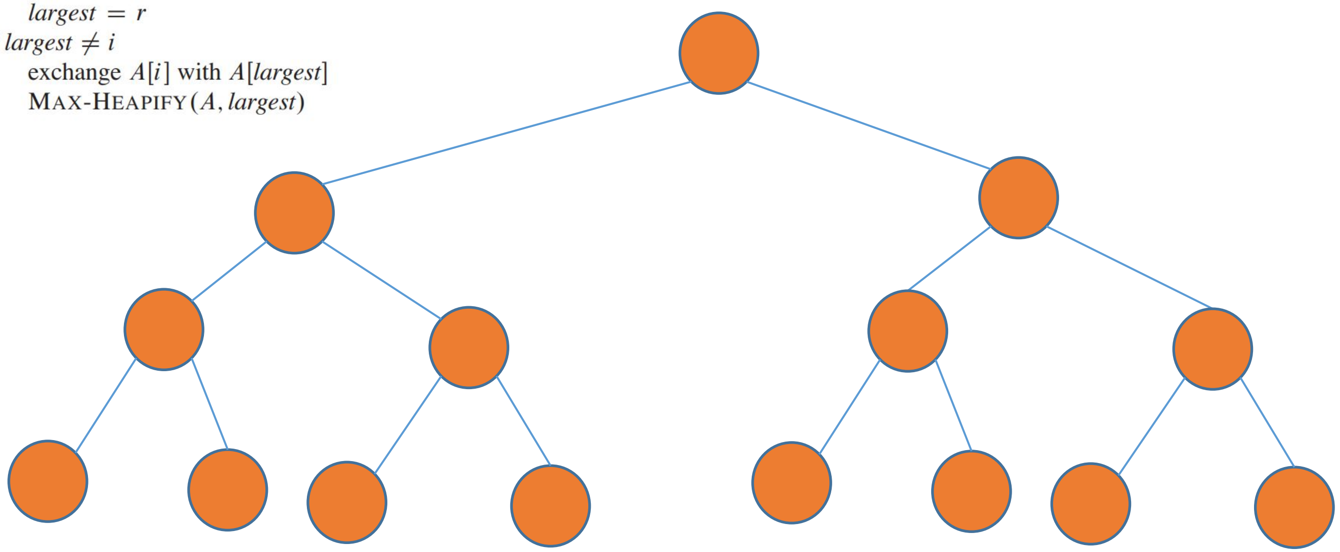


Time Complexity Analysis of Max Heapify

MAX-HEAPIFY(A, i)

```
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

MaxHeapify

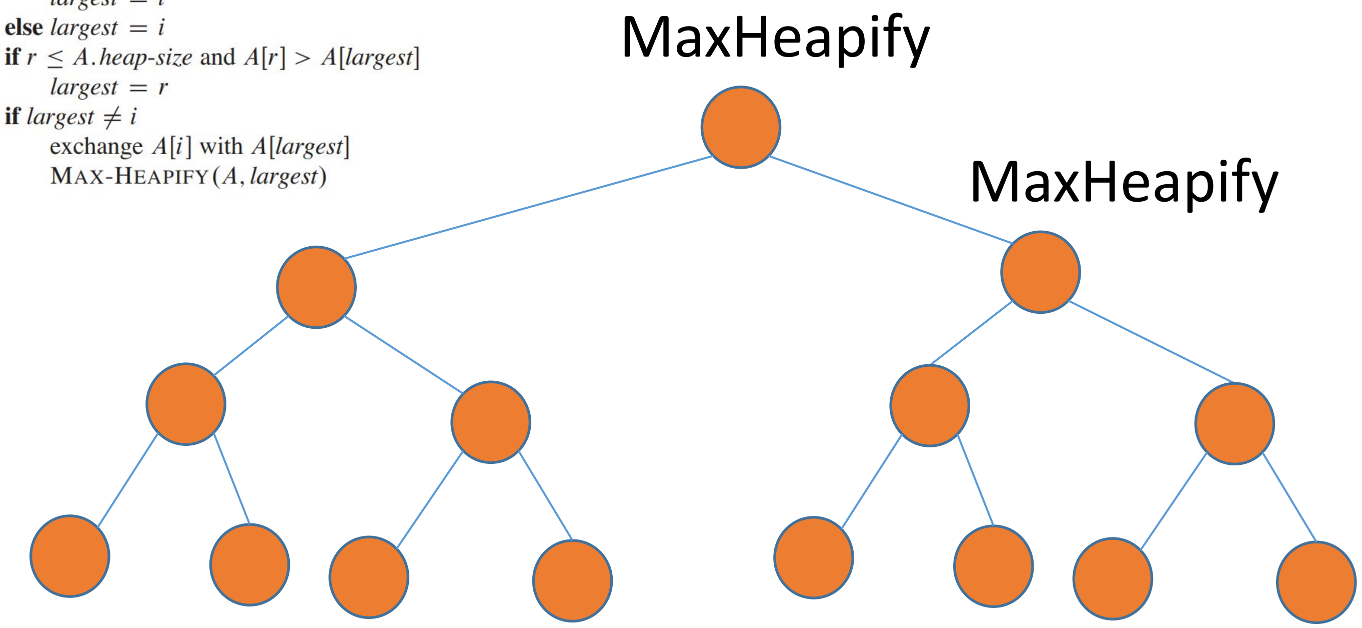


T(n) =b (1st element is greater than its children)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Time Complexity Analysis of Max Heapify

```
MAX-HEAPIFY(A, i)
 l = LEFT(i)
 r = RIGHT(i)
 if l ≤ A.heap-size and A[l] > A[i]
     largest = l
 else largest = i
 if r ≤ A.heap-size and A[r] > A[largest]
     largest = r
 if largest ≠ i
     exchange A[i] with A[largest]
     MAX-HEAPIFY(A, largest)
```

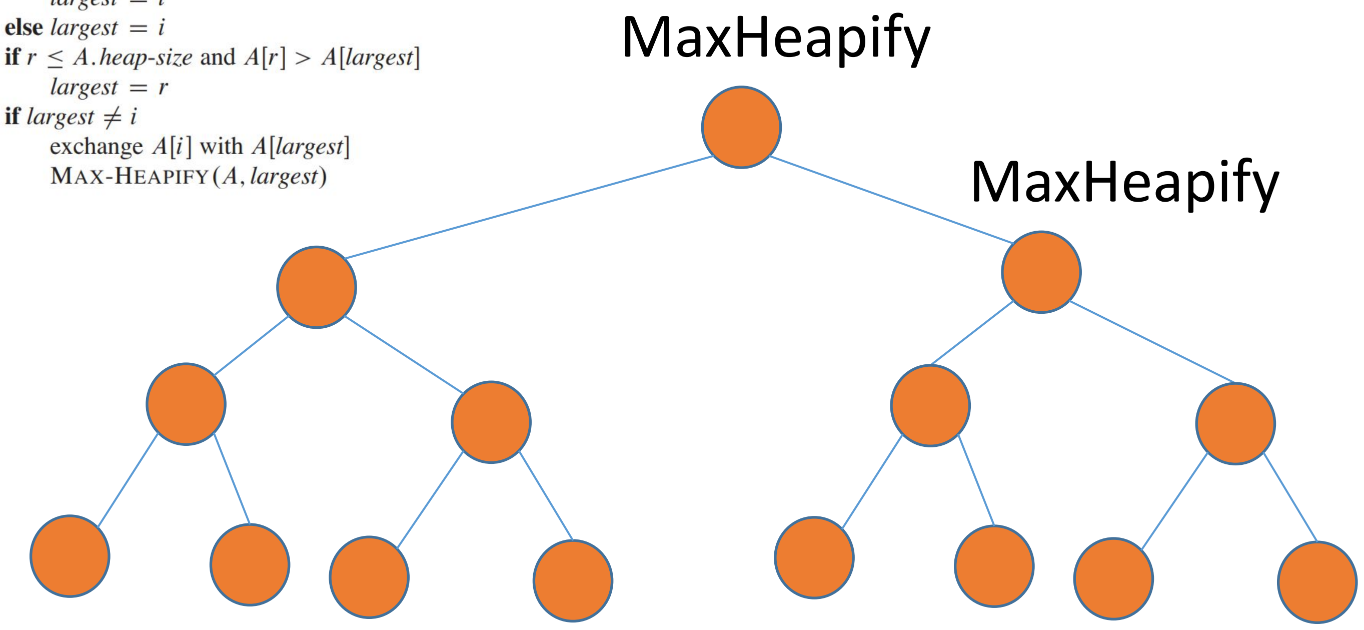


$$T(n) = T(n/2)+b$$

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

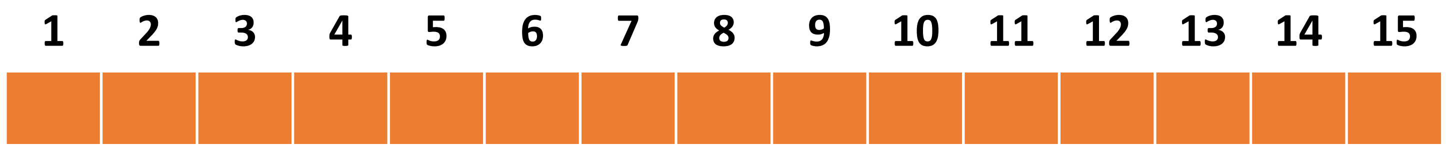
Time Complexity Analysis of Max Heapify

```
MAX-HEAPIFY(A, i)
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```



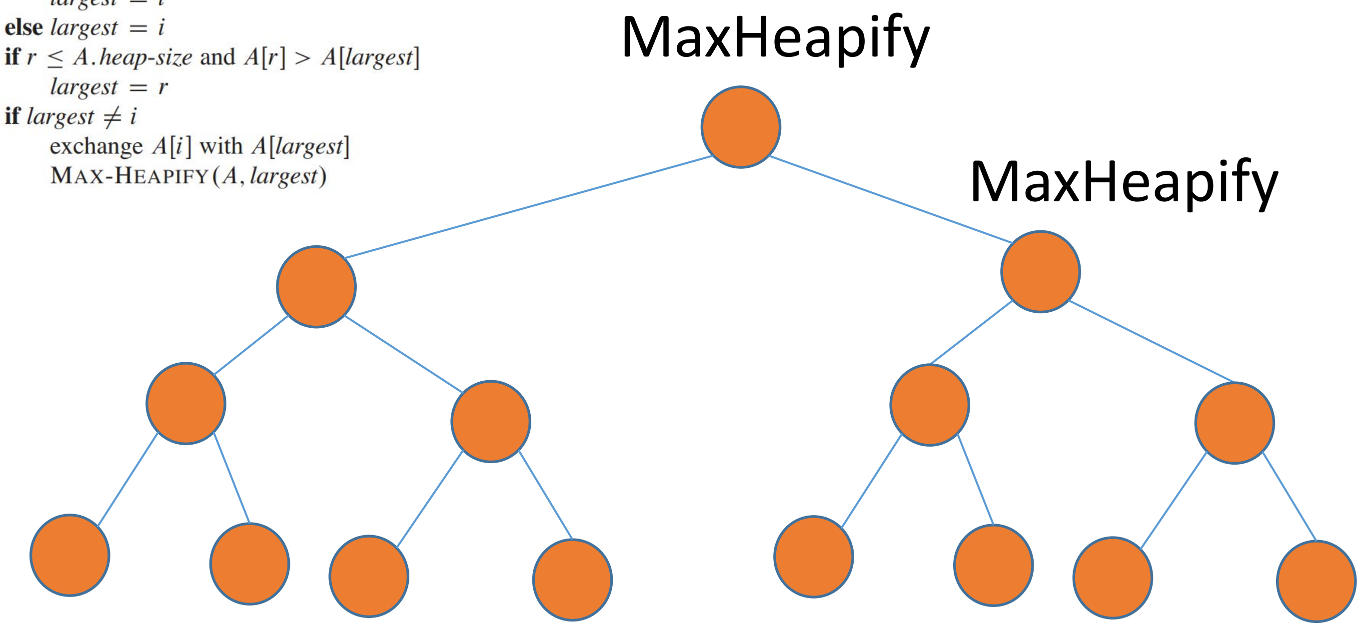
T(n)

Max Heapify may or may not be called 2 more times



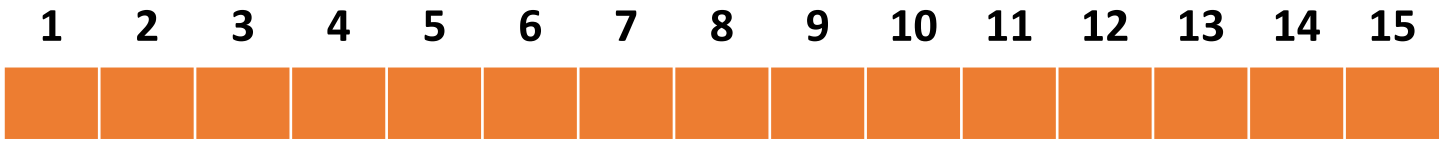
Time Complexity Analysis of Max Heapify

```
MAX-HEAPIFY(A, i)
 l = LEFT(i)
 r = RIGHT(i)
 if l ≤ A.heap-size and A[l] > A[i]
     largest = l
 else largest = i
 if r ≤ A.heap-size and A[r] > A[largest]
     largest = r
 if largest ≠ i
     exchange A[i] with A[largest]
     MAX-HEAPIFY(A, largest)
```



$T(n) <= T(n/2)+b$

Max Heapify may or may not be called 2 more times

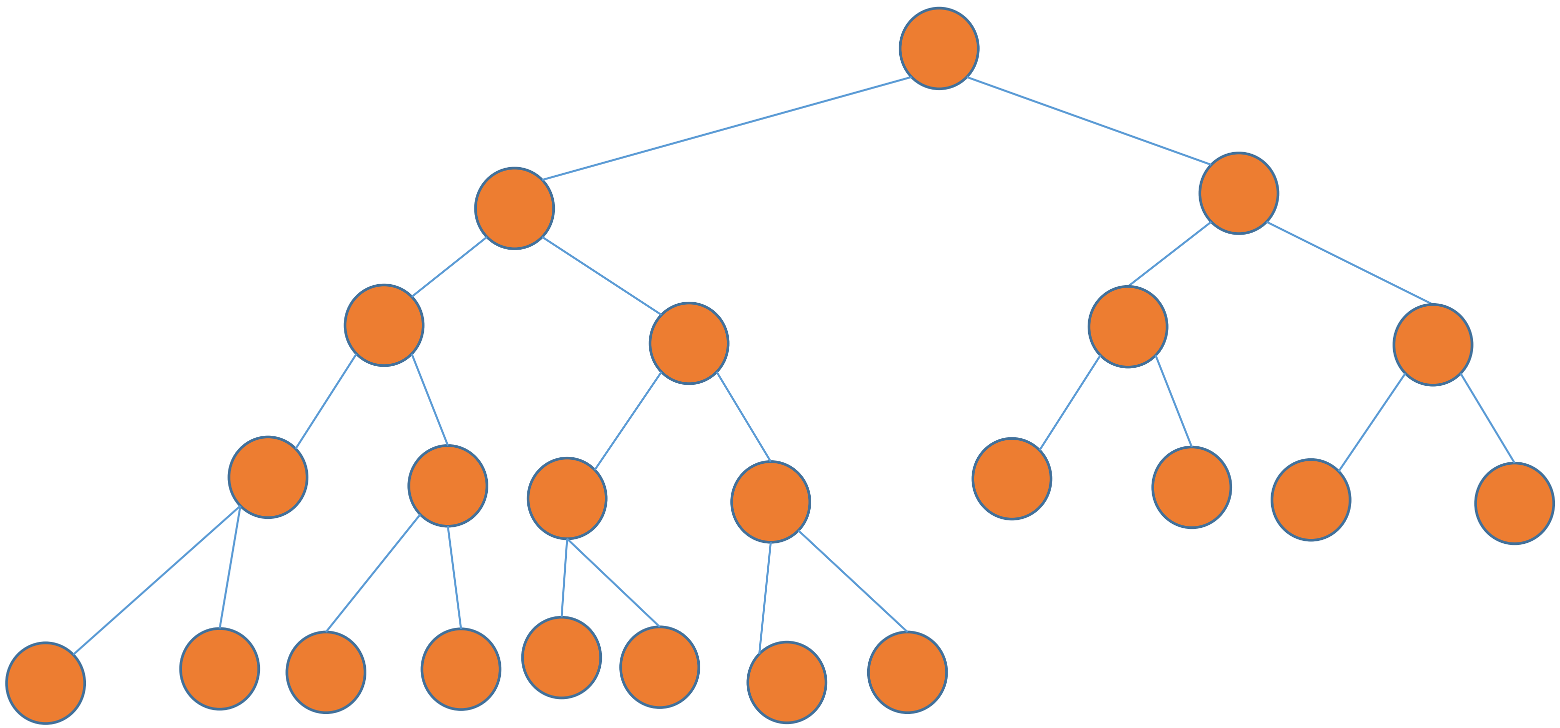


Time Complexity Analysis of Max Heapify

- That WAS NOT the worst case

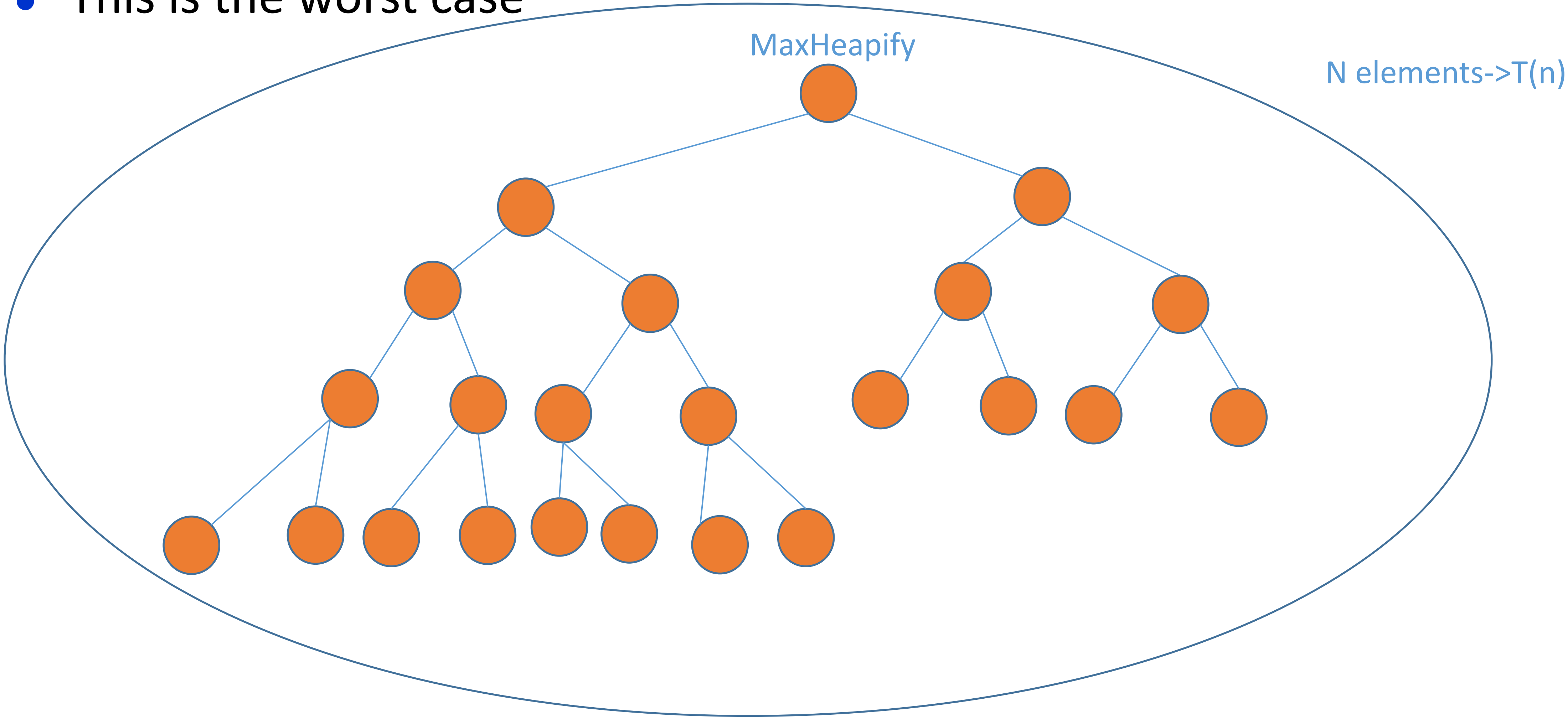
Time Complexity Analysis of Max Heapify

- This is the worst case



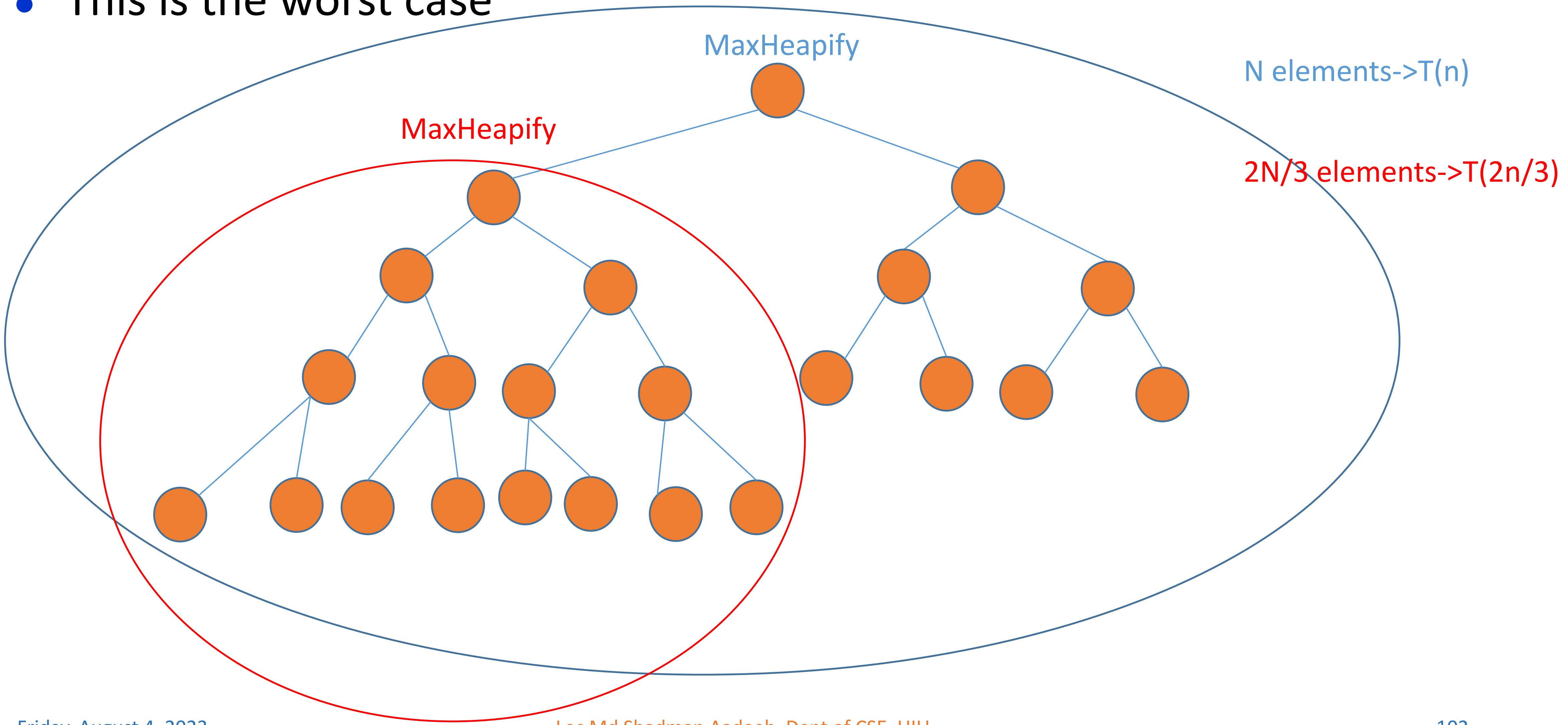
Time Complexity Analysis of Max Heapify

- This is the worst case



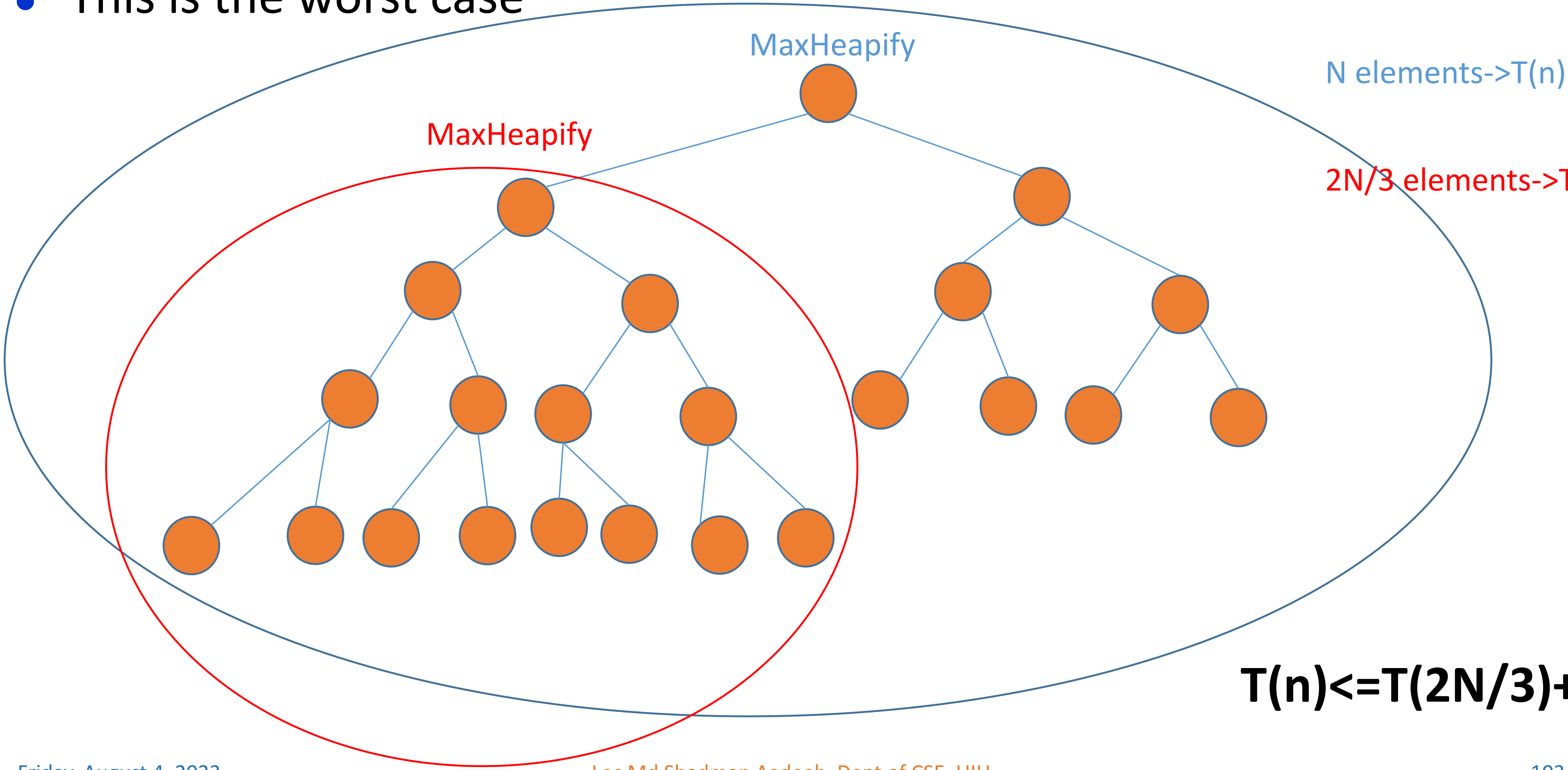
Time Complexity Analysis of Max Heapify

- This is the worst case



Time Complexity Analysis of Max Heapify

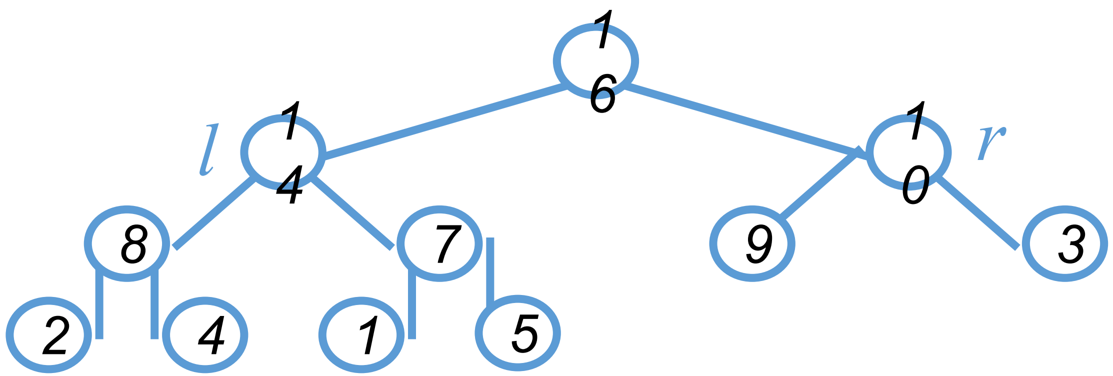
- This is the worst case



T(n)<=T(2N/3)+b

Time Complexity Analysis-Max Heapify

- **Max Heapify**
  - Fixing up relationships among the elements $A[i]$, $A[l]$, and $A[r]$ takes $\Theta(1)$ time
  - *If the heap at i has n elements, at most how many elements can the subtrees at l or r have?*




```
MAX-HEAPIFY(A, i)
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

- Answer: $2n/3$ (worst case: bottom row half full)
- So time taken by **`Heapify()`** is given by

  $T(n) \leq T(2n/3) + b$

Time Complexity Analysis-Max Heapify

- So we have

  $T(n) \leq T(2n/3) + \mathrm{b}$

- Solving the recurrence, we have

  $T(n) = O(\lg n)$

- Thus, **`Heapify()`** takes $O(h)$ time for a node at height h

Time Complexity Analysis-Build Max Heap

- We can compute a simple upper bound on the running time of BUILD-MAXHEAP as follows.
- Each call to MAX-HEAPIFY costs O(lg n) time, and BUILDMAX-HEAP makes **O(n) such calls.**
- Thus, the running time is **O(n lg n).**

```
BUILD-MAX-HEAP(A)
A.heap-size = A.length
for i = ⌊A.length/2⌋ downto 1
    MAX-HEAPIFY(A, i)
```

Time Complexity Analysis-Heap Sort

```
HEAPSORT(A)
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```

- The HEAPSORT procedure takes time O(n lg n)
- since the call to BUILD-MAXHEAP takes time O(n)
- Each of the n - 1 calls to MAX-HEAPIFY takes time O(lg n)

Start

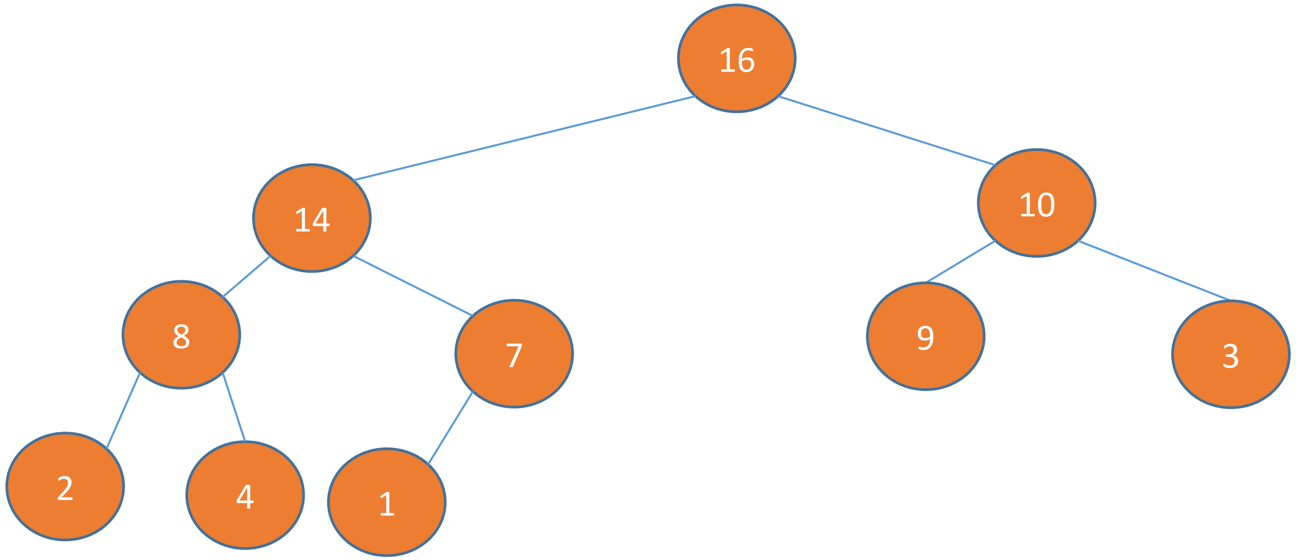
MAX HEAP AS A PRIORITY QUEUE

Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

Initially the array must be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

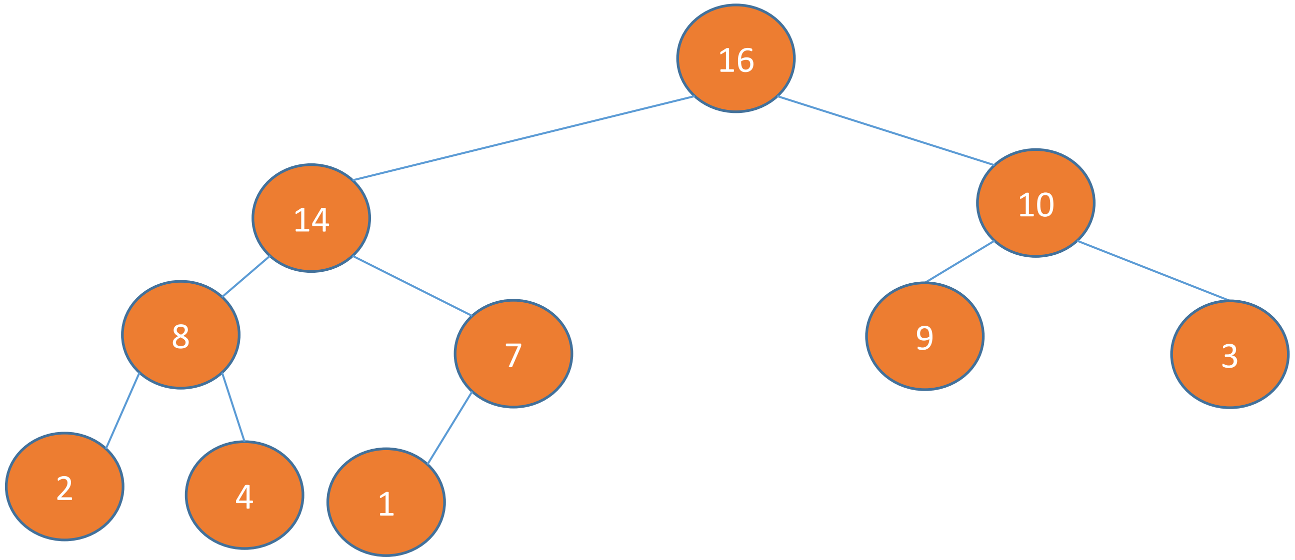


Max heap as a priority queue

- **Priority Queue functions**
  - **HEAP-MAXIMUM** – checks what is the age of the eldest citizen
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



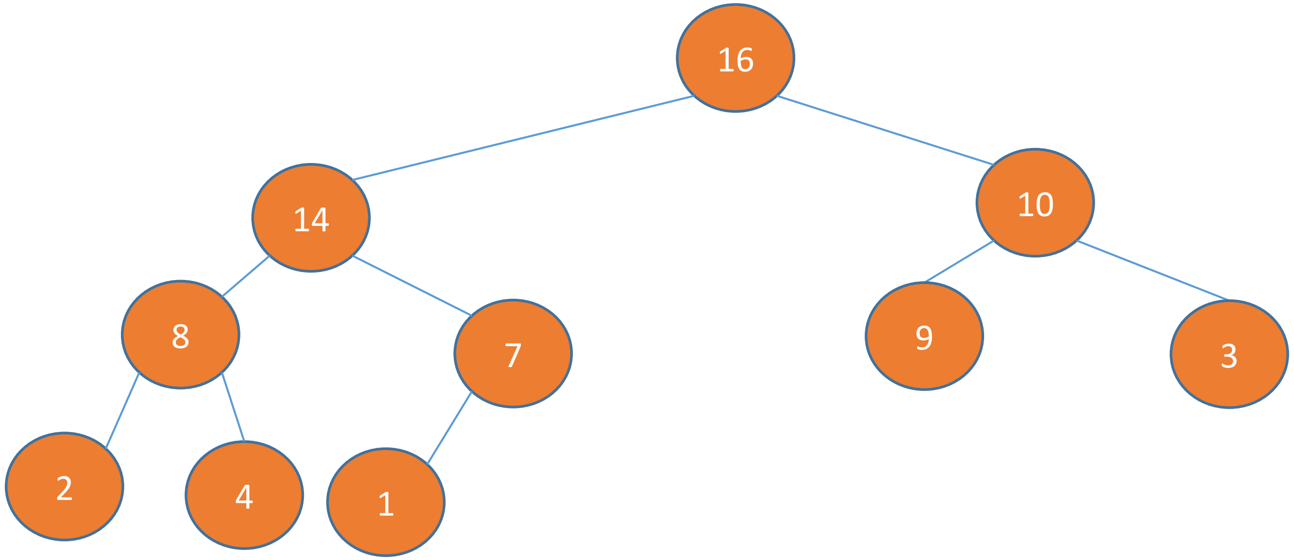
HEAP-MAXIMUM(A)
1 **return** $A[1]$

Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

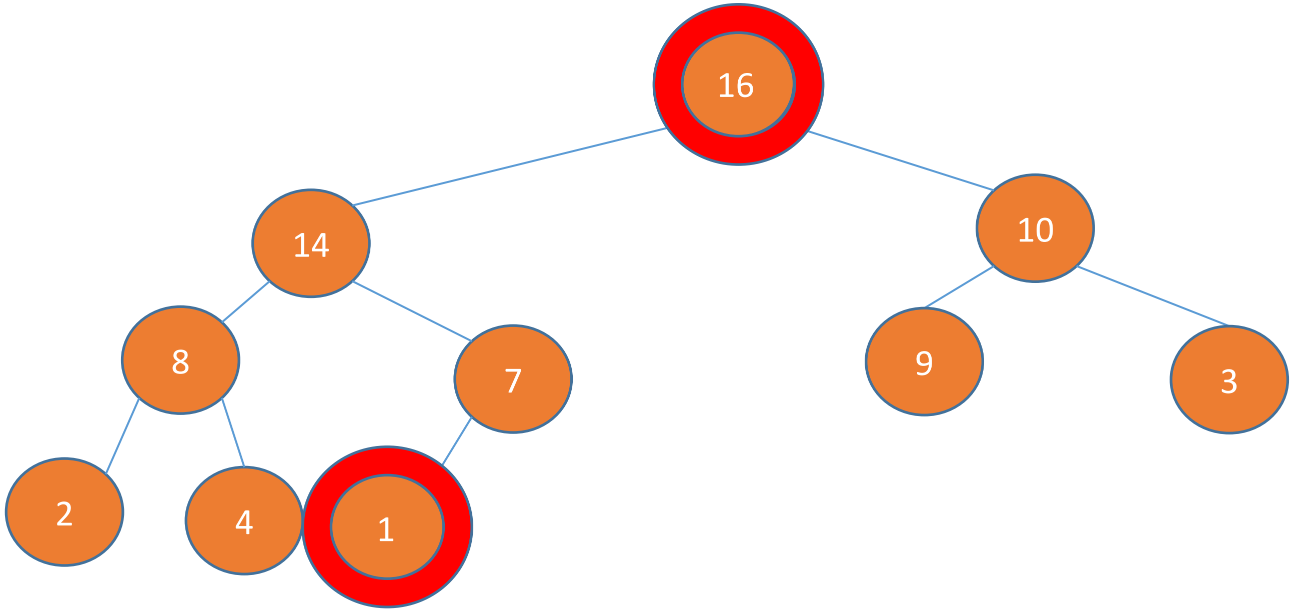


Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



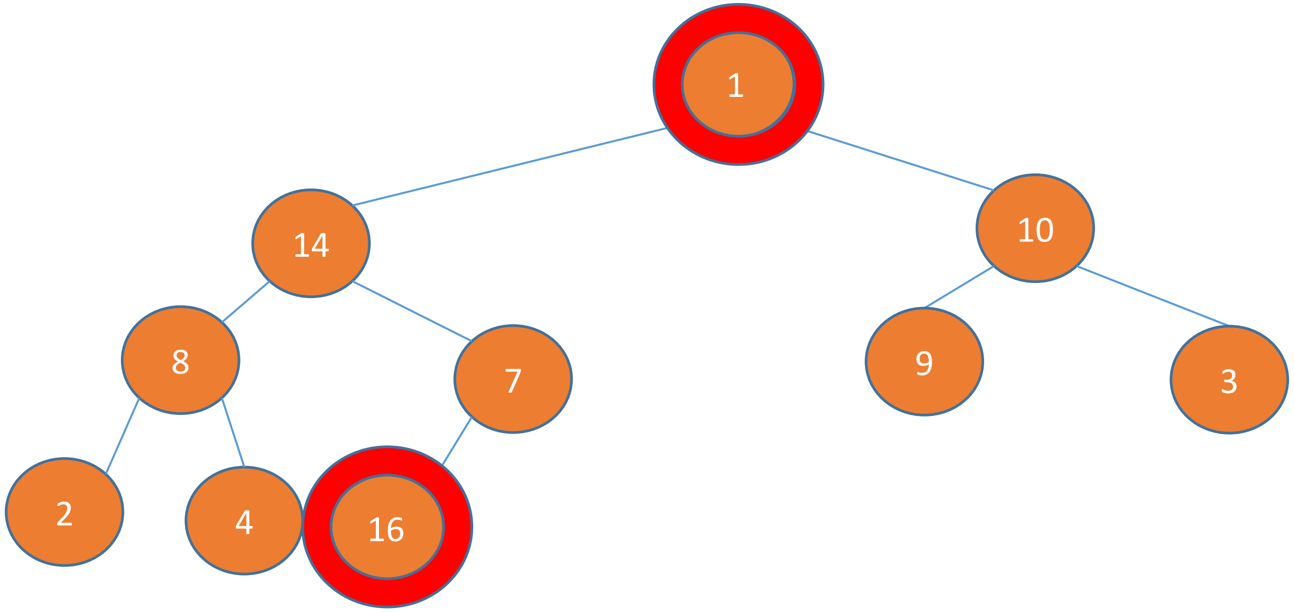
1.Swap the last element with the root

Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
1	14	10	8	7	9	3	2	4	16



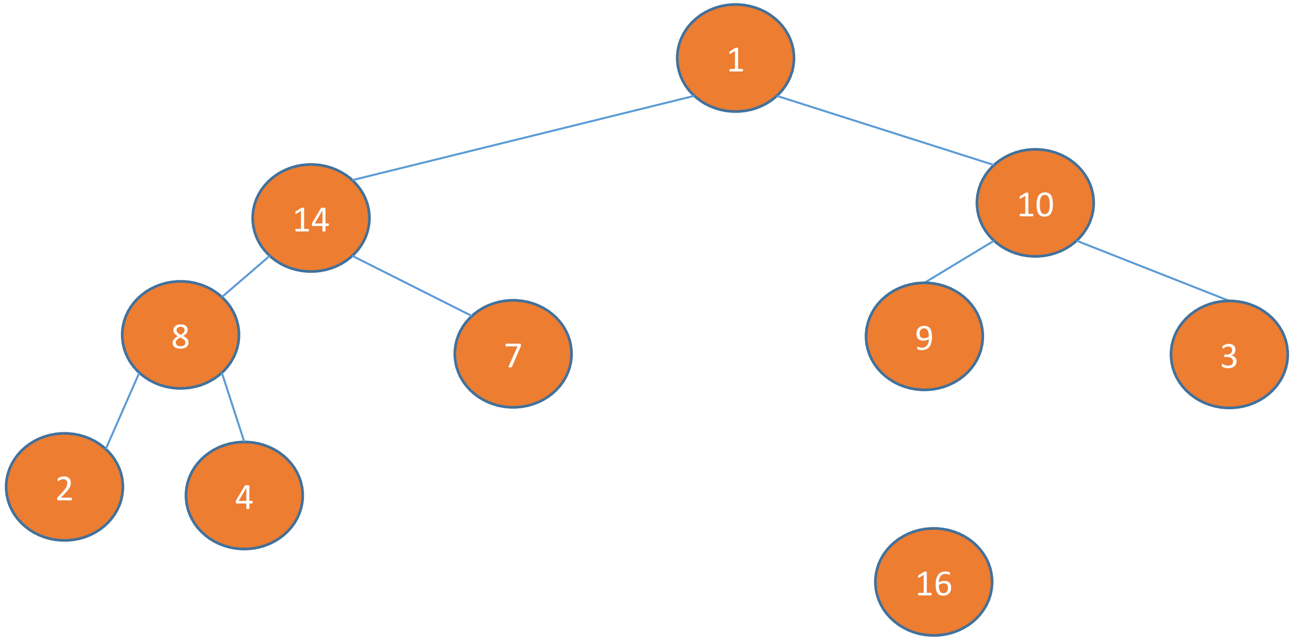
1.Swap the last element with the root

Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
1	14	10	8	7	9	3	2	4	16



1.Swap the last element with the root
2.Remove the largest element

Max heap as a priority queue

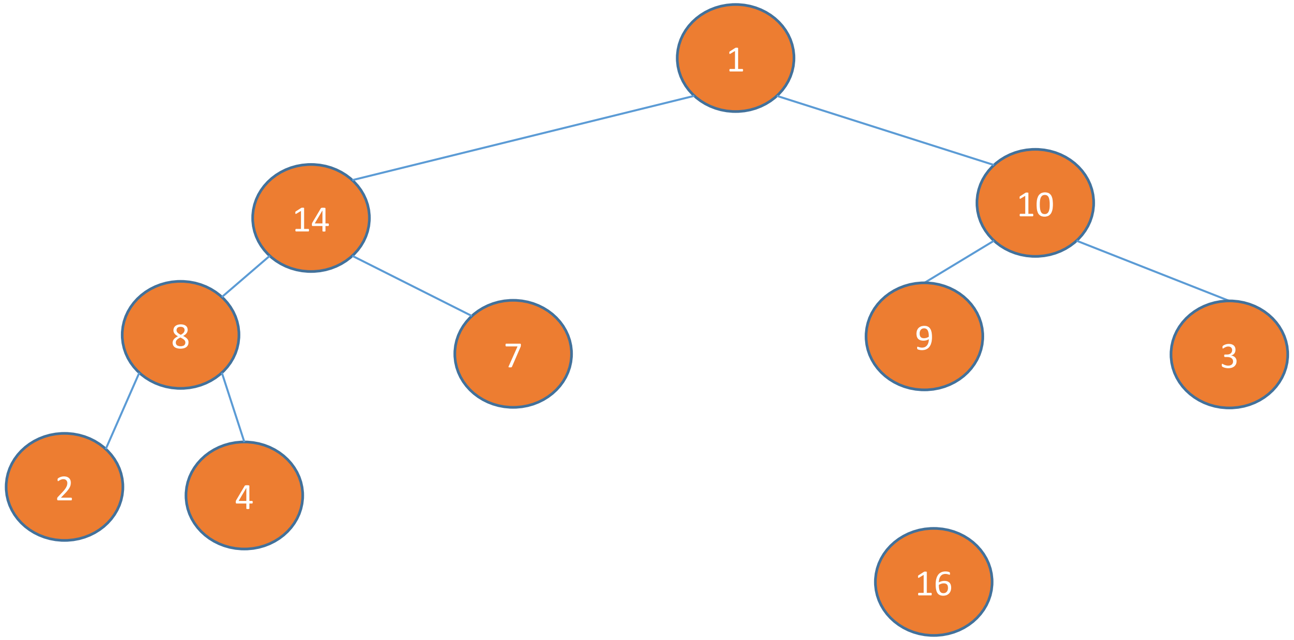
- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
1	14	10	8	7	9	3	2	4	16

Heap Size

1.Swap the last element with the root
2.Remove the largest element

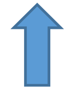


Max heap as a priority queue

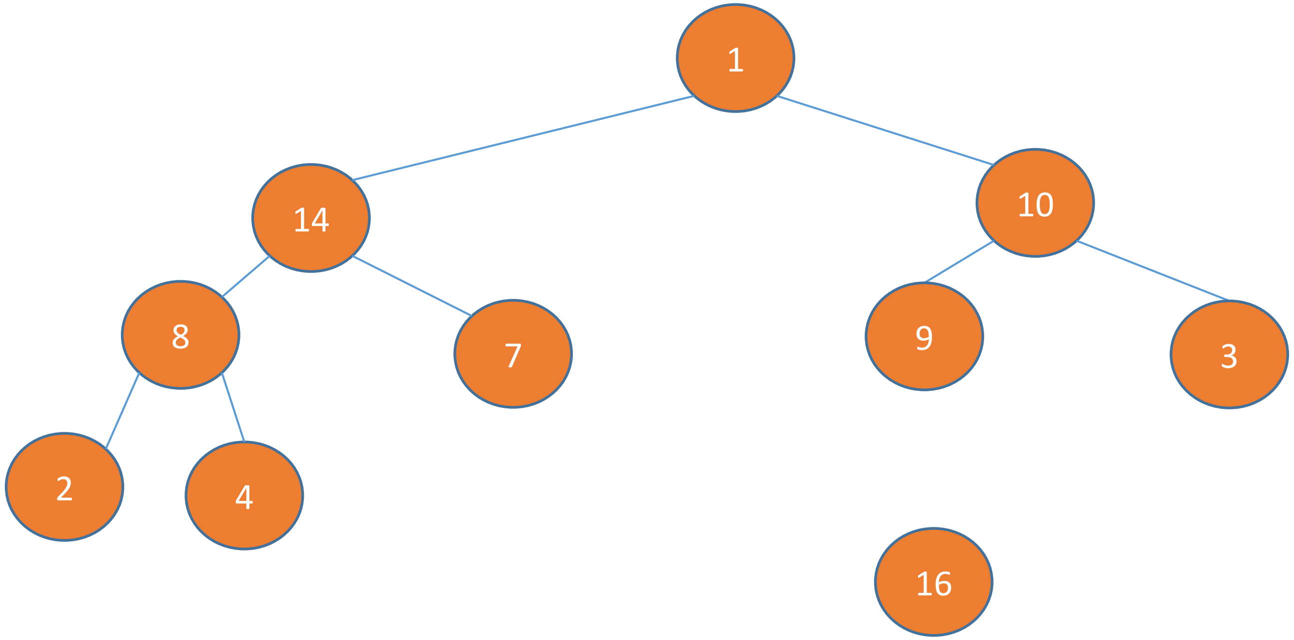
- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
1	14	10	8	7	9	3	2	4	16

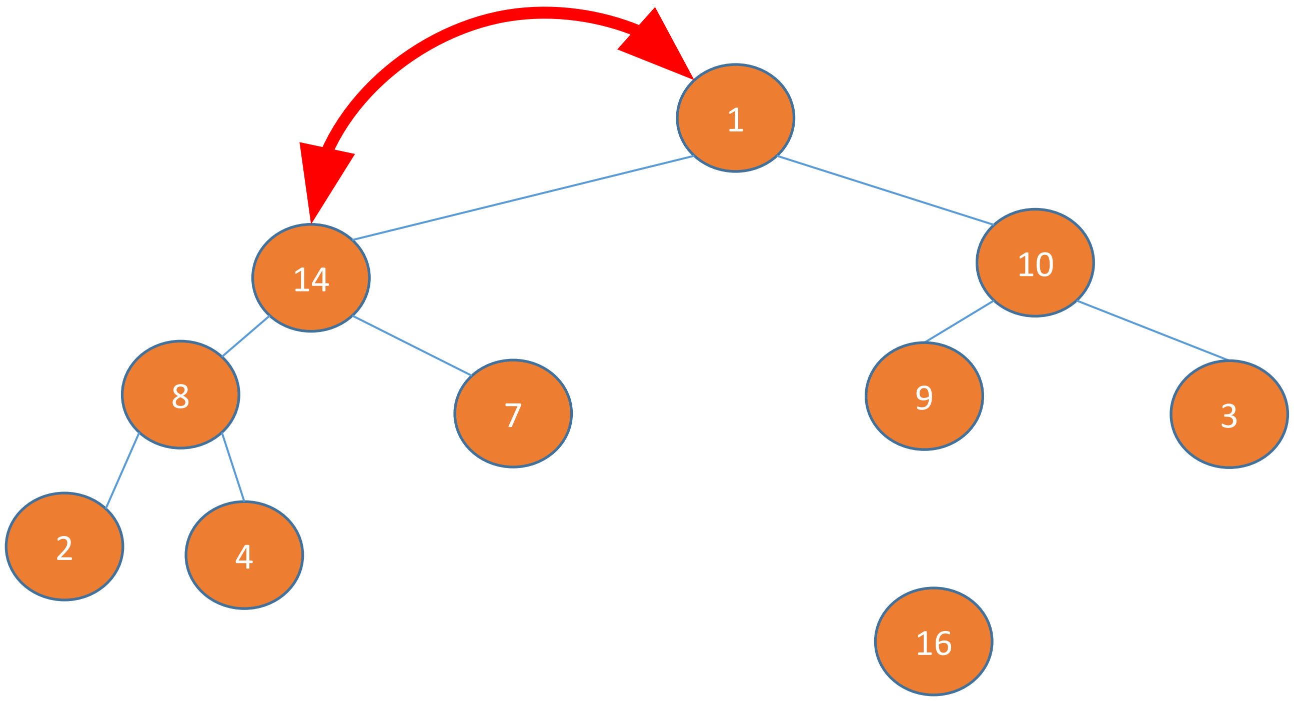


1.Swap the last element with the root
2.Remove the largest element
3.Run a max heapify



Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY



Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
1	14	10	8	7	9	3	2	4	16

Heap Size

1.Swap the last element with the root
2.Remove the largest element
3.Run a max heapify

Max heap as a priority queue

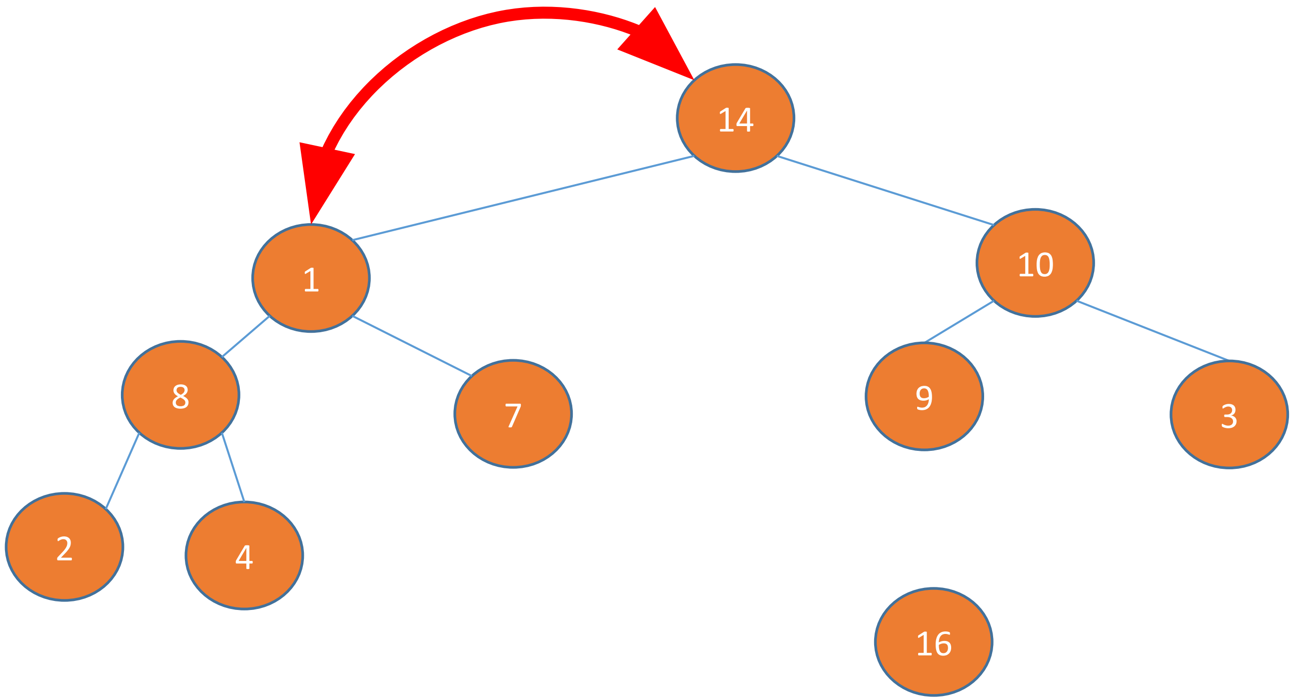
- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
14	1	10	8	7	9	3	2	4	16

Heap Size

1. **Swap the last element with the root**
2. **Remove the largest element**
3. **Run a max heapify**



Max heap as a priority queue

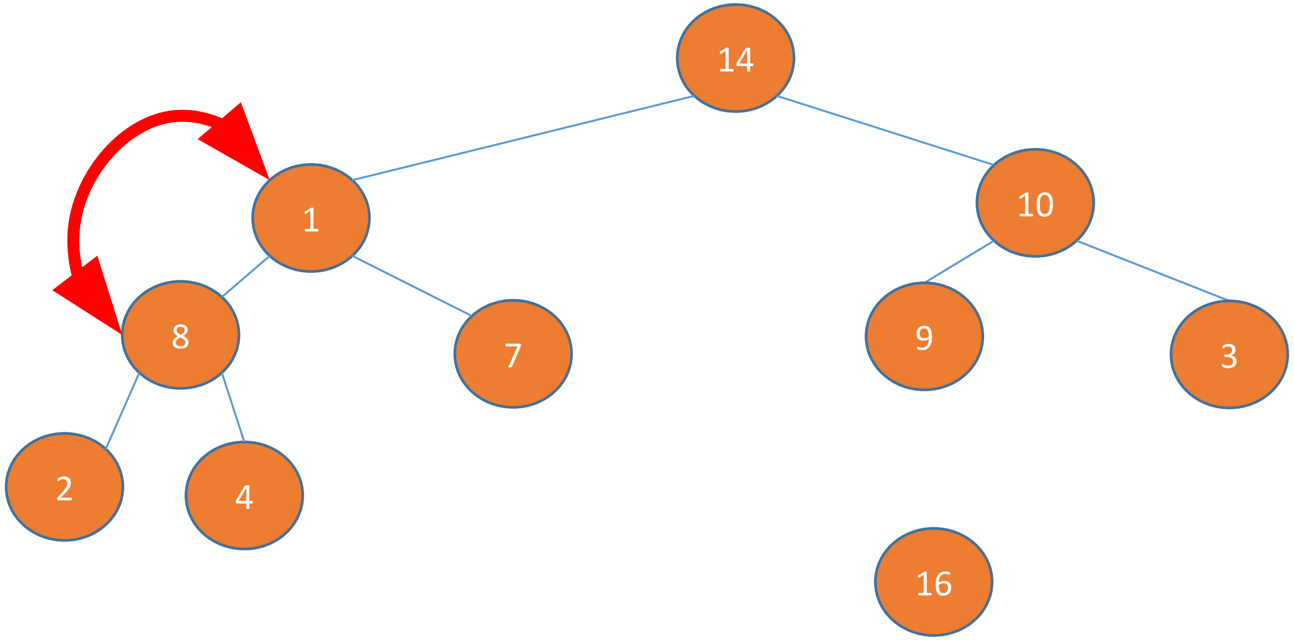
- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
14	1	10	8	7	9	3	2	4	16

Heap Size

1. **Swap the last element with the root**
2. **Remove the largest element**
3. **Run a max heapify**



Max heap as a priority queue

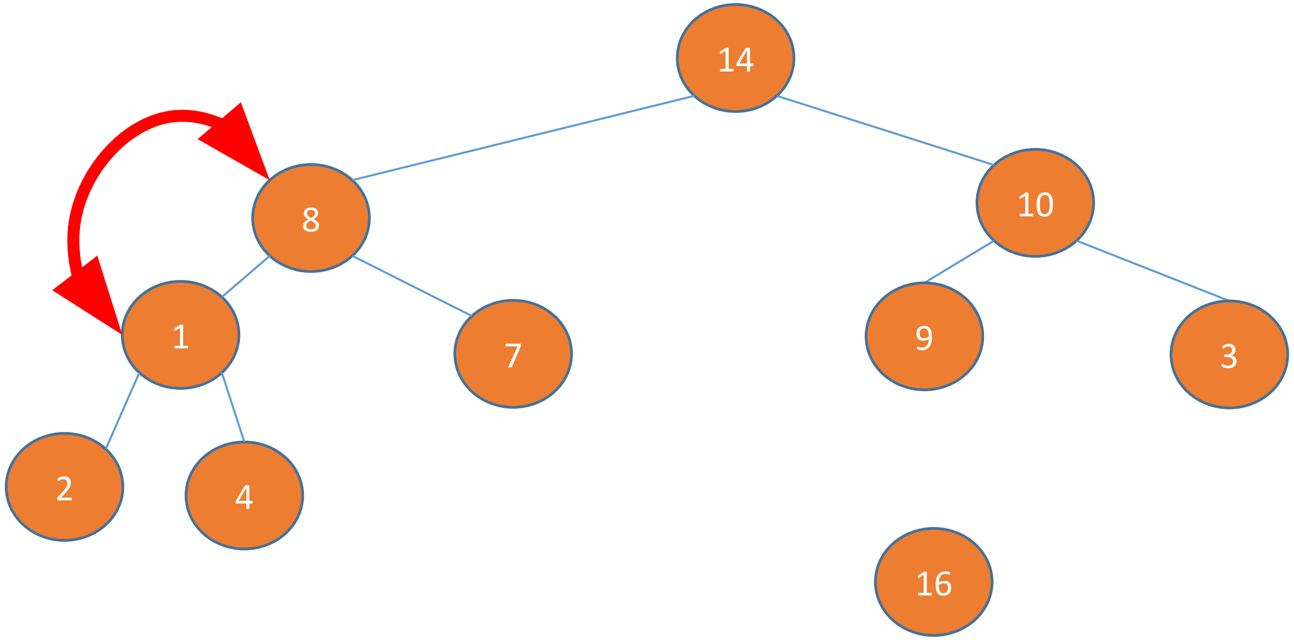
- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
14	8	10	1	7	9	3	2	4	16

Heap Size

1.Swap the last element with the root
2.Remove the largest element
3.Run a max heapify



Max heap as a priority queue

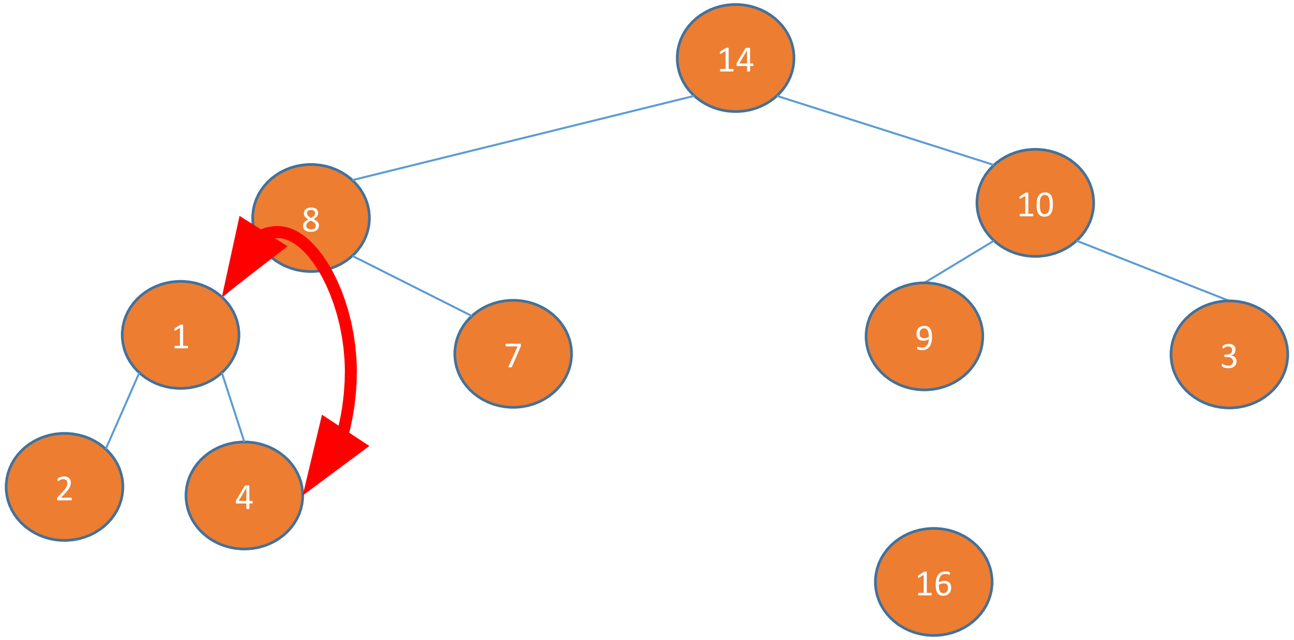
- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
14	8	10	1	7	9	3	2	4	16

Heap Size

1.Swap the last element with the root
2.Remove the largest element
3.Run a max heapify



Max heap as a priority queue

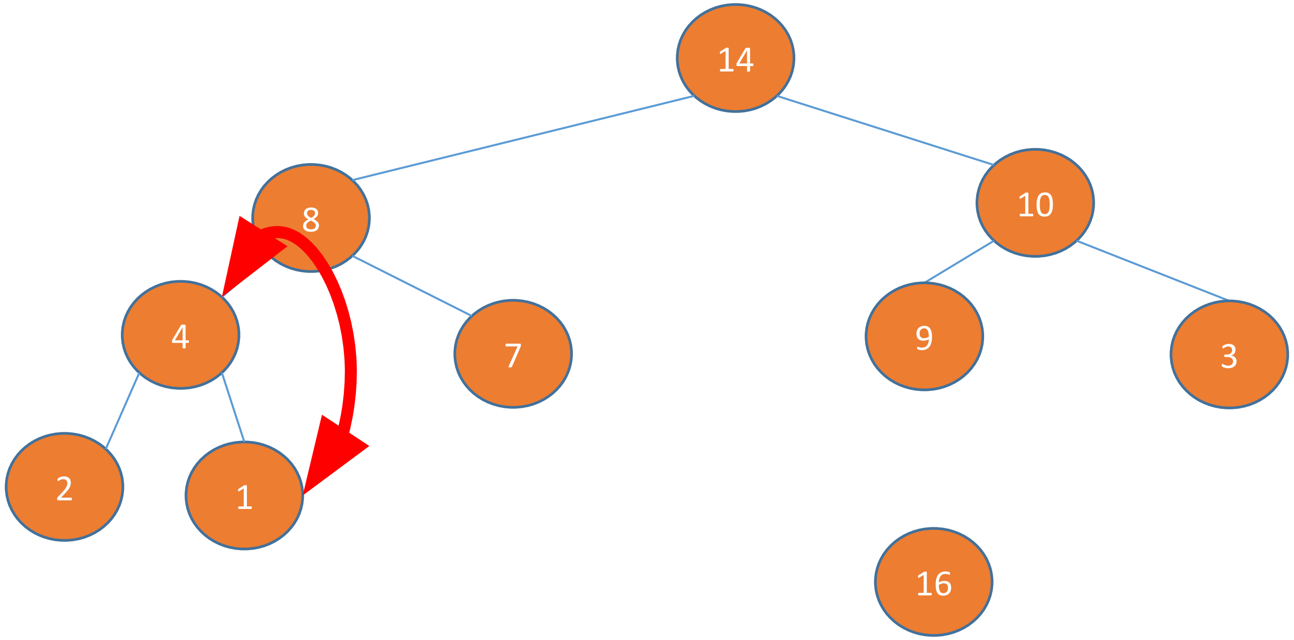
- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
14	8	10	4	7	9	3	2	1	16

Heap Size

1.Swap the last element with the root
2.Remove the largest element
3.Run a max heapify



Max heap as a priority queue

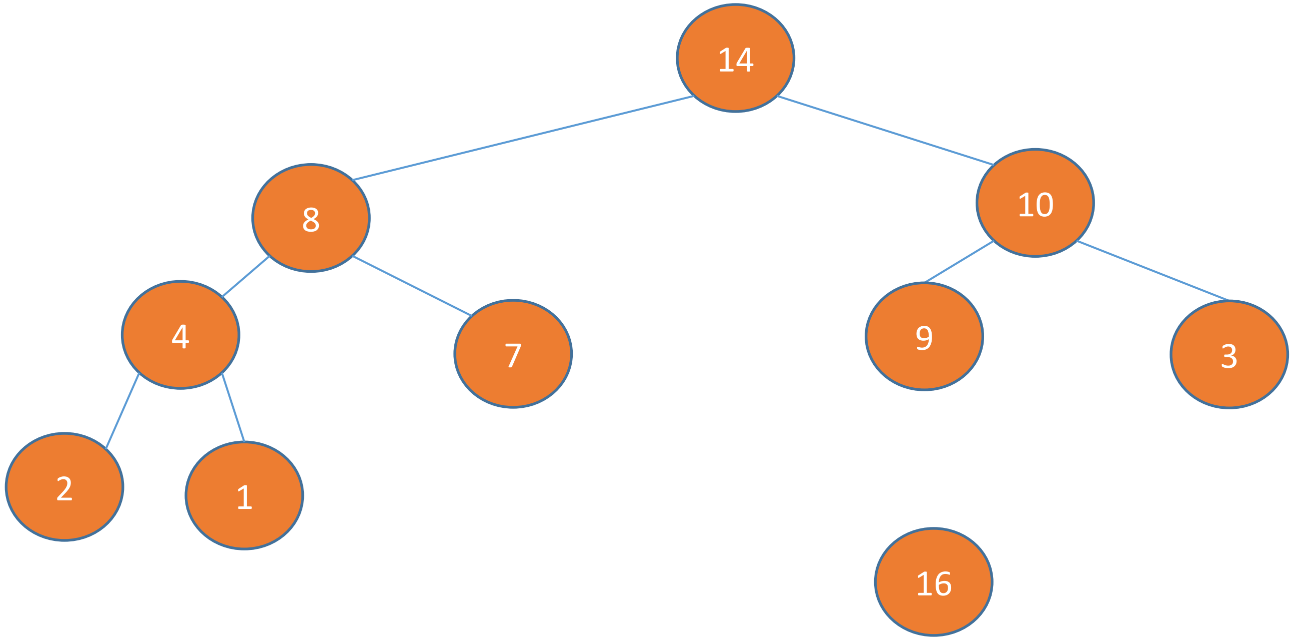
- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
14	8	10	4	7	9	3	2	1	16

Heap Size

1.Swap the last element with the root
2.Remove the largest element
3.Run a max heapify
Observe that it is still a max heap



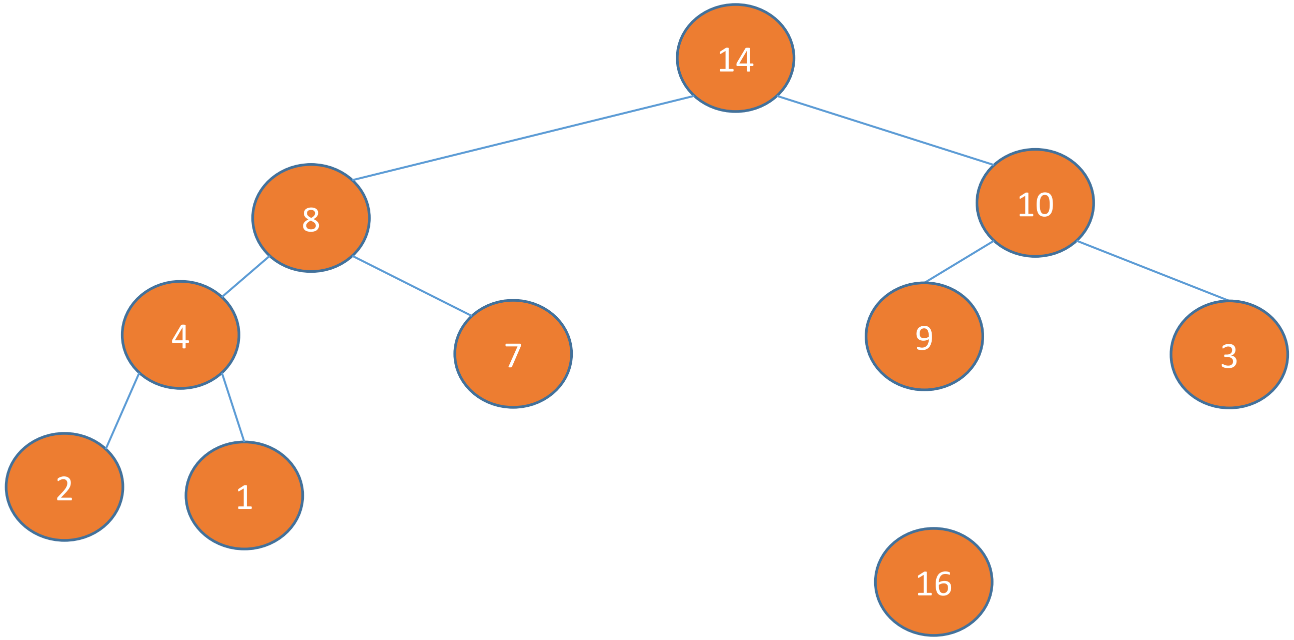
Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
14	8	10	4	7	9	3	2	1	16

Heap Size



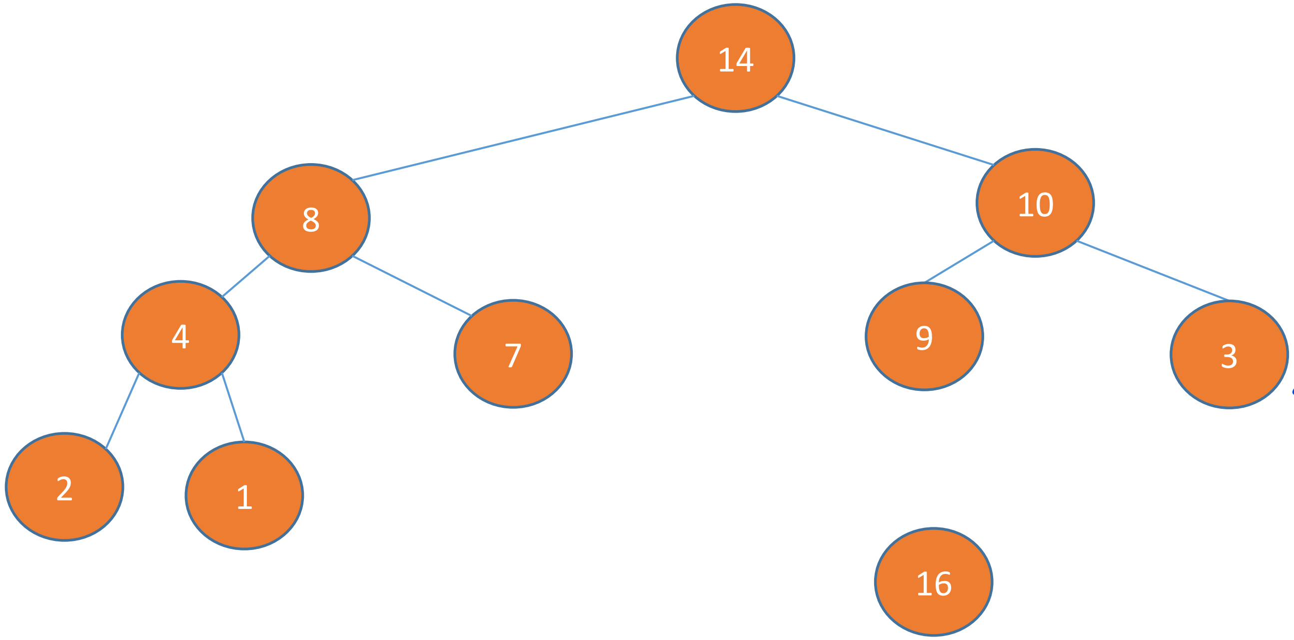
1.Swap the last element with the root
2.Remove the largest element
3.Run a max heapify
Observe that it is still a max heap
4. Return A[heapsize+1]

Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
14	8	10	4	7	9	3	2	1	16



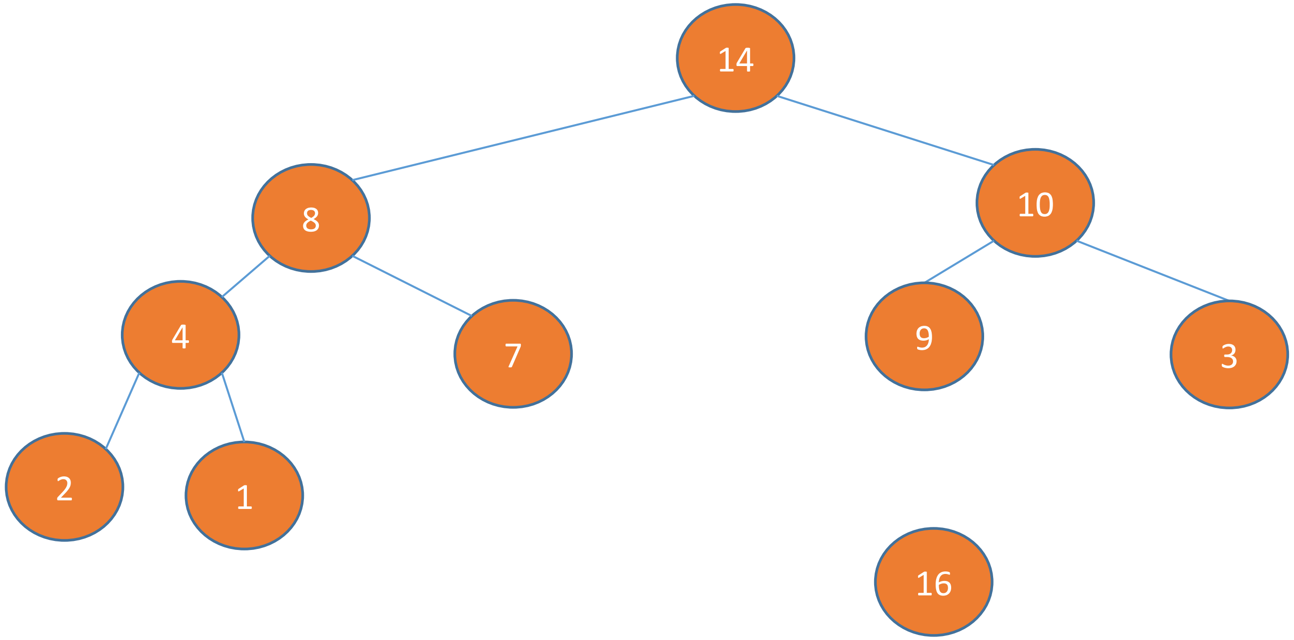

```
HEAP-EXTRACT-MAX(A)
if A.heap-size < 1
    error "heap underflow"
max = A[1]
A[1] = A[A.heap-size]
A.heap-size = A.heap-size − 1
MAX-HEAPIFY(A, 1)
return max
```

Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - **HEAP-EXTRACT-MAX –** extract the eldest citizen
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
14	8	10	4	7	9	3	2	1	16

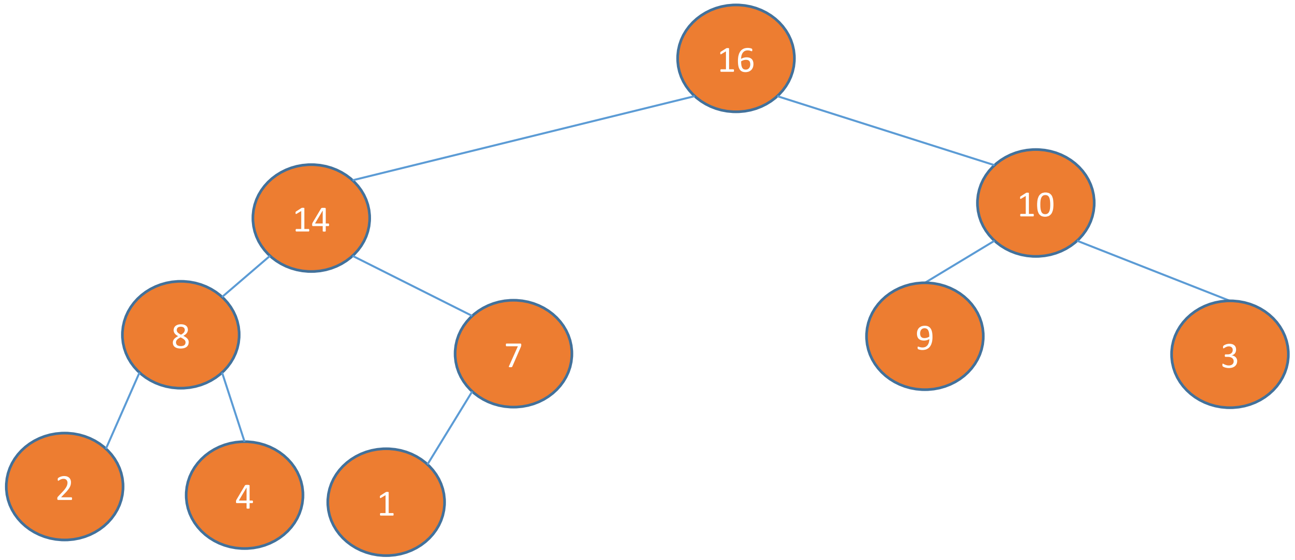


Takes O(logn time)

```
HEAP-EXTRACT-MAX(A)
if A.heap-size < 1
    error "heap underflow"
max = A[1]
A[1] = A[A.heap-size]
A.heap-size = A.heap-size − 1
MAX-HEAPIFY(A, 1)   O(logn)
return max
```

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

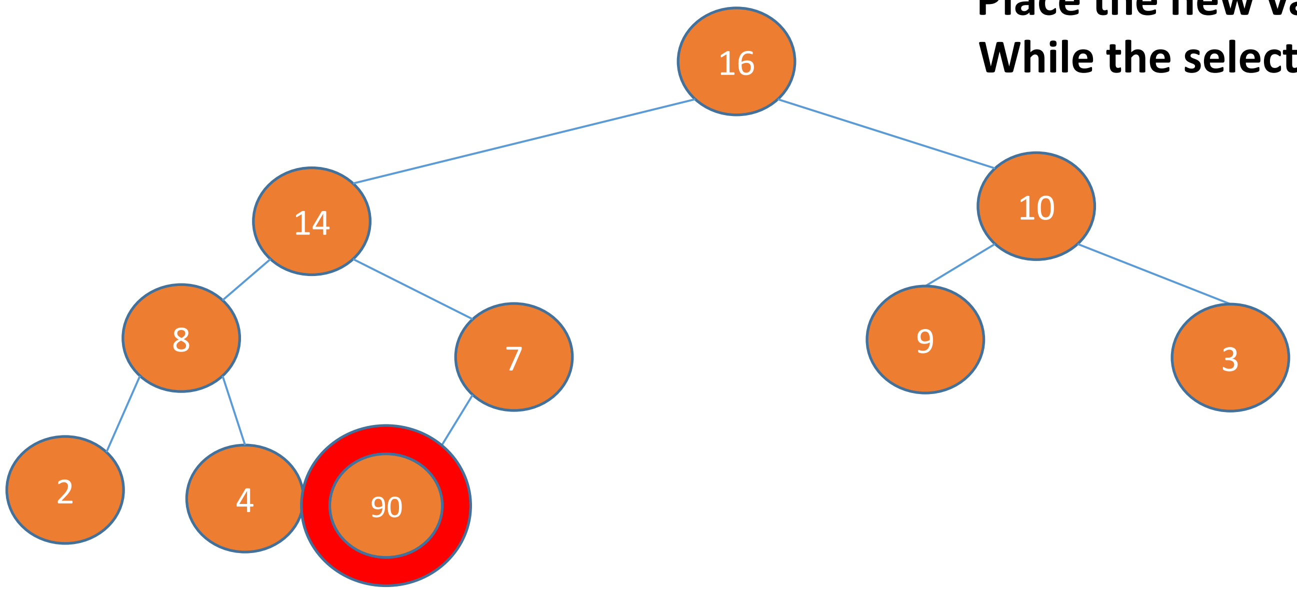
1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



HeapIncreaseKey(A,10,90)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



Place the new value at the index

While the selected index is not a root and is greater than parent

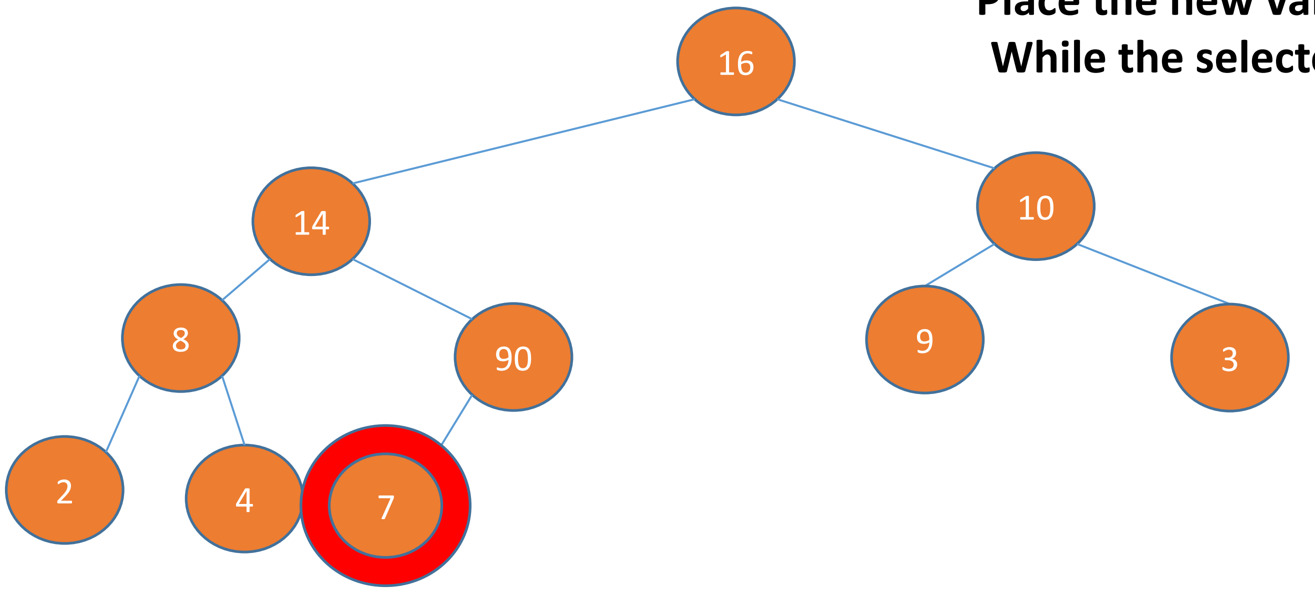
Change the values with parent

The parent is now the selected index

HeapIncreaseKey(A,10,90)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



Place the new value at the index

While the selected index is not a root and is greater than parent

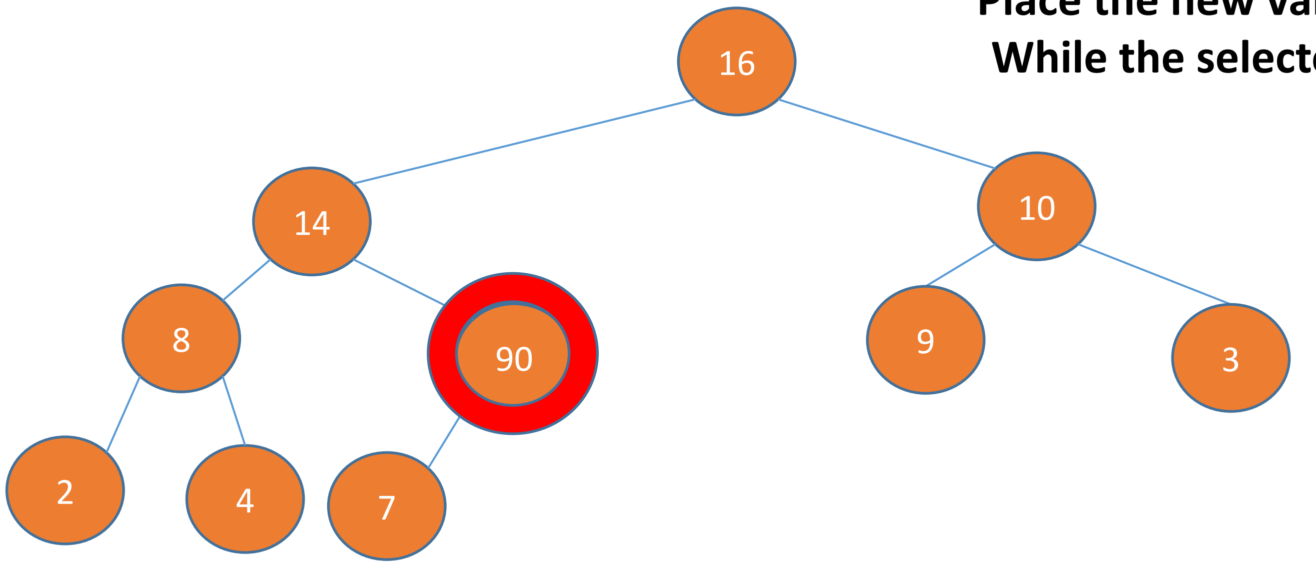
Change the values with parent

The parent is now the selected index

HeapIncreaseKey(A,10,90)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

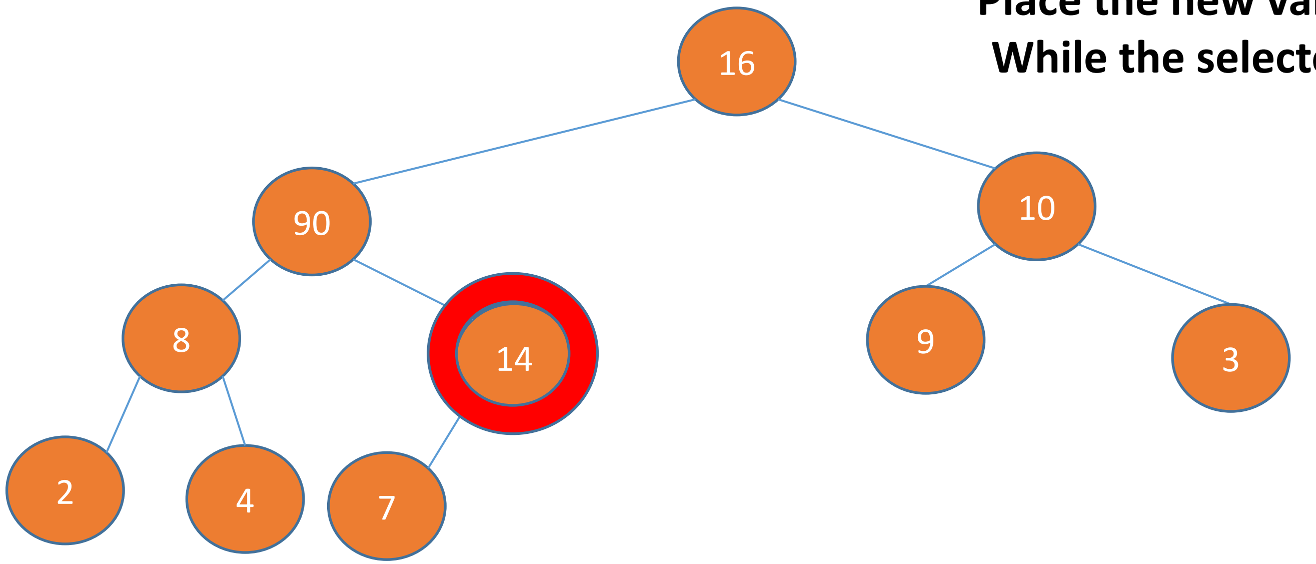


Place the new value at the index
While the selected index is not a root and is greater than parent
Change the values with parent
The parent is now the selected index

HeapIncreaseKey(A,10,90)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



Place the new value at the index

While the selected index is not a root and is greater than parent

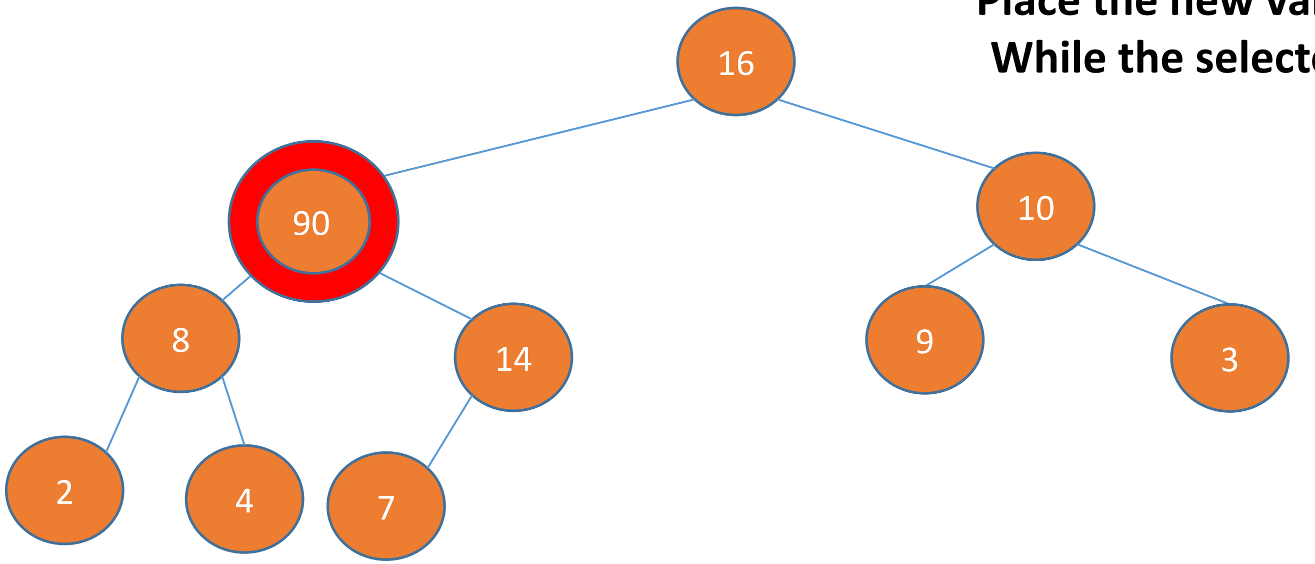
Change the values with parent

The parent is now the selected index

HeapIncreaseKey(A,10,90)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



Place the new value at the index

While the selected index is not a root and is greater than parent

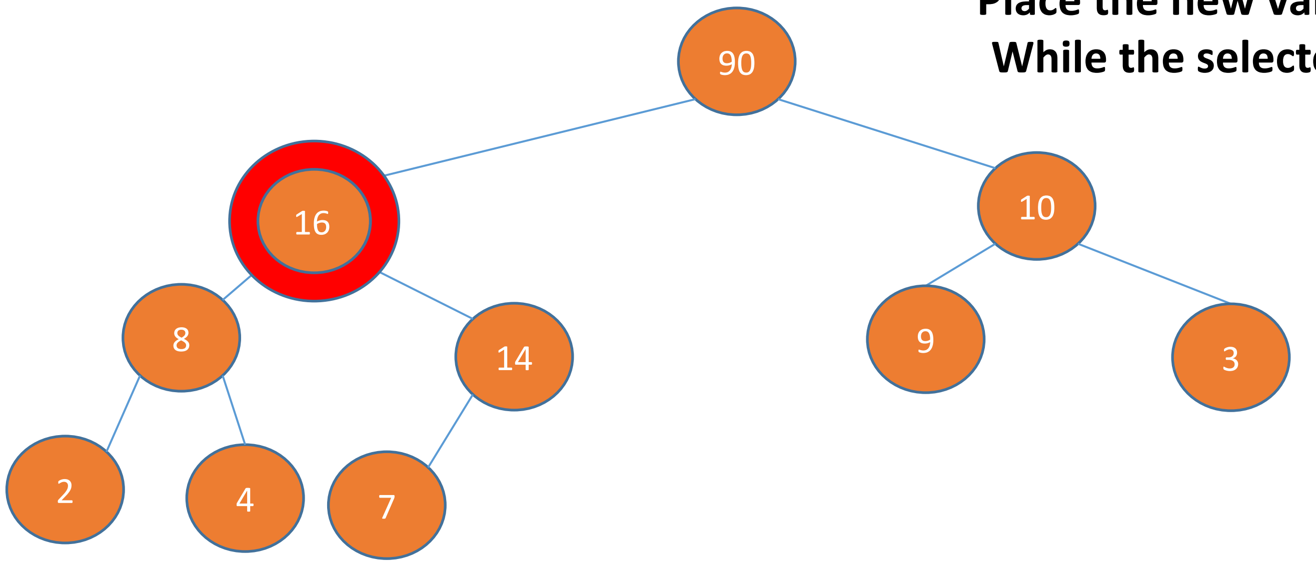
Change the values with parent

The parent is now the selected index

HeapIncreaseKey(A,10,90)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

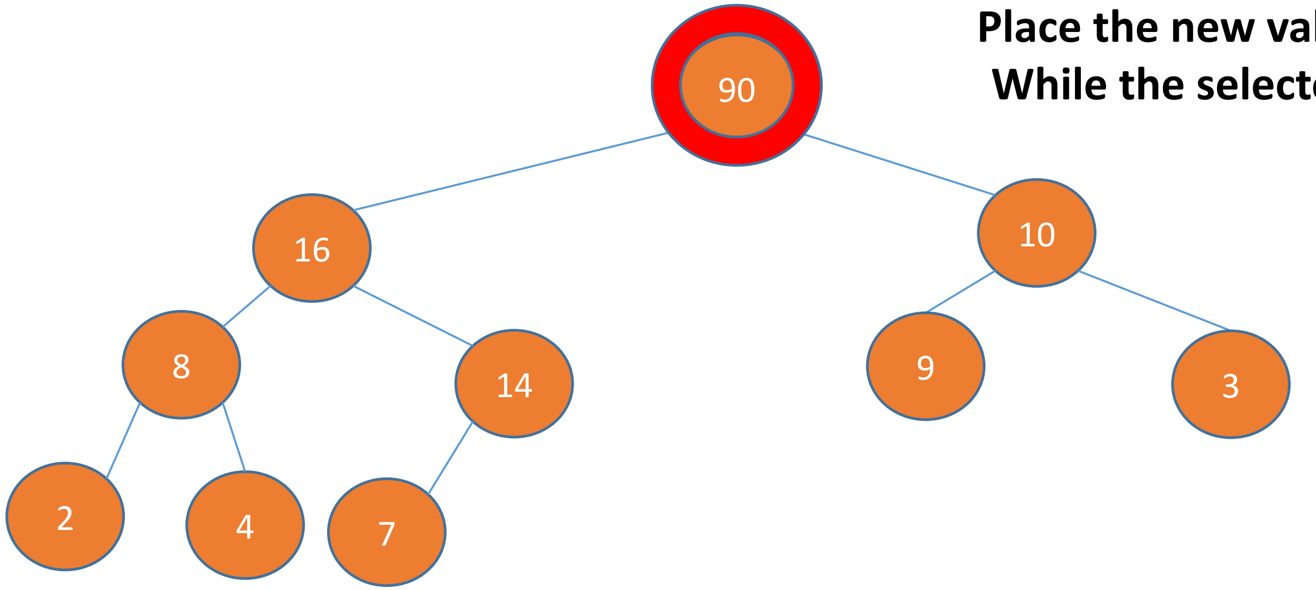


Place the new value at the index
While the selected index is not a root and is greater than parent
Change the values with parent
The parent is now the selected index

HeapIncreaseKey(A,10,90)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



Place the new value at the index

While the selected index is not a root and is greater than parent

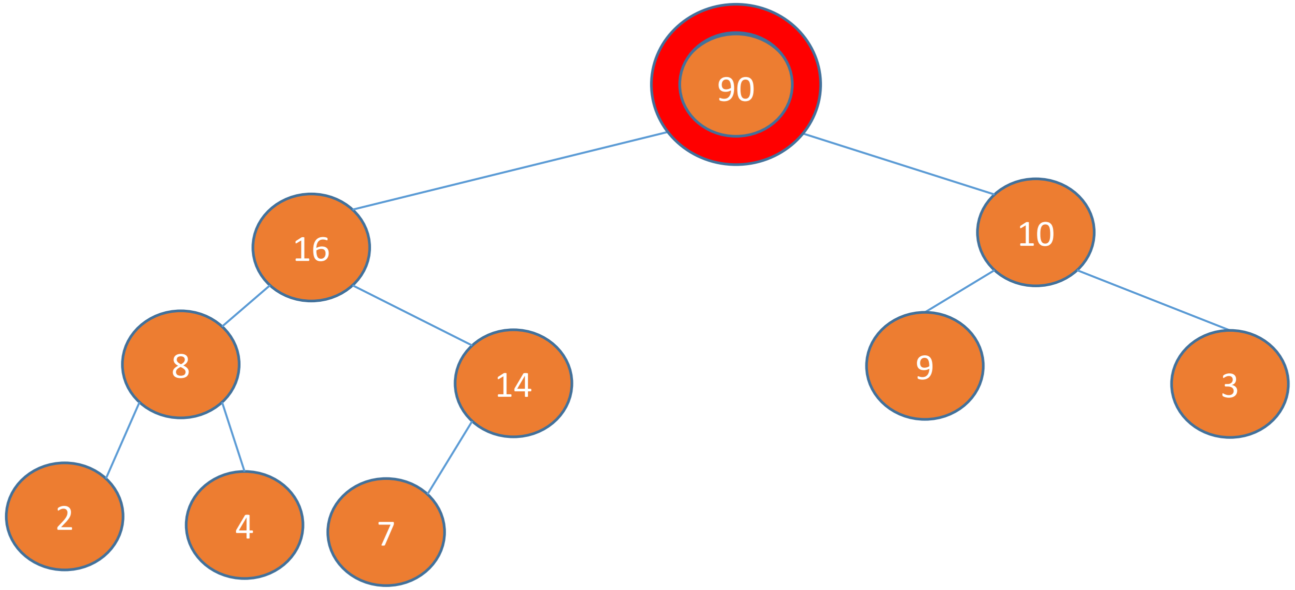
Change the values with parent

The parent is now the selected index

HeapIncreaseKey(A,10,90)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



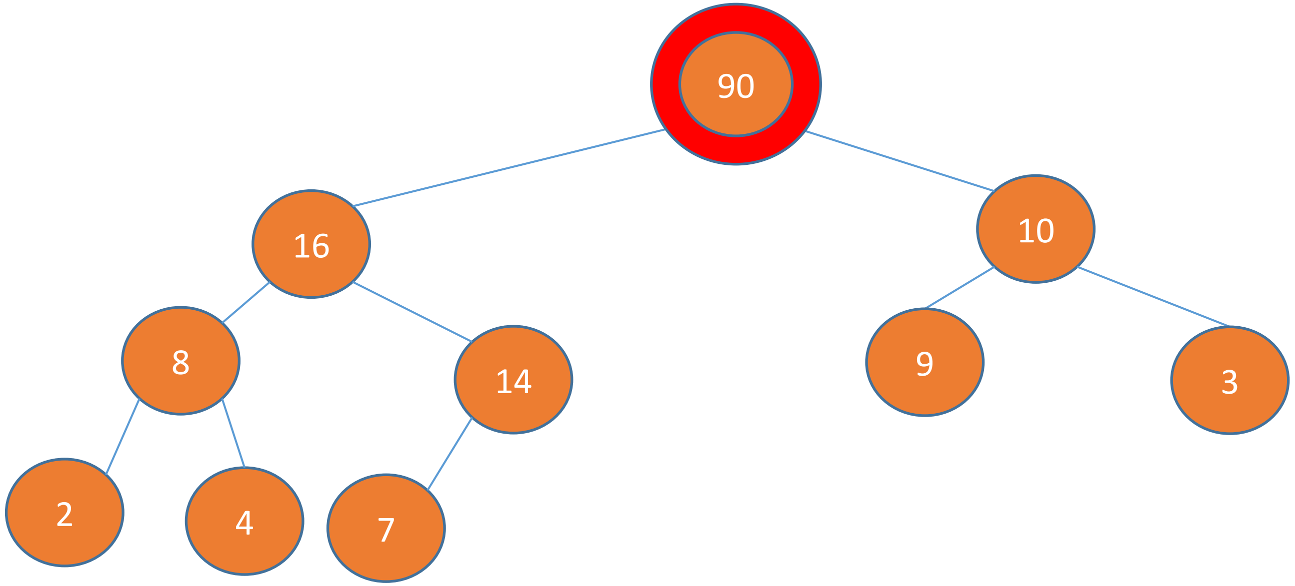
HEAP-INCREASE-KEY(A, i, key)

1 **if** $key < A[i]$
2 **error** "new key is smaller than current key"
3 $A[i] = key$
4 **while** $i > 1$ and $A[\text{PARENT}(i)] < A[i]$
5 exchange $A[i]$ with $A[\text{PARENT}(i)]$
6 $i = \text{PARENT}(i)$

HeapIncreaseKey(A,10,90)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - MAX-HEAP-INSERT
  - HEAP-EXTRACT-MAX
  - **HEAP-INCREASE-KEY –** increase the value of a particular index

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1




```
HEAP-INCREASE-KEY(A, i, key)
if key < A[i]
    error "new key is smaller than current key"
A[i] = key
while i > 1 and A[PARENT(i)] < A[i]
    exchange A[i] with A[PARENT(i)]
    i = PARENT(i)
```

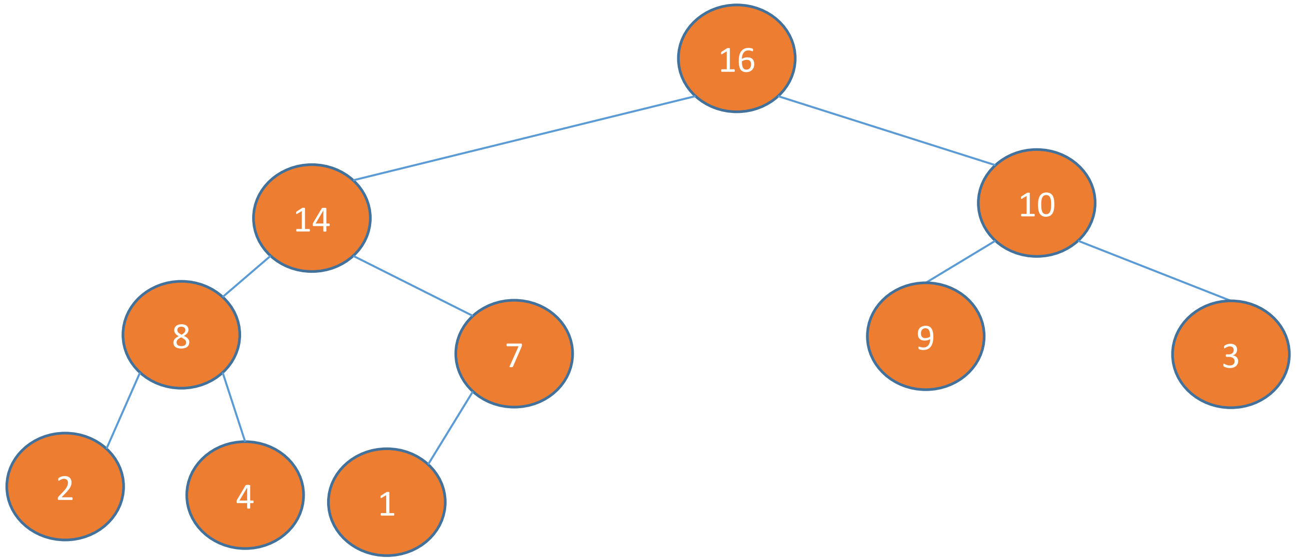
Climbs the height of the tree – O(logn) time

Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - **MAX-HEAP-INSERT**
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

Initially the array must
be a max heap (run a max heapify)

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1

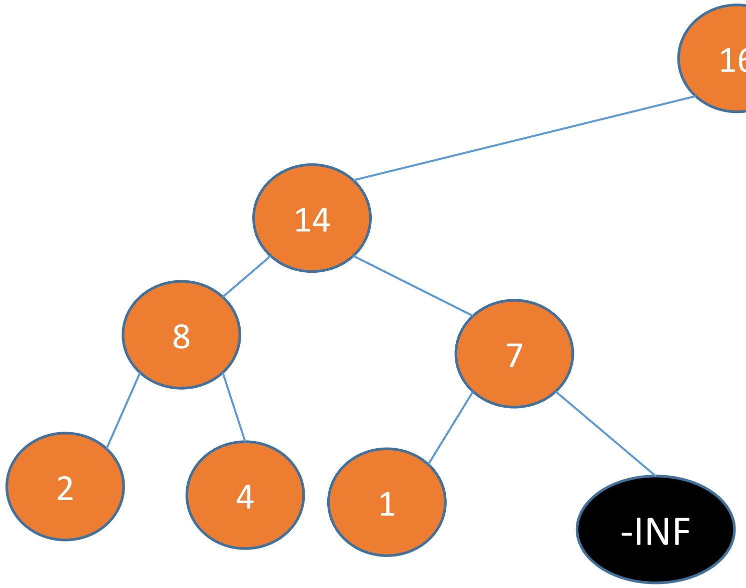


Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - **MAX-HEAP-INSERT**
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

Heap Increase Key (A, heapsize, new value)

1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	-inf

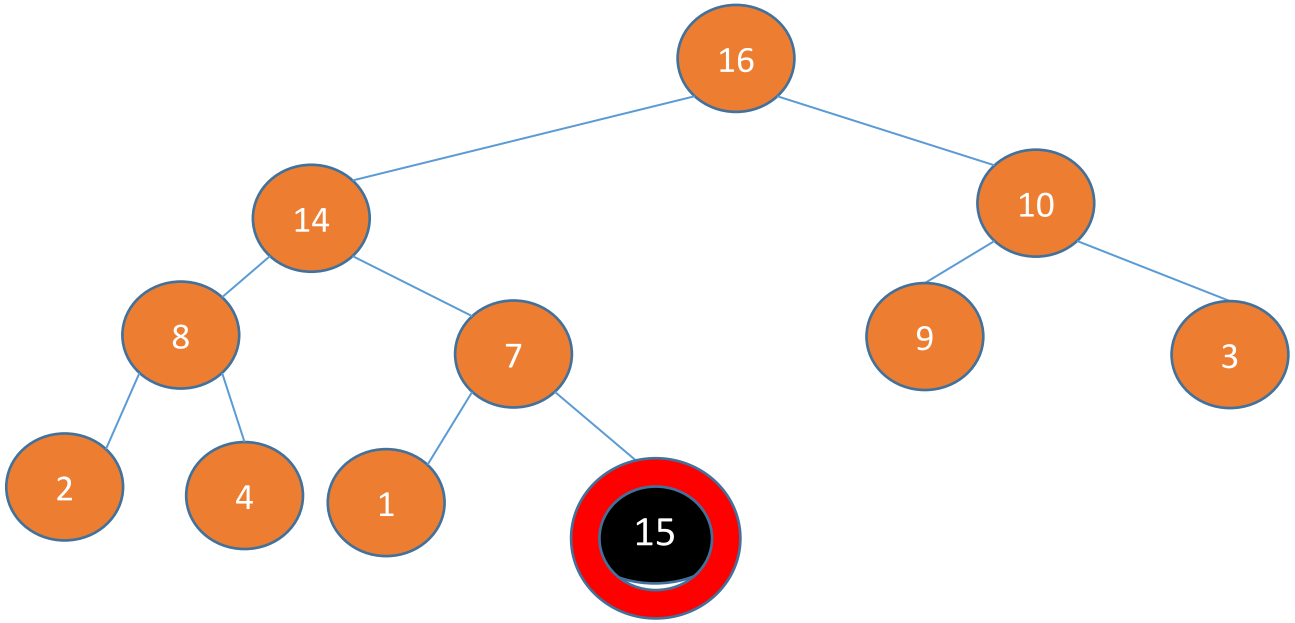


Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - **MAX-HEAP-INSERT**
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

Heap Increase Key (A, heapsize, new value)

1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	-inf

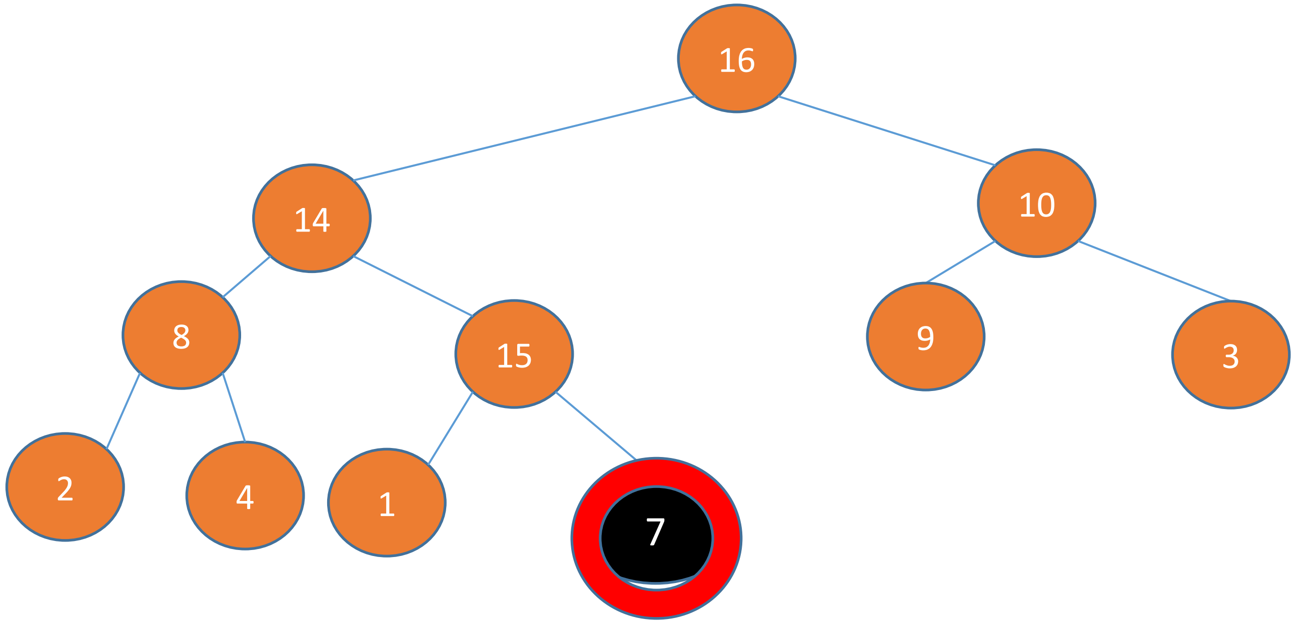


Max heap as a priority queue

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - **MAX-HEAP-INSERT**
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

Heap Increase Key (A, heapsize, new value)

1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	-inf

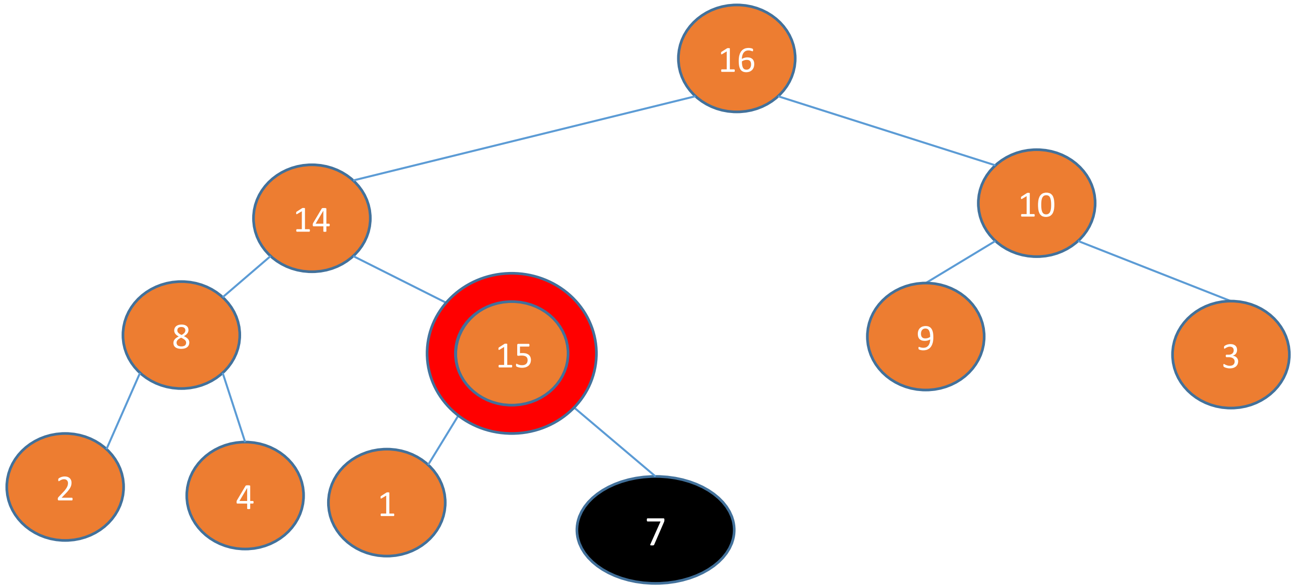


Max heap as a priority queue

Heap Increase Key (A, heapsize, new value)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - **MAX-HEAP-INSERT**
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	-inf

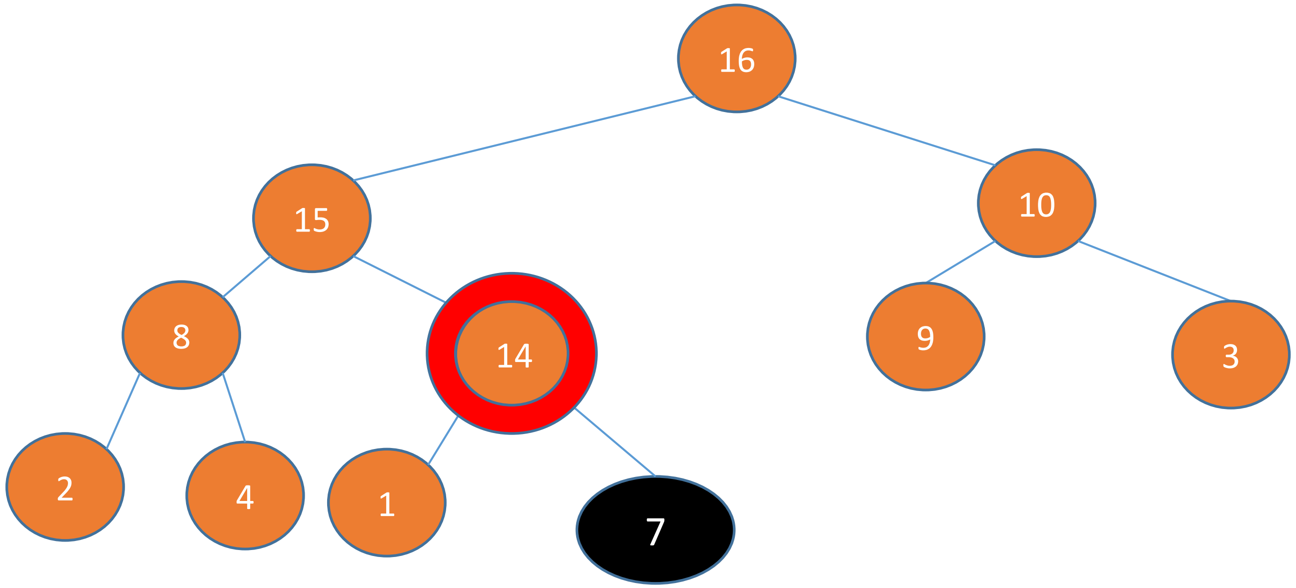


Max heap as a priority queue

Heap Increase Key (A, heapsize, new value)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - **MAX-HEAP-INSERT**
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	-inf

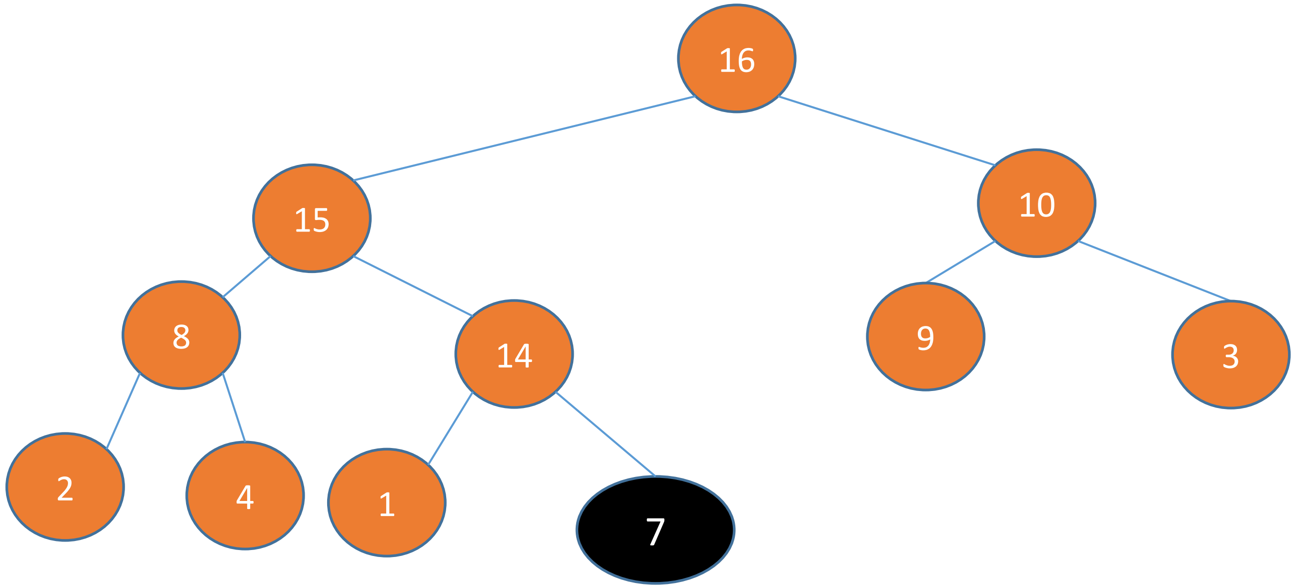


Max heap as a priority queue

Heap Increase Key (A, heapsize, new value)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - **MAX-HEAP-INSERT**
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	-inf

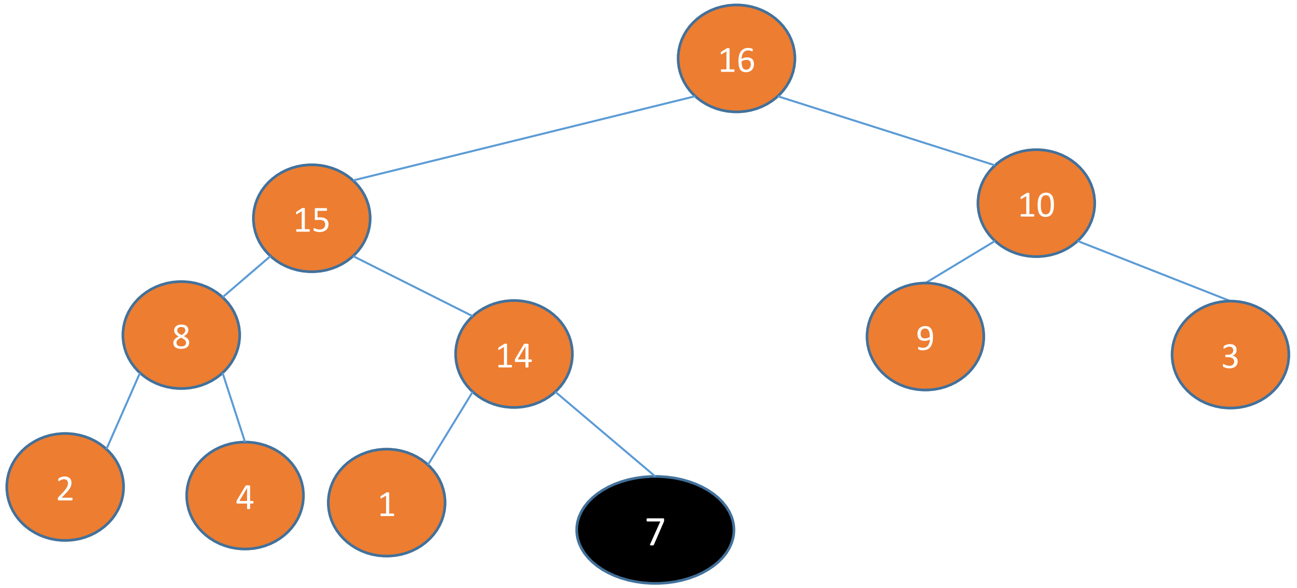


Max heap as a priority queue

Heap Increase Key (A, heapsize, new value)

- **Priority Queue functions**
  - HEAP-MAXIMUM
  - **MAX-HEAP-INSERT**
  - HEAP-EXTRACT-MAX
  - HEAP-INCREASE-KEY

1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	-inf



Time?
Takes O(logn) time

Thank You